

# Supplementary Material

## Meta-Learning Orchestration of Learning Modalities on Resource-Constrained Robots: A Systematic Review

Yeshwanth Guru and Dev Kunwar Singh Chauhan

Department of Mechanical Engineering, Amrita Vishwa Vidyapeetham, Chennai, India

Contents: S1 PRISMA 2020 checklist; S2 search strategy; S3 corrections to the initial list; S4 evidence, edge and signal coding of the 180 works; S5 quality appraisal; S6 study summaries. All tables are generated from the data package (Data Availability of the article).

### S1 PRISMA 2020 checklist

| Item                          | Requirement                                    | Location                                              |
|-------------------------------|------------------------------------------------|-------------------------------------------------------|
| Title                         | Identify the report as a systematic review     | Title                                                 |
| Abstract                      | Structured summary                             | Abstract                                              |
| Rationale                     | Rationale in the context of existing knowledge | Sections I-A, I-E                                     |
| Objectives                    | Explicit research questions                    | Section I-D (RQ1–RQ5)                                 |
| Eligibility criteria          | Inclusion and exclusion criteria               | Section II-A                                          |
| Information sources           | Sources and date last searched                 | Section II-B; S2                                      |
| Search strategy               | Full search strategy                           | S2 (stage-2 strings not logged; concept blocks given) |
| Selection process             | Screening method and reviewers                 | Sections II-A, II-F                                   |
| Data collection process       | Extraction method                              | Section II-C                                          |
| Data items                    | Variables extracted                            | Section II-C; codebook of the data package            |
| Study risk of bias            | Appraisal method                               | Section II-E; S5                                      |
| Effect measures               | Not applicable (no pooled effect)              | –                                                     |
| Synthesis methods             | Narrative synthesis and counts                 | Section II-F                                          |
| Reporting bias assessment     | Discussed as a threat to validity              | Section II-F                                          |
| Certainty assessment          | Evidence levels E1–E6                          | Section II-D                                          |
| Study selection               | Flow of records                                | Section II-B; Fig. 2                                  |
| Study characteristics         | Characteristics of each work                   | S4; data package                                      |
| Risk of bias in studies       | Appraisal results                              | Table II; S5                                          |
| Results of syntheses          | Synthesis by topic and RQ                      | Sections III–XV                                       |
| Discussion                    | Interpretation, limitations, implications      | Sections II-F, XV–XVII                                |
| Registration and protocol     | Registration                                   | Section II (OSF, retrospective)                       |
| Support                       | Funding                                        | Acknowledgment                                        |
| Competing interests           | Declarations                                   | Cover letter                                          |
| Availability of data and code | Data and code                                  | Data Availability; supplementary code                 |

### S2 Search strategy

Stage 1: curated reading list of 108 works, each checked against its arXiv or publisher record (corrections in S3). Stage 2 (September 2026): structured searches of arXiv, the proceedings of CoRL, RSS, ICRA, IROS, NeurIPS, ICML and ICLR, and publisher sites, in ten topic streams, with backward citation chasing from the included reviews; 72 works added. The exact strings and hit counts were not logged; the concept blocks below reconstruct their scope.

```
Block A (agent):      robot* OR embodied OR "mobile manipulat*" OR "vision-language-action" OR VLA
Block B (mechanism):  orchestrat* OR routing OR router OR gating OR "mixture of experts" OR cascade
                      OR "meta-learn*" OR "adaptive computation" OR "early exit" OR "tool use"
Block C (signal):     uncertainty OR confidence OR calibrat* OR conformal OR "failure detection"
                      OR anomaly OR "out-of-distribution"
Block D (constraint): edge OR on-device OR embedded OR "Raspberry Pi" OR Jetson OR "low-cost"
                      OR quantiz* OR "parameter-efficient" OR LoRA OR budget

Streams 1, 5, 6:      A AND C          Streams 3, 8: B AND (C OR D)
Streams 2, 7:         A AND D          Stream 4: A AND ("reinforcement learning" AND (fine-tun* OR residual))
Stream 9:             A AND (shield OR simplex OR "safety filter")
Stream 10:            A AND D AND (benchmark OR evaluation OR "statistical")
```

Stage 3 (4 October 2026): targeted web search on the title topic (five topic and four verification queries). Seven new records: three excluded (talk page; non-embodied agent; LLM-only compute allocation) and four recorded as candidates for full-text screening (uncertainty-based LLM routing, arXiv:2502.11021; a systematic survey of efficient VLAs, arXiv:2510.17111; budgeted skill practice, arXiv:2608.13415; online metareasoning, AAMAS 2026). None was added to the corpus.

## S3 Corrections to the initial list

- Liang 2023 Initial list gave 2022 (arXiv); published at ICRA 2023.
- Yao 2023a Initial list gave 2022 (arXiv); published at ICLR 2023.
- Singh 2023 Also a journal version: *Autonomous Robots* 47(8), 999–1012 (2023), doi:10.1007/s10514-023-10135-3.
- Wang 2024a Initial list gave 2023 (arXiv); TMLR 2024.
- Brohan 2023a Initial list gave 2022 (arXiv); RSS 2023.
- Jiang 2023 Initial list gave 2022 (arXiv); ICML 2023.
- Octo 2024 The RSS proceedings list individual authors; cite them rather than "Octo Model Team".
- Black 2024 Initial list gave arXiv 2024; published at RSS 2025.
- Bousmalis 2023 Initial list gave 2023; TMLR 2024.
- Li 2024a Initial list gave 2023 (arXiv); ICLR 2024.
- Burda 2019 Initial list gave 2018 (arXiv); ICLR 2019.
- Hafner 2023 DreamerV3: the initial list cited the 2023 arXiv version (2301.04104, "Mastering diverse domains through world models"); cite the Nature 2025 version.
- Nagabandi 2019 Initial list gave 2018 (arXiv); ICLR 2019.
- Hospedales 2022 Initial list gave 2021 (online); journal issue 2022.
- Baltrusaitis 2019 Initial list gave 2018 (online); journal issue 2019.
- Schaul 2016 Initial list gave 2015 (arXiv); ICLR 2016.
- Hu 2022 Initial list gave 2021 (arXiv); ICLR 2022.
- DeLange 2022 Initial list gave 2021 (online); journal issue 2022.
- Puigcerver 2024 Initial list gave 2023 (arXiv); ICLR 2024.
- Wang 2024b Full title: "Sparse Diffusion Policy: A Sparse, Reusable, and Flexible Policy for Robot Learning".
- Reuss 2025 Authors and arXiv id verified.
- Huang 2025 arXiv:2503.08564; page numbers from project BibTeX.
- SMP 2026 Authors verified (arXiv, Jan 2026).
- Cadene 2024 Software. A citable paper now exists: Cadene et al. (2026), ICLR 2026, arXiv:2602.22818 (added as a separate entry).
- OXE 2024 Initial list gave 2023 (arXiv); ICRA 2024.
- MoDE (Reuss 2025): full authors Reuss, Pari, Agrawal, Lioutikov; arXiv:2412.12953; ICLR 2025.
  - MoE-LoCo (Huang 2025): authors Runhan Huang, Shaoting Zhu, Yilun Du, Hang Zhao; arXiv:2503.08564; DOI 10.1109/IROS60139.2025.11246585.
  - SMP (arXiv:2601.21251): authors Ce Hao, Xuanran Zhai, Yaohua Liu, Harold Soh.
  - Sparse Diffusion Policy: CoRL 2024, PMLR 270, 649–665; first author Yixiao Wang.
  - DreamerV3: published as "Mastering diverse control tasks through world models", *Nature* 640, 647–653 (2025).
  - $\pi 0$ : published at RSS 2025 (DOI 10.15607/RSS.2025.XXI.010).
  - Octo: cite the RSS 2024 author list (Ghosh, Walke, Pertsch, Black, Mees, Dasari, ...) rather than "Octo Model Team".
  - Shinn 2023 (Reflexion): arXiv lists Edward Berman as an author.
  - Ross 2011 (DAGGER): Gordon, G.J. and Bagnell, J.A.
  - Rakelly 2019 (PEARL): author order Rakelly, Zhou, Quillen, Finn, Levine.
  - McMahan 2017 (FedAvg): first author H. Brendan McMahan.

- Cai 2020 (TinyTL): arXiv title says "Reduce Activations, Not Trainable Parameters"; the NeurIPS title uses "Reduce Memory, Not Parameters". Use the NeurIPS title.
- LeRobot: a citable paper now exists (Cadene et al., ICLR 2026, arXiv:2602.22818). Cite it alongside the software.
- Kairouz 2021: volume and pages not confirmed; Pang 2021: arXiv journal-ref gives 2020 online, 2021 issue.

## S4 Evidence level, edge class and confidence-signal class of the 180 works

Edge class: P Pi-class or embedded CPU; J embedded GPU; W workstation GPU; D datacenter or cloud; U not stated. Signal class: C conformal or calibrated; E epistemic uncertainty; F failure, anomaly or novelty score; V policy variance or entropy; Q routing score or predicted quality; N none. Basis: F full text, A abstract, S standard reference. Reviews are not coded.

### LLM planners and orchestrators (23)

| Work                     | Level | Edge | Signal | Basis | Key result                                                                                                                                  |
|--------------------------|-------|------|--------|-------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Ahn et al. (2022) [1]    | E4    | D    | Q      | A     | Abstract reports successful completion of long-horizon abstract instructions and that grounding is needed; no numbers in abstract.          |
| Huang et al. (2022) [2]  | E4    | D    | F      | A     | Abstract reports substantially improved instruction completion with closed-loop feedback; no numbers in abstract.                           |
| Huang et al. (2022) [3]  | E3    | D    | N      | A     | Abstract: substantially improved executability over LLM baseline; executability–correctness trade-off observed; no numbers given.           |
| Shah et al. (2022) [4]   | E4    | D    | N      | A     | Demonstrated long-horizon language-guided navigation; no numbers in abstract.                                                               |
| Liang et al. (2023) [5]  | E4    | D    | N      | A     | 39.8% of HumanEval problems solved with hierarchical code generation.                                                                       |
| Driess et al. (2023) [6] | E4    | D    | N      | A     | PaLM-E-562B reported state of the art on OK-VQA; positive transfer from joint training.                                                     |
| Yao et al. (2023) [7]    | E3    | D    | N      | A     | +34% (ALFWorld) and +10% (WebShop) absolute success over imitation/RL baselines with 1–2 in-context examples.                               |
| Schick et al. (2023) [8] | E3    | W    | N      | A     | Abstract: substantially improved zero-shot performance, often competitive with much larger models; no numbers given.                        |
| Shinn et al. (2023) [9]  | E3    | U    | N      | A     | 91% pass@1 on HumanEval vs 80% for GPT-4.                                                                                                   |
| Singh et al. (2023) [10] | E4    | D    | N      | A     | State-of-the-art success rates in VirtualHome (per abstract); no numbers in abstract.                                                       |
| Huang et al. (2023) [11] | E4    | D    | Q      | A     | Large-scale study reported; no numbers in abstract.                                                                                         |
| Yu et al. (2023) [12]    | E4    | D    | N      | A     | 90% of tasks solved vs 50% for Code-as-Policies primitive-skill baseline.                                                                   |
| Song et al. (2023) [13]  | E3    | D    | N      | A     | Competitive with full-data baselines using <0.5% of paired training data.                                                                   |
| Yao et al. (2023) [14]   | E2    | D    | Q      | A     | Game of 24: 74% success vs 4% for GPT-4 chain-of-thought.                                                                                   |
| Ren et al. (2023) [15]   | E4    | D    | C      | A     | Statistical task-completion guarantees with reduced help; favourable versus ensembles and prompt-tuning baselines (no numbers in abstract). |
| Chen et al. (2023) [16]  | E3    | W    | N      | A     | Llama2-7B fine-tuned on 500 GPT-4 trajectories: 77% HotpotQA performance increase.                                                          |
| Wang et al. (2024) [17]  | E3    | D    | Q      | A     | 3.3× more unique items, 2.3× longer distances, up to 15.3× faster tech-tree milestones vs prior SOTA.                                       |
| Liang et al. (2024) [18] | E3    | U    | C      | A     | Improved compliance and safety over SOTA LLM planners; tighter conformal bounds with fewer clarifications (no numbers in abstract).         |

| Work                     | Level | Edge | Signal | Basis | Key result                                                                                                              |
|--------------------------|-------|------|--------|-------|-------------------------------------------------------------------------------------------------------------------------|
| Wang et al. (2024) [19]  | E3    | U    | C      | A     | Achieves user-specified success rates with fewer help requests; more efficient than baselines (no numbers in abstract). |
| Dalal et al. (2024) [20] | E3    | D    | N      | F     | >85% success; RS-NutAssembly 96% $\pm$ 8% vs SayCan 23% $\pm$ 21% and TAMP 20% $\pm$ 30%; Kitchen 100% vs baselines 0%. |
| Zeng et al. (2024) [21]  | E3    | W    | N      | A     | AgentLM-70B comparable to GPT-3.5-turbo on unseen agent tasks; general abilities maintained.                            |
| Yao et al. (2024) [22]   | E3    | D    | N      | A     | Agents improve over trials and outperform baselines not using environment gradients (no figures in the abstract).       |
| Zhai et al. (2024) [23]  | E3    | W    | N      | F     | EZPoints 50.0% vs GPT-4V 10.5%; Blackjack 40.2% vs 25.5%; ALFWorld avg 21.7% vs 19.4%; CoT removal hurts.               |

## VLA and robot foundation models (22)

| Work                            | Level | Edge | Signal | Basis | Key result                                                                                                                                            |
|---------------------------------|-------|------|--------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Radford et al. (2021) [24]      | E3    | W    | N      | A     | Zero-shot ImageNet accuracy matching original ResNet-50 without its 1.28M training examples.                                                          |
| Shridhar et al. (2021) [25]     | E4    | W    | N      | A     | Single multi-task policy on 10 sim + 9 real tasks better than or comparable to single-task policies.                                                  |
| Reed et al. (2022) [26]         | E4    | W    | N      | A     | One network performs diverse tasks including real-robot block stacking (no numbers in abstract).                                                      |
| Shridhar et al. (2022) [27]     | E4    | W    | N      | A     | Significantly outperforms image-to-action and 3D ConvNet baselines (no specific numbers in abstract).                                                 |
| Brohan et al. (2023) [28]       | E5    | W    | N      | A     | Promising scaling properties with data size, model size and diversity (no numbers in abstract).                                                       |
| Brohan et al. (2023) [29]       | E4    | D    | N      | A     | Improved generalisation to novel objects and emergent semantic reasoning (6,000 trials).                                                              |
| Jiang et al. (2023) [30]        | E3    | W    | N      | A     | Up to $2.9\times$ zero-shot success over alternatives; $2.7\times$ better with $10\times$ less data.                                                  |
| Kim et al. (2024) [31]          | E5    | W    | N      | A     | +16.5% absolute success over RT-2-X (55B) with $7\times$ fewer parameters; +20.4% over Diffusion Policy in multi-task fine-tuning.                    |
| Ghosh et al. (2024) [32]        | E5    | W    | N      | A     | Effective fine-tuning to new observation/action spaces within a few hours on consumer GPUs (9 platforms).                                             |
| Black et al. (2024) [33]        | E5    | W    | N      | A     | Zero-shot, language-following and fine-tuning capabilities demonstrated (no numbers in abstract).                                                     |
| Bousmalis et al. (2024) [34]    | E5    | D    | N      | A     | Adaptation to new tasks/robots with 100–1,000 examples; zero-shot generalisation reported.                                                            |
| Li et al. (2024) [35]           | E3    | W    | N      | A     | Exceeds state of the art by a large margin on the tested benchmark (no numbers in abstract).                                                          |
| Liu et al. (2024) [36]          | E4    | W    | N      | A     | About $3\times$ faster inference than existing VLAs; policy head $\sim 0.1\%$ of parameters (other figures not verified).                             |
| Shukor et al. (2025) [37]       | E4    | W    | N      | A     | Performance comparable to VLAs about $10\times$ larger; trainable on one GPU, deployable on consumer GPUs or CPUs (abstract gives no further numbers) |
| Wen et al. (2025) [38]          | E4    | W    | N      | A     | Faster and more data-efficient than OpenVLA with comparable or superior performance (no numbers in abstract)                                          |
| Budzianowski et al. (2025) [39] | E2    | U    | N      | A     | 7x inference speed-up from removing autoregression; comparable training characteristics to OpenVLA; improved memory efficiency                        |
| Kim et al. (2025) [40]          | E4    | W    | N      | A     | LIBERO average 76.5% to 97.1%; 26x throughput; up to +15% absolute over $\pi 0$ , RDT-1B, Diffusion Policy, ACT on ALOHA                              |

| Work                            | Level | Edge | Signal | Basis | Key result                                                                                                       |
|---------------------------------|-------|------|--------|-------|------------------------------------------------------------------------------------------------------------------|
| Pertsch et al. (2025) [41]      | E4    | W    | N      | A     | Enables tasks where binning fails; matches diffusion VLAs with up to 5x less training time                       |
| Wang et al. (2025) [42]         | E4    | W    | N      | A     | Matches OpenVLA-OFT; 11.0x less memory; 4.4x lower end-to-end latency                                            |
| Ravichandran et al. (2025) [43] | E4    | U    | N      | A     | Student rises from 10–20% to over 93% of GPT-4o performance with synthetic data only; transfers across platforms |
| Yu et al. (2025) [44]           | R     | –    | –      | A     |                                                                                                                  |
| Zhang et al. (2026) [45]        | E3    | W    | N      | A     | Task success above full-precision baselines; ~70% relative memory saving on quantised components                 |

## Perception and epistemic uncertainty (9)

| Work                                | Level | Edge | Signal | Basis | Key result                                                                                                                                   |
|-------------------------------------|-------|------|--------|-------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Gal and Ghahramani (2016) [46]      | E2    | P    | E      | A     | Considerable improvement in predictive log-likelihood and RMSE over prior methods (no numbers in abstract)                                   |
| Lakshminarayanan et al. (2017) [47] | E2    | P    | E      | A     | Well-calibrated uncertainty, as good as or better than approximate Bayesian NNs; higher uncertainty on OOD examples (no numbers in abstract) |
| Kendall and Gal (2017) [48]         | E2    | W    | E      | A     | New state-of-the-art segmentation and depth-regression results at time of publication (no numbers in abstract)                               |
| Guo et al. (2017) [49]              | E2    | P    | C      | A     | Modern networks poorly calibrated; temperature scaling effective on most datasets (no numbers in abstract)                                   |
| Hendrycks and Gimpel (2017) [50]    | E2    | P    | F      | A     | Baseline effective across all domains, but can be surpassed (no numbers in abstract)                                                         |
| Sensoy et al. (2018) [51]           | E2    | P    | E      | A     | Reported strong OOD detection and adversarial robustness (no numbers in abstract)                                                            |
| Amini et al. (2020) [52]            | E2    | P    | E      | A     | Well-calibrated uncertainty; robustness to adversarial and OOD samples (no numbers in abstract)                                              |
| Gawlikowski et al. (2023) [53]      | R     | –    | –      | A     |                                                                                                                                              |
| Ren et al. (2024) [54]              | E4    | D    | C      | A     | Improved performance and efficiency over uncalibrated and non-VLM baselines (no numbers in abstract)                                         |

## Reinforcement learning and policy variance (17)

| Work                            | Level | Edge | Signal | Basis | Key result                                                                                                                                   |
|---------------------------------|-------|------|--------|-------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Mnih et al. (2015) [55]         | E3    | P    | N      | S     | Human-comparable play on many of 49 games (from standard knowledge; (not stated in the abstract)                                             |
| Osband et al. (2016) [56]       | E3    | P    | E      | A     | Substantially improved learning times and performance across most Atari games (no numbers in abstract)                                       |
| Schulman et al. (2017) [57]     | E3    | P    | N      | A     | Outperforms other online policy-gradient methods; favourable balance of sample complexity, simplicity and wall time (no numbers in abstract) |
| Andrychowicz et al. (2017) [58] | E4    | P    | N      | A     | HER crucial for training success in ablations; simulated policies complete the task on a physical robot (no numbers in abstract)             |

| Work                           | Level | Edge | Signal | Basis | Key result                                                                                                                      |
|--------------------------------|-------|------|--------|-------|---------------------------------------------------------------------------------------------------------------------------------|
| Haarnoja et al. (2018) [59]    | E3    | P    | V      | A     | State-of-the-art performance over prior on- and off-policy methods; stable across seeds (no numbers in abstract)                |
| Fujimoto et al. (2018) [60]    | E3    | P    | N      | A     | Outperforms state of the art in every environment tested (no numbers in abstract)                                               |
| Kalashnikov et al. (2018) [61] | E5    | W    | N      | A     | 96% grasp success on unseen objects; Q-network over 1.2M parameters; emergent regrasping and pre-grasp manipulation             |
| Burda et al. (2019) [62]       | E3    | P    | F      | A     | State of the art on Montezuma’s Revenge; better than average human without demonstrations; occasionally completes level 1       |
| Kumar et al. (2020) [63]       | E3    | P    | N      | A     | Often 2–5x higher final return than prior offline RL methods                                                                    |
| Hafner et al. (2025) [64]      | E3    | W    | N      | A     | Outperforms specialised methods across over 150 tasks; first to collect diamonds in Minecraft from scratch without human data   |
| Yuan et al. (2025) [65]        | E3    | U    | N      | A     | Improves offline-trained policies while preserving smooth motion; avoids erratic pure-RL behaviour (no numbers in abstract)     |
| Ankile et al. (2025) [66]      | E3    | U    | N      | A     | Improves reliability over chunked BC (no figures in the abstract)                                                               |
| Wagenmaker et al. (2025) [67]  | E4    | U    | N      | A     | Highly sample-efficient real-world policy improvement without modifying base weights (no numbers in abstract)                   |
| Sobol Mark et al. (2025) [68]  | E4    | W    | Q      | A     | Up to 2x performance and sample-efficiency gains; OpenVLA real-world success 40% to 70% in 40 minutes                           |
| Chen et al. (2025) [69]        | E4    | W    | N      | A     | 96.3% average success within 45–90 min online; 144% improvement in success rate and 1.9x shorter episodes vs supervised methods |
| Lu et al. (2025) [70]          | E3    | D    | Q      | A     | +4.5% over strongest fine-tuned baseline; matches $\pi$ 0-FAST                                                                  |
| Li et al. (2026) [71]          | E4    | D    | N      | A     | SoTA on LIBERO; outperforms $\pi$ 0 on RoboTwin; surpasses SFT in real world (no numbers in abstract); ‘pushcut’ phenomenon     |

## Imitation learning and behavioral fidelity (10)

| Work                        | Level | Edge | Signal | Basis | Key result                                                                                                                                                         |
|-----------------------------|-------|------|--------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ross et al. (2011) [72]     | E2    | P    | N      | A     | Outperforms prior approaches (no numbers in abstract); theoretical guarantee under induced distribution                                                            |
| Ho and Ermon (2016) [73]    | E3    | P    | N      | A     | Significant gains over prior model-free imitation methods on complex high-dimensional environments (no figures stated in abstract).                                |
| Duan et al. (2017) [74]     | E3    | P    | N      | A     | Qualitative claim of generalisation to unseen conditions and tasks via soft attention; no numbers in abstract.                                                     |
| Osa et al. (2018) [75]      | R     | –    | –      | A     |                                                                                                                                                                    |
| Lynch et al. (2019) [76]    | E3    | P    | N      | A     | Play covers ~4x more interaction space than demonstrations; Play-LMP substantially outperforms per-task expert policies on 18 tasks, more robust to perturbations. |
| Florence et al. (2021) [77] | E4    | W    | N      | A     | Often outperforms explicit MSE/MDN BC; competitive with or better than offline RL on D4RL human-expert tasks without rewards; real tasks with 1 mm precision.      |

| Work                    | Level | Edge | Signal | Basis | Key result                                                                                                                                        |
|-------------------------|-------|------|--------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Jang et al. (2021) [78] | E5    | W    | N      | A     | 24 unseen tasks performed with 44% average success without robot demonstrations for those tasks.                                                  |
| Chi et al. (2023) [79]  | E4    | W    | N      | A     | Consistent outperformance of prior state of the art, average improvement 46.9%.                                                                   |
| Zhao et al. (2023) [80] | E4    | W    | N      | A     | 80–90% success on six tasks with ten minutes of demonstrations.                                                                                   |
| Wu et al. (2024) [81]   | E3    | W    | C      | F     | PerAct 0.382→0.414, RVT 0.602→0.623, CLIPort mean reward 0.803→0.833; stack-blocks with distractors 79.63%→97.11%; assembling kits 48.32%→55.44%. |

## Fault detection, adaptation and recovery (16)

| Work                              | Level | Edge | Signal | Basis | Key result                                                                                                                               |
|-----------------------------------|-------|------|--------|-------|------------------------------------------------------------------------------------------------------------------------------------------|
| Bongard et al. (2006) [82]        | E4    | U    | E      | S     | Robot inferred its morphology and generated compensatory locomotion after damage (no figures reported here).                             |
| Cully et al. (2015) [83]          | E4    | P    | E      | S     | Compensatory behaviours found within a small number of trials and short time (exact figures not verified).                               |
| Tobin et al. (2017) [84]          | E4    | W    | N      | A     | Real-world localisation accuracy 1.5 cm, robust to distractors and partial occlusion; successful grasping in clutter.                    |
| Khalastchi and Kalech (2018) [85] | R     | –    | –      | S     |                                                                                                                                          |
| Nagabandi et al. (2019) [86]      | E4    | P    | N      | A     | Rapid online adaptation demonstrated qualitatively in simulation and on the real millirobot (no numbers in abstract).                    |
| Pang et al. (2021) [87]           | R     | –    | –      | A     |                                                                                                                                          |
| Kumar et al. (2021) [88]          | E4    | P    | N      | A     | Adaptation in fractions of a second; state-of-the-art performance in real and simulated experiments (no numbers in abstract).            |
| Liu et al. (2023) [89]            | E4    | D    | F      | A     | Informative failure explanations that assist successful correction planning (no numbers in abstract).                                    |
| Agia et al. (2024) [90]           | E4    | D    | F      | A     | Detects 18% more failures than either detector alone; significantly outperforms baselines.                                               |
| Sinha et al. (2024) [91]          | E3    | D    | F      | A     | Fast classifier outperforms autoregressive GPT reasoning even with small language models (no numbers in abstract).                       |
| Shi et al. (2024) [92]            | E4    | W    | N      | A     | Significant performance improvement without extra teleoperation (no numbers in abstract).                                                |
| Duan et al. (2025) [93]           | E4    | W    | F      | A     | +10.3% over GPT-4o ICL; +35.3% over average of six models; +21.4% average task success in downstream frameworks vs GPT-4 models.         |
| Ye et al. (2025) [94]             | E4    | W    | F      | A     | +34.1% failure-analysis accuracy vs GPT-4o; 29.1% relative improvement across four real tasks; lower latency than GPT-4o.                |
| Römer et al. (2025) [95]          | E4    | U    | F      | A     | Better separation of failures from benign OOD and earlier, more accurate prediction than existing methods (no numbers in abstract).      |
| Xu et al. (2025) [96]             | E4    | U    | F      | A     | More accurate and faster detection than SOTA baselines; learned signals, particularly flow-based density, mostly consistently effective. |
| Gu et al. (2025) [97]             | E4    | W    | F      | A     | State-of-the-art failure detection; best accuracy vs detection-time trade-off (no numbers in abstract).                                  |

## Meta-learning (10)

| Work                            | Level | Edge | Signal | Basis | Key result                                                                                                                                                    |
|---------------------------------|-------|------|--------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Duan et al. (2016) [98]         | E3    | P    | N      | S     | Learned agents adapt within few episodes; comparable to hand-designed algorithms on small problems (no numbers cited).                                        |
| Andrychowicz et al. (2016) [99] | E2    | U    | N      | A     | Outperforms hand-designed optimisers on trained task families and generalises to similar tasks (no numbers in abstract).                                      |
| Finn et al. (2017) [100]        | E3    | P    | N      | A     | State of the art on two few-shot classification benchmarks; good few-shot regression; faster RL fine-tuning (no numbers in abstract).                         |
| Snell et al. (2017) [101]       | E3    | P    | N      | A     | Excellent few-shot results; state of the art zero-shot on CU-Birds (no numbers in abstract).                                                                  |
| Nichol et al. (2018) [102]      | E2    | P    | N      | A     | First-order algorithms perform well on few-shot classification benchmarks (no numbers in abstract).                                                           |
| Yu et al. (2019) [103]          | E3    | W    | N      | A     | Single tasks learnable, but algorithms struggle with multiple tasks even with as few as ten training tasks.                                                   |
| Rakelly et al. (2019) [104]     | E3    | P    | E      | A     | 20–100x better sample efficiency and higher asymptotic performance than prior meta-RL algorithms.                                                             |
| Zintgraf et al. (2020) [105]    | E3    | P    | E      | A     | Structured uncertainty-driven exploration in grid-world; higher online return than existing methods on MuJoCo (no numbers in abstract).                       |
| Hospedales et al. (2022) [106]  | R     | –    | –      | A     |                                                                                                                                                               |
| Zhu et al. (2025) [107]         | E4    | W    | N      | F     | Forward transfer 0.86 vs 0.71 (OBJECT), 0.69 vs 0.57 (SPATIAL), 0.58 vs 0.52 (GOAL) vs LoRA; real robot 84.0% vs 60.0% (20 demos), 48.0% vs 34.0% (10 demos). |

## Uncertainty fusion and calibration (13)

| Work                             | Level | Edge | Signal | Basis | Key result                                                                                                 |
|----------------------------------|-------|------|--------|-------|------------------------------------------------------------------------------------------------------------|
| McMahan et al. (2017) [108]      | E3    | P    | N      | A     | Robust to non-IID and unbalanced data; 10–100x fewer communication rounds than synchronised SGD.           |
| Kendall et al. (2018) [109]      | E3    | W    | E      | A     | Learnt weighting outperforms separately trained single-task models (abstract; no figures in the abstract). |
| Baltrušaitis et al. (2019) [110] | R     | –    | –      | A     |                                                                                                            |
| Li et al. (2020) [111]           | E2    | P    | N      | A     | +22% average absolute test accuracy over FedAvg in highly heterogeneous settings.                          |
| Kairouz et al. (2021) [112]      | R     | –    | –      | A     |                                                                                                            |
| Qi et al. (2021) [113]           | R     | –    | –      | A     |                                                                                                            |
| Han et al. (2021) [114]          | E3    | P    | E      | A     | Improved accuracy, reliability and robustness reported qualitatively in abstract.                          |
| Han et al. (2022) [115]          | E3    | P    | E      | A     | Improved accuracy, robustness and trustworthiness claimed; no figures in the abstract.                     |
| Han et al. (2022) [116]          | E3    | P    | Q      | A     | Reported superior performance and trustworthiness versus prior methods (no figures in the abstract).       |
| Lindemann et al. (2023) [117]    | E3    | P    | C      | A     | Valid probabilistic safety guarantees demonstrated; no numeric results in the abstract.                    |
| Zollo and Zemel (2025) [118]     | E3    | W    | C      | A     | Miscalibration observed and reduced by proposed remedies (no figures in the abstract).                     |
| Römer et al. (2026) [119]        | E3    | W    | E      | A     | Better-calibrated uncertainty; strong failure detection; at least 22% fewer samples than baselines.        |

| Work                     | Level | Edge | Signal | Basis | Key result                                                                                                              |
|--------------------------|-------|------|--------|-------|-------------------------------------------------------------------------------------------------------------------------|
| Yuan et al. (2026) [120] | E4    | W    | C      | A     | Disentangles confident, ambiguous and incapable cases; fewer interventions than baselines (no figures in the abstract). |

## On-device adaptation and continual learning (23)

| Work                               | Level | Edge | Signal | Basis | Key result                                                                                                                                     |
|------------------------------------|-------|------|--------|-------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Schaul et al. (2016) [121]         | E3    | W    | N      | A     | Outperforms uniform-replay DQN on 41 of 49 games; state of the art at the time.                                                                |
| Rusu et al. (2016) [122]           | E3    | W    | N      | A     | Outperforms pretrain-and-finetune baselines; transfer at sensory and control layers.                                                           |
| Kirkpatrick et al. (2017) [123]    | E3    | P    | N      | A     | Retains performance on earlier tasks (no figures in the abstract).                                                                             |
| Lopez-Paz and Ranzato (2017) [124] | E3    | P    | N      | A     | Strong performance versus state of the art (no figures in the abstract).                                                                       |
| Rebuffi et al. (2017) [125]        | E3    | P    | N      | A     | Learns many classes incrementally where other strategies fail (no figures in the abstract).                                                    |
| Houlsby et al. (2019) [126]        | E3    | W    | N      | A     | Within 0.4% of full fine-tuning on GLUE with 3.6% added parameters per task.                                                                   |
| Rolnick et al. (2019) [127]        | E3    | U    | N      | A     | Substantially reduced forgetting; matches task-aware methods; random-discard buffer nearly as good as unbounded.                               |
| Cai et al. (2020) [128]            | E3    | P    | N      | A     | Up to 6.5x memory saving; up to 34.1% over last-layer tuning; 7.3–12.9x saving vs full Inception-V3 fine-tuning.                               |
| Lin et al. (2020) [129]            | E3    | P    | N      | A     | >70% ImageNet top-1 on MCU; 4.8x memory and 1.7–3.3x speed gains; 3.5x less SRAM, 5.7x less Flash.                                             |
| Li and Liang (2021) [130]          | E3    | W    | N      | A     | 0.1% parameters; comparable in full data, better in low data and on unseen topics.                                                             |
| Hu et al. (2022) [131]             | E3    | D    | N      | A     | 10,000x fewer trainable parameters and 3x less GPU memory than full GPT-3 fine-tuning; on-par or better quality; no added latency.             |
| Lin et al. (2022) [132]            | E3    | P    | N      | A     | Training within 256 KB SRAM and 1 MB Flash; <1/1000 memory of PyTorch/TensorFlow; matches accuracy on VWW.                                     |
| Liu et al. (2022) [133]            | E3    | W    | N      | A     | Super-human on RAFT; +6% absolute over prior state of the art; lower compute than ICL.                                                         |
| De Lange et al. (2022) [134]       | R     | –    | –      | F     |                                                                                                                                                |
| Dettmers et al. (2023) [135]       | E3    | W    | N      | A     | 65B model fine-tuned on one 48 GB GPU; Guanaco reaches 99.3% of ChatGPT on Vicuna after 24 h on one GPU.                                       |
| Liu et al. (2023) [136]            | E4    | W    | N      | A     | +8% (sim) and +27% (real) success over SOTA; 2x faster convergence; 85% memory reduction.                                                      |
| Liu et al. (2023) [137]            | E3    | W    | N      | A     | Sequential fine-tuning beats lifelong methods in forward transfer; no encoder dominates; supervised pretraining can hurt.                      |
| Zhu et al. (2023) [138]            | S     | J    | N      | A     | Up to 15x speed-up vs TensorFlow on Raspberry Pi; 5.6x memory saving on Jetson Orin; LLaMA-2-7B fine-tuning at 550 tokens/s (7.9x vs PyTorch). |
| Malladi et al. (2023) [139]        | E3    | W    | N      | A     | 30B model on one A100 80GB; up to 12x memory reduction; up to 2x fewer GPU-hours; comparable to backprop.                                      |
| Liu et al. (2024) [140]            | E3    | W    | N      | A     | LoRA best post-adaptation performance with 1% of full fine-tuning parameters; no catastrophic forgetting.                                      |

| Work                      | Level | Edge | Signal | Basis | Key result                                                                                                              |
|---------------------------|-------|------|--------|-------|-------------------------------------------------------------------------------------------------------------------------|
| Wan et al. (2024) [141]   | E4    | W    | N      | A     | Outperforms state-of-the-art baselines by over 11% in success rate.                                                     |
| Li et al. (2025) [142]    | E3    | W    | N      | A     | Enables fine-tuning on CPU-only devices; reduced convergence time and memory (no figures in the abstract).              |
| Römer et al. (2026) [143] | E3    | W    | Q      | A     | Strong new-task performance with mitigated forgetting; surpasses exemplar-based baselines (no figures in the abstract). |

## Adaptive computation, mixture of experts and routing (20)

| Work                           | Level | Edge | Signal | Basis | Key result                                                                                                                                                                                                              |
|--------------------------------|-------|------|--------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jacobs et al. (1991) [144]     | E2    | P    | Q      | A     | Experts specialise on input subsets (no figures in the abstract).                                                                                                                                                       |
| Bengio et al. (2015) [145]     | E3    | P    | N      | F     | Abstract reports improved computation speed without loss of approximation quality; speed-ups larger on CPU than GPU (per the full text; exact figures not quoted here).                                                 |
| Graves (2016) [146]            | E3    | P    | Q      | F     | Dramatic gains on synthetic tasks; little gain on Wikipedia modelling but interpretable allocation of steps at word/punctuation boundaries.                                                                             |
| Shazeer et al. (2017) [147]    | E3    | D    | Q      | A     | >1000x capacity increase with minor efficiency loss; significantly better than state of the art at lower computational cost (abstract).                                                                                 |
| Rosenbaum et al. (2018) [148]  | E3    | P    | Q      | A     | Significant accuracy gains, sharper convergence; ~constant per-task training cost; cross-stitch performance with 85% less training time on CIFAR-100 (20 tasks).                                                        |
| Riquelme et al. (2021) [149]   | E3    | D    | Q      | A     | Matches SOTA with as little as half the inference compute; 15B-parameter model reaches 90.35% on ImageNet.                                                                                                              |
| Hoque et al. (2021) [150]      | E4    | P    | F      | A     | 100% execution-time success in sim and physical tasks; +58% human and +80% robot performance vs next-best algorithm in user study.                                                                                      |
| Fedus et al. (2022) [151]      | E3    | D    | Q      | A     | Up to 7x pre-training speed-up vs T5-Base/Large; 4x speed-up over T5-XXL at trillion-parameter scale.                                                                                                                   |
| Puigcerver et al. (2024) [152] | E3    | D    | Q      | A     | Soft MoE Huge/14 (128 experts, 16 MoE layers): >40x parameters of ViT-H/14 with only 2% more inference time and better quality.                                                                                         |
| Wang et al. (2024) [153]       | E4    | W    | Q      | F     | 0.76 avg success on MimicGen with 53.3M active params (vs 52.6M single-task); 0.94 retained success in continual learning at 9.2M active params; 0.80 transfer success with 0.1M trainable params vs 0.70 from scratch. |
| Chen et al. (2024) [154]       | E3    | D    | Q      | A     | Matches best single LLM (GPT-4) with up to 98% cost reduction, or +4% accuracy over GPT-4 at equal cost.                                                                                                                |
| Ding et al. (2024) [155]       | E3    | D    | Q      | A     | Up to 40% fewer large-model calls with no quality drop; router ~0.036 s vs 0.46–14.61 s LLM latency (full text).                                                                                                        |
| Aggarwal et al. (2024) [156]   | E3    | D    | Q      | A     | Over 50% reduction in computational cost for comparable performance; consistently beats strong baselines.                                                                                                               |
| Yue et al. (2024) [157]        | E3    | W    | N      | A     | 5.2–6.5x reduction in LLM computational cost and 2–6x reduction in LLM GPU memory without performance loss.                                                                                                             |

| Work                      | Level | Edge | Signal | Basis | Key result                                                                                                                                                           |
|---------------------------|-------|------|--------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reuss et al. (2025) [158] | E3    | W    | Q      | A     | −40% active parameters, −90% inference cost; CALVIN ABC 4.01, LIBERO-90 0.95; +57% average over diffusion baselines with 90% fewer FLOPs.                            |
| Huang et al. (2025) [159] | E4    | J    | Q      | F     | 87.9% mixed-terrain success in simulation; successful real deployment across all 9 tasks; reduced gradient conflict.                                                 |
| Ong et al. (2025) [160]   | E3    | D    | Q      | A     | Cost reductions of over 2x in some cases without quality loss; routers transfer when model pairs change.                                                             |
| Hao et al. (2026) [161]   | E4    | W    | Q      | A     | Higher success rates and markedly lower inference cost than large diffusion baselines (no figures in abstract).                                                      |
| Liu et al. (2026) [162]   | E3    | U    | N      | F     | >60% lower LLM inference time and 50–75% fewer tokens; success e.g. 82.7±1.7% vs 84.0% (navigation), 76.4±1.9% vs 78.5% (inspection), 69.5±2.1% vs 71.2% (delivery). |
| Zhang et al. (2026) [163] | E4    | W    | Q      | A     | +8% mean success rate across ten tasks; up to 5.6x lower computational cost than standard LLMs.                                                                      |

## Platforms, benchmarks and evaluation (11)

| Work                            | Level | Edge | Signal | Basis | Key result                                                                                                                                                             |
|---------------------------------|-------|------|--------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| James et al. (2020) [164]       | S     | W    | N      | A     | 100 tasks with unlimited demonstrations; no performance numbers in abstract.                                                                                           |
| Mandlekar et al. (2021) [165]   | E4    | W    | N      | A     | Lessons on design sensitivity, demonstration quality and stopping criteria; no numbers in abstract.                                                                    |
| Mees et al. (2022) [166]        | S     | W    | N      | A     | Multi-context imitation baseline performs poorly; no numbers in abstract.                                                                                              |
| Kemp et al. (2022) [167]        | E4    | W    | N      | A     | Design justified analytically and by home deployments; no numbers in abstract.                                                                                         |
| Fu et al. (2024) [168]          | E4    | W    | N      | A     | Co-training increases success rates by up to 90% with 50 demonstrations per task.                                                                                      |
| Cadene et al. (2024) [169]      | S     | W    | N      | F     | Software release; no experimental results in the repository description.                                                                                               |
| Collaboration (2024) [170]      | E5    | D    | N      | A     | RT-X shows positive transfer, improving multiple robots’ capabilities.                                                                                                 |
| Kress-Gazit et al. (2024) [171] | E4    | U    | N      | A     | Best-practice guidelines; no quantitative results in abstract.                                                                                                         |
| Snyder et al. (2025) [172]      | E4    | U    | N      | A     | Up to 32% fewer evaluation trials than state-of-the-art baselines; >160 simulation rollouts saved in a multi-task comparison.                                          |
| Team et al. (2025) [173]        | E5    | W    | N      | A     | Multitask pretraining improves success, robustness and data efficiency; performance scales predictably with pretraining (no figures in the abstract).                  |
| Cadene et al. (2026) [174]      | S     | W    | N      | F     | >16,000 datasets from >2,200 contributors; ACT ~5 ms / 211 MB vs $\pi 0$ ~209 ms / 13.3 GB on RTX 4090; async 73.3% vs sync 78.3% success with 19 vs 9 cubes per 60 s. |

## Safe learning (6)

| Work                              | Level | Edge | Signal | Basis | Key result |
|-----------------------------------|-------|------|--------|-------|------------|
| García and Fernández (2015) [175] | R     | –    | –      | A     |            |

| Work                          | Level | Edge | Signal | Basis | Key result                                                                                          |
|-------------------------------|-------|------|--------|-------|-----------------------------------------------------------------------------------------------------|
| Alshiekh et al. (2018) [176]  | E2    | P    | N      | A     | Safety enforced while preserving convergence under stated conditions; no numbers in abstract.       |
| Phan et al. (2020) [177]      | E3    | P    | N      | A     | Safety guarantees with limited performance loss; no numbers in the abstract.                        |
| Brunke et al. (2022) [178]    | R     | –    | –      | A     | Unified framework and open challenges; call for physics-based benchmarks.                           |
| Wabersich et al. (2023) [179] | R     | –    | –      | F     | Unified treatment of three filter classes and comparative example.                                  |
| Hsu et al. (2024) [180]       | R     | –    | –      | A     | Unified modular view; directions for scalable synthesis, robust monitoring, efficient intervention. |

## S5 Quality appraisal of the 161 primary studies

Q1 method specification; Q2 baseline comparison; Q3 physical validation; Q4 compute reporting; Q5 quantitative evaluation; Q6 code or model release; Q7 confidence signal. M met, P partly met, N not met, ? not assessable without the full text. Coded by the rules of the codebook from the extraction fields; to be confirmed against the full texts.

| Work                            | Level | Q1 | Q2 | Q3 | Q4 | Q5 | Q6 | Q7 |
|---------------------------------|-------|----|----|----|----|----|----|----|
| Ahn et al. (2022) [1]           | E4    | M  | ?  | M  | N  | ?  | ?  | P  |
| Huang et al. (2022) [2]         | E4    | M  | P  | M  | N  | ?  | ?  | M  |
| Huang et al. (2022) [3]         | E3    | M  | P  | N  | N  | ?  | ?  | N  |
| Shah et al. (2022) [4]          | E4    | M  | ?  | M  | N  | ?  | ?  | N  |
| Liang et al. (2023) [5]         | E4    | P  | N  | M  | N  | M  | ?  | N  |
| Driess et al. (2023) [6]        | E4    | M  | N  | M  | M  | P  | ?  | N  |
| Yao et al. (2023) [7]           | E3    | P  | M  | N  | N  | M  | ?  | N  |
| Schick et al. (2023) [8]        | E3    | M  | P  | N  | N  | ?  | ?  | N  |
| Shinn et al. (2023) [9]         | E3    | M  | M  | N  | P  | M  | ?  | N  |
| Singh et al. (2023) [10]        | E4    | P  | ?  | M  | N  | ?  | ?  | N  |
| Huang et al. (2023) [11]        | E4    | M  | ?  | M  | N  | ?  | ?  | P  |
| Yu et al. (2023) [12]           | E4    | M  | M  | M  | P  | M  | ?  | N  |
| Song et al. (2023) [13]         | E3    | P  | M  | N  | N  | M  | ?  | N  |
| Yao et al. (2023) [14]          | E2    | P  | M  | N  | N  | M  | ?  | P  |
| Ren et al. (2023) [15]          | E4    | M  | P  | M  | N  | ?  | ?  | M  |
| Chen et al. (2023) [16]         | E3    | M  | N  | N  | M  | M  | M  | N  |
| Wang et al. (2024) [17]         | E3    | M  | M  | N  | N  | M  | ?  | P  |
| Liang et al. (2024) [18]        | E3    | M  | P  | N  | N  | ?  | ?  | M  |
| Wang et al. (2024) [19]         | E3    | M  | P  | N  | P  | ?  | ?  | M  |
| Dalal et al. (2024) [20]        | E3    | P  | M  | N  | N  | M  | ?  | N  |
| Zeng et al. (2024) [21]         | E3    | P  | N  | N  | M  | P  | ?  | N  |
| Yao et al. (2024) [22]          | E3    | M  | P  | N  | N  | P  | ?  | N  |
| Zhai et al. (2024) [23]         | E3    | M  | M  | N  | M  | M  | ?  | N  |
| Radford et al. (2021) [24]      | E3    | P  | N  | N  | N  | P  | ?  | N  |
| Shridhar et al. (2021) [25]     | E4    | P  | P  | M  | N  | P  | ?  | N  |
| Reed et al. (2022) [26]         | E4    | P  | ?  | M  | N  | ?  | ?  | N  |
| Shridhar et al. (2022) [27]     | E4    | M  | P  | M  | N  | P  | ?  | N  |
| Brohan et al. (2023) [28]       | E5    | M  | ?  | M  | N  | ?  | ?  | N  |
| Brohan et al. (2023) [29]       | E4    | M  | P  | M  | N  | P  | ?  | N  |
| Jiang et al. (2023) [30]        | E3    | P  | M  | N  | M  | M  | ?  | N  |
| Kim et al. (2024) [31]          | E5    | M  | M  | M  | M  | M  | M  | N  |
| Ghosh et al. (2024) [32]        | E5    | M  | N  | M  | N  | P  | M  | N  |
| Black et al. (2024) [33]        | E5    | M  | ?  | M  | N  | ?  | ?  | N  |
| Bousmalis et al. (2024) [34]    | E5    | M  | N  | M  | N  | P  | ?  | N  |
| Li et al. (2024) [35]           | E3    | M  | P  | N  | N  | ?  | ?  | N  |
| Liu et al. (2024) [36]          | E4    | M  | M  | M  | N  | M  | ?  | N  |
| Shukor et al. (2025) [37]       | E4    | M  | N  | M  | N  | M  | ?  | N  |
| Wen et al. (2025) [38]          | E4    | M  | P  | M  | P  | ?  | ?  | N  |
| Budzianowski et al. (2025) [39] | E2    | M  | M  | N  | M  | M  | ?  | N  |

| Work                                | Level | Q1 | Q2 | Q3 | Q4 | Q5 | Q6 | Q7 |
|-------------------------------------|-------|----|----|----|----|----|----|----|
| Kim et al. (2025) [40]              | E4    | M  | M  | M  | M  | M  | ?  | N  |
| Pertsch et al. (2025) [41]          | E4    | P  | N  | M  | M  | M  | ?  | N  |
| Wang et al. (2025) [42]             | E4    | M  | N  | M  | M  | M  | ?  | N  |
| Ravichandran et al. (2025) [43]     | E4    | M  | M  | M  | M  | M  | ?  | N  |
| Zhang et al. (2026) [45]            | E3    | M  | M  | N  | M  | M  | ?  | N  |
| Gal and Ghahramani (2016) [46]      | E2    | M  | P  | N  | N  | ?  | ?  | M  |
| Lakshminarayanan et al. (2017) [47] | E2    | P  | P  | N  | N  | ?  | ?  | M  |
| Kendall and Gal (2017) [48]         | E2    | M  | ?  | N  | N  | ?  | ?  | M  |
| Guo et al. (2017) [49]              | E2    | M  | ?  | N  | N  | ?  | ?  | M  |
| Hendrycks and Gimpel (2017) [50]    | E2    | P  | P  | N  | N  | ?  | ?  | M  |
| Sensoy et al. (2018) [51]           | E2    | P  | ?  | N  | N  | ?  | ?  | M  |
| Amini et al. (2020) [52]            | E2    | P  | ?  | N  | N  | ?  | ?  | M  |
| Ren et al. (2024) [54]              | E4    | P  | P  | M  | P  | ?  | ?  | M  |
| Mnih et al. (2015) [55]             | E3    | P  | N  | N  | N  | P  | ?  | N  |
| Osband et al. (2016) [56]           | E3    | M  | P  | N  | N  | ?  | ?  | M  |
| Schulman et al. (2017) [57]         | E3    | P  | P  | N  | N  | ?  | ?  | N  |
| Andrychowicz et al. (2017) [58]     | E4    | M  | ?  | M  | N  | ?  | ?  | N  |
| Haarnoja et al. (2018) [59]         | E3    | P  | P  | N  | N  | ?  | ?  | M  |
| Fujimoto et al. (2018) [60]         | E3    | P  | P  | N  | N  | ?  | ?  | N  |
| Kalashnikov et al. (2018) [61]      | E5    | P  | M  | M  | M  | M  | ?  | N  |
| Burda et al. (2019) [62]            | E3    | P  | P  | N  | N  | P  | ?  | M  |
| Kumar et al. (2020) [63]            | E3    | P  | M  | N  | N  | M  | ?  | N  |
| Hafner et al. (2025) [64]           | E3    | M  | P  | N  | N  | P  | ?  | N  |
| Yuan et al. (2025) [65]             | E3    | M  | P  | N  | N  | ?  | ?  | N  |
| Ankile et al. (2025) [66]           | E3    | P  | P  | N  | N  | ?  | ?  | N  |
| Wagenmaker et al. (2025) [67]       | E4    | M  | P  | M  | P  | ?  | ?  | N  |
| Sobol Mark et al. (2025) [68]       | E4    | M  | N  | M  | M  | M  | ?  | P  |
| Chen et al. (2025) [69]             | E4    | M  | M  | M  | N  | M  | ?  | N  |
| Lu et al. (2025) [70]               | E3    | P  | M  | N  | M  | M  | ?  | P  |
| Li et al. (2026) [71]               | E4    | P  | P  | M  | N  | ?  | ?  | N  |
| Ross et al. (2011) [72]             | E2    | P  | P  | N  | N  | ?  | ?  | N  |
| Ho and Ermon (2016) [73]            | E3    | M  | P  | N  | N  | P  | ?  | N  |
| Duan et al. (2017) [74]             | E3    | M  | ?  | N  | N  | ?  | ?  | N  |
| Lynch et al. (2019) [76]            | E3    | M  | M  | N  | N  | M  | ?  | N  |
| Florence et al. (2021) [77]         | E4    | M  | P  | M  | N  | P  | ?  | N  |
| Jang et al. (2021) [78]             | E5    | M  | N  | M  | N  | M  | ?  | N  |
| Chi et al. (2023) [79]              | E4    | M  | M  | M  | N  | M  | ?  | N  |
| Zhao et al. (2023) [80]             | E4    | M  | N  | M  | N  | M  | ?  | N  |
| Wu et al. (2024) [81]               | E3    | M  | N  | N  | N  | M  | ?  | M  |
| Bongard et al. (2006) [82]          | E4    | M  | N  | M  | N  | P  | ?  | M  |
| Cully et al. (2015) [83]            | E4    | M  | N  | M  | N  | P  | ?  | M  |
| Tobin et al. (2017) [84]            | E4    | M  | N  | M  | N  | P  | ?  | N  |
| Nagabandi et al. (2019) [86]        | E4    | M  | ?  | M  | N  | ?  | ?  | N  |
| Kumar et al. (2021) [88]            | E4    | M  | ?  | P  | N  | ?  | ?  | N  |
| Liu et al. (2023) [89]              | E4    | M  | ?  | M  | N  | ?  | ?  | M  |
| Agia et al. (2024) [90]             | E4    | M  | M  | M  | N  | M  | ?  | M  |
| Sinha et al. (2024) [91]            | E3    | M  | P  | N  | N  | ?  | ?  | M  |
| Shi et al. (2024) [92]              | E4    | M  | P  | M  | P  | ?  | ?  | N  |
| Duan et al. (2025) [93]             | E4    | M  | M  | M  | N  | M  | M  | M  |
| Ye et al. (2025) [94]               | E4    | M  | M  | M  | P  | M  | ?  | M  |
| Römer et al. (2025) [95]            | E4    | M  | P  | M  | N  | ?  | ?  | M  |
| Xu et al. (2025) [96]               | E4    | M  | P  | M  | N  | ?  | ?  | M  |
| Gu et al. (2025) [97]               | E4    | M  | P  | M  | N  | ?  | ?  | M  |
| Duan et al. (2016) [98]             | E3    | M  | ?  | N  | N  | ?  | ?  | N  |
| Andrychowicz et al. (2016) [99]     | E2    | M  | P  | N  | N  | ?  | ?  | N  |
| Finn et al. (2017) [100]            | E3    | M  | ?  | N  | N  | ?  | ?  | N  |
| Snell et al. (2017) [101]           | E3    | M  | ?  | N  | N  | ?  | ?  | N  |
| Nichol et al. (2018) [102]          | E2    | M  | ?  | N  | N  | ?  | ?  | N  |
| Yu et al. (2019) [103]              | E3    | M  | N  | N  | N  | P  | M  | N  |

| Work                               | Level | Q1 | Q2 | Q3 | Q4 | Q5 | Q6 | Q7 |
|------------------------------------|-------|----|----|----|----|----|----|----|
| Rakelly et al. (2019) [104]        | E3    | M  | M  | N  | P  | M  | ?  | M  |
| Zintgraf et al. (2020) [105]       | E3    | M  | P  | N  | N  | ?  | ?  | M  |
| Zhu et al. (2025) [107]            | E4    | M  | M  | M  | N  | M  | ?  | N  |
| McMahan et al. (2017) [108]        | E3    | M  | M  | N  | M  | M  | ?  | N  |
| Kendall et al. (2018) [109]        | E3    | M  | P  | N  | N  | ?  | ?  | M  |
| Li et al. (2020) [111]             | E2    | P  | M  | N  | N  | M  | ?  | N  |
| Han et al. (2021) [114]            | E3    | P  | P  | N  | N  | ?  | ?  | M  |
| Han et al. (2022) [115]            | E3    | P  | P  | N  | N  | ?  | ?  | M  |
| Han et al. (2022) [116]            | E3    | P  | P  | N  | N  | ?  | ?  | P  |
| Lindemann et al. (2023) [117]      | E3    | P  | N  | N  | N  | P  | ?  | M  |
| Zollo and Zemel (2025) [118]       | E3    | P  | N  | N  | N  | ?  | ?  | M  |
| Römer et al. (2026) [119]          | E3    | P  | M  | N  | N  | M  | ?  | M  |
| Yuan et al. (2026) [120]           | E4    | M  | P  | M  | N  | ?  | ?  | M  |
| Schaul et al. (2016) [121]         | E3    | P  | M  | N  | N  | M  | ?  | N  |
| Rusu et al. (2016) [122]           | E3    | P  | P  | N  | N  | P  | ?  | N  |
| Kirkpatrick et al. (2017) [123]    | E3    | P  | N  | N  | N  | ?  | ?  | N  |
| Lopez-Paz and Ranzato (2017) [124] | E3    | P  | P  | N  | P  | P  | ?  | N  |
| Rebuffi et al. (2017) [125]        | E3    | P  | N  | N  | P  | P  | ?  | N  |
| Houlsby et al. (2019) [126]        | E3    | P  | N  | N  | N  | M  | ?  | N  |
| Rolnick et al. (2019) [127]        | E3    | M  | N  | N  | N  | ?  | ?  | N  |
| Cai et al. (2020) [128]            | E3    | P  | M  | N  | M  | M  | ?  | N  |
| Lin et al. (2020) [129]            | E3    | P  | N  | N  | M  | M  | ?  | N  |
| Li and Liang (2021) [130]          | E3    | P  | N  | N  | N  | M  | ?  | N  |
| Hu et al. (2022) [131]             | E3    | M  | M  | N  | M  | M  | ?  | N  |
| Lin et al. (2022) [132]            | E3    | P  | N  | N  | M  | M  | ?  | N  |
| Liu et al. (2022) [133]            | E3    | P  | M  | N  | P  | M  | ?  | N  |
| Dettmers et al. (2023) [135]       | E3    | P  | N  | N  | M  | M  | M  | N  |
| Liu et al. (2023) [136]            | E4    | P  | M  | M  | M  | M  | ?  | N  |
| Liu et al. (2023) [137]            | E3    | M  | P  | N  | M  | P  | ?  | N  |
| Malladi et al. (2023) [139]        | E3    | P  | N  | N  | M  | M  | ?  | N  |
| Liu et al. (2024) [140]            | E3    | P  | N  | N  | N  | M  | ?  | N  |
| Wan et al. (2024) [141]            | E4    | M  | M  | P  | N  | M  | ?  | N  |
| Li et al. (2025) [142]             | E3    | P  | N  | N  | P  | ?  | ?  | N  |
| Römer et al. (2026) [143]          | E3    | P  | P  | N  | N  | ?  | ?  | P  |
| Jacobs et al. (1991) [144]         | E2    | P  | N  | N  | N  | ?  | ?  | P  |
| Bengio et al. (2015) [145]         | E3    | M  | P  | N  | N  | P  | ?  | N  |
| Graves (2016) [146]                | E3    | M  | N  | N  | P  | P  | ?  | P  |
| Shazeer et al. (2017) [147]        | E3    | M  | M  | N  | P  | M  | ?  | P  |
| Rosenbaum et al. (2018) [148]      | E3    | M  | N  | N  | P  | M  | ?  | P  |
| Riquelme et al. (2021) [149]       | E3    | M  | N  | N  | M  | M  | ?  | P  |
| Hoque et al. (2021) [150]          | E4    | M  | M  | M  | N  | M  | ?  | M  |
| Fedus et al. (2022) [151]          | E3    | P  | M  | N  | M  | M  | ?  | P  |
| Puigcerver et al. (2024) [152]     | E3    | M  | N  | N  | N  | M  | ?  | P  |
| Wang et al. (2024) [153]           | E4    | M  | P  | M  | M  | P  | ?  | P  |
| Chen et al. (2024) [154]           | E3    | M  | M  | N  | M  | M  | ?  | P  |
| Ding et al. (2024) [155]           | E3    | M  | M  | N  | M  | M  | ?  | P  |
| Aggarwal et al. (2024) [156]       | E3    | M  | M  | N  | M  | M  | ?  | P  |
| Yue et al. (2024) [157]            | E3    | M  | N  | N  | M  | M  | ?  | N  |
| Reuss et al. (2025) [158]          | E3    | P  | M  | N  | M  | M  | ?  | P  |
| Huang et al. (2025) [159]          | E4    | M  | N  | M  | M  | M  | ?  | P  |
| Ong et al. (2025) [160]            | E3    | M  | M  | N  | M  | M  | ?  | P  |
| Hao et al. (2026) [161]            | E4    | M  | P  | M  | P  | P  | ?  | P  |
| Liu et al. (2026) [162]            | E3    | M  | M  | N  | M  | M  | ?  | N  |
| Zhang et al. (2026) [163]          | E4    | M  | M  | M  | M  | M  | ?  | P  |
| Mandlekar et al. (2021) [165]      | E4    | M  | ?  | M  | N  | ?  | ?  | N  |
| Kemp et al. (2022) [167]           | E4    | M  | ?  | M  | N  | ?  | ?  | N  |
| Fu et al. (2024) [168]             | E4    | M  | N  | M  | P  | M  | ?  | N  |
| Collaboration (2024) [170]         | E5    | P  | P  | M  | N  | P  | ?  | N  |
| Kress-Gazit et al. (2024) [171]    | E4    | M  | N  | M  | N  | P  | ?  | N  |

| Work                         | Level | Q1 | Q2 | Q3 | Q4 | Q5 | Q6 | Q7 |
|------------------------------|-------|----|----|----|----|----|----|----|
| Snyder et al. (2025) [172]   | E4    | M  | M  | M  | N  | M  | ?  | N  |
| Team et al. (2025) [173]     | E5    | P  | P  | M  | P  | P  | ?  | N  |
| Alshiekh et al. (2018) [176] | E2    | M  | ?  | N  | N  | ?  | ?  | N  |
| Phan et al. (2020) [177]     | E3    | M  | ?  | N  | N  | ?  | ?  | N  |

## S6 Study summaries

One summary per included work, grouped by topic. The level and the basis of each summary are given in brackets; summaries based on an abstract should be read with that limitation, and figures not given in the abstract are not quoted.

### LLM planners and orchestrators

**Ahn et al. (2022) (E4; abstract).** Ahn et al. [1] address the difficulty that large language models (LLMs) encode rich procedural knowledge but lack grounding in what a particular robot can actually do in its current environment. Their method (SayCan) combines two scores for each candidate natural-language skill: the LLM’s estimate that the skill is a useful next step towards the instruction (‘say’), and the value function of a pretrained, language-conditioned low-level skill that estimates whether the skill can succeed from the current state (‘can’). The product of these scores selects the next skill, so the robot acts as the model’s ‘hands and eyes’ while the LLM supplies task-level semantics. The system is evaluated on real-world, long-horizon, abstract instructions executed by a mobile manipulator; a later revision adds PaLM results, chain-of-thought prompting, multilingual instructions and an LLM-size ablation. The abstract reports that grounding is necessary and that the approach completes long-horizon instructions, but it gives no success rates, which are given only in the full text. Limitations are a fixed library of pretrained skills and a large cloud-hosted LLM. For the orchestration framework, SayCan is the canonical example of fusing a language prior with a learned feasibility signal to select among skills; its affordance value is directly analogous to the calibrated confidence signals the proposed orchestrator would fuse when choosing a learning modality.

**Huang et al. (2022) (E4; abstract).** Huang et al. [2] investigate whether LLM planners for robots benefit from closed-loop feedback expressed in language. Inner Monologue feeds textual environment feedback — success detection, passive scene descriptions and active human-answered queries — back into the LLM prompt so that the model forms an ‘inner monologue’ that it uses to replan, recover from failures and handle changed goals, without any additional training of the language model. The approach is evaluated in three domains: simulated and real tabletop rearrangement and long-horizon mobile manipulation in a real kitchen. The abstract reports that closed-loop language feedback substantially improves high-level instruction completion, including in challenging long-horizon settings, but gives no quantitative figures in the abstract itself; numbers are given only in the full text. Limitations include reliance on the quality of external feedback modules (a poor success detector propagates errors), prompt-length growth over long episodes, and a large cloud-hosted LLM. For the orchestration framework, Inner Monologue shows how heterogeneous signals can be verbalised and fused by a language model to trigger recovery; the proposed orchestrator would replace free-text feedback with calibrated numerical confidence, but success detection and scene description remain natural inputs for deciding when to invoke fault recovery.

**Huang et al. (2022) (E3; abstract).** Huang et al. [3] ask whether world knowledge in pre-trained language models can ground high-level household tasks (e.g. ‘make breakfast’) into sequences of actionable steps without further training. They find that sufficiently large, appropriately prompted LLMs can decompose tasks into mid-level plans zero-shot, but that such free-form plans often fail to map onto the admissible actions of an environment. They therefore propose a procedure that conditions on existing demonstrations and semantically translates the generated steps into admissible actions. Evaluation is conducted in the VirtualHome simulator, with automatic executability metrics and a human evaluation of correctness. The abstract reports that the method substantially improves executability over the LLM baseline and that human evaluation reveals a trade-off between executability and correctness; specific values are not given in the abstract. The work is purely simulated, operates at the symbolic action level without perception or low-level control, and uses large models. Its relevance to the orchestration framework is methodological: the admissible-action translation step is a simple way of constraining an LLM’s output to a fixed vocabulary, which a small on-device orchestrator could use to map its reasoning onto the four learning modalities.

**Shah et al. (2022) (E4; abstract).** Shah et al. [4] present LM-Nav, a system for language-instructed robot navigation assembled entirely from pre-trained models without fine-tuning or language-annotated robot data. GPT-3 extracts a se-

quence of landmarks from the instruction, CLIP ([24]) grounds those landmarks in images stored in a topological graph, and the ViNG goal-conditioned navigation model plans and executes over that graph. The system is instantiated on a real mobile robot and demonstrated on long-horizon navigation through complex outdoor environments. The abstract provides no quantitative results. Limitations include dependence on a prior exploration graph, a cloud language model, and landmark-based instructions. For the framework, LM-Nav illustrates modular composition of frozen perception, language and control models, with CLIP similarity serving as a potential (uncalibrated) perception-confidence signal.

**Liang et al. (2023) (E4; abstract).** Liang et al. [5] repurpose code-writing LLMs to generate robot policy code from natural-language commands. Given few-shot examples of commands (as comments) and corresponding code, the model composes calls to perception and control-primitive APIs, uses control-flow and third-party libraries such as NumPy and Shapely, and supports hierarchical code generation in which undefined functions are recursively defined. The resulting language model programs express reactive policies (e.g. impedance controllers) and waypoint-based policies, demonstrated on multiple real robot platforms. Hierarchical code generation also solves 39.8% of HumanEval problems. Limitations are reliance on a well-designed API and perception stack and the absence of any uncertainty estimate on generated code. For the framework, code-as-policies is a competing orchestration paradigm in which the LLM writes glue logic rather than selecting modalities.

**Driess et al. (2023) (E4; abstract).** Driess et al. [6] propose embodied language models that ingest continuous sensor modalities directly, interleaving encoded images and state estimates with text tokens in a pre-trained LLM so that words are grounded in percepts. PaLM-E is trained end-to-end on a mixture of robotics and general vision-language tasks and is applied to sequential robotic manipulation planning, visual question answering and captioning across multiple robot embodiments. The largest model, PaLM-E-562B, reports state-of-the-art performance on OK-VQA while retaining general language capabilities, and the abstract notes positive transfer from joint training across domains. The limitation for practical deployment is scale: the model is far beyond any edge device and serves only as a high-level planner. For the framework, PaLM-E represents the monolithic alternative to a small modular orchestrator.

**Yao et al. (2023) (E3; abstract).** Yao et al. [7] observe that reasoning (e.g. chain-of-thought prompting) and acting (action-plan generation) in LLMs had been studied separately, and propose ReAct, a prompting paradigm in which the model interleaves free-form reasoning traces with task-specific actions. Reasoning traces help the model induce, track and update plans and handle exceptions, while actions let it query external sources such as knowledge bases or environments for new information. ReAct is evaluated on knowledge-intensive question answering and fact verification (HotpotQA, FEVER), where interaction with a Wikipedia API reduces hallucination and error propagation relative to chain-of-thought alone, and on two interactive decision-making benchmarks, ALFWorld and WebShop. With only one or two in-context examples, ReAct improves absolute success rates over imitation and reinforcement learning baselines by 34% on ALFWorld and 10% on WebShop. The resulting trajectories are also more interpretable. Limitations are reliance on prompting of a large LLM, sensitivity to prompt design and growing context length, and the absence of any calibrated uncertainty over actions. For the orchestration framework, ReAct provides the standard reason-act loop that a local orchestrator could run, with 'actions' being invocations of learning modalities and observations being their confidence signals; it is the primary architectural baseline for agentic control.

**Schick et al. (2023) (E3; abstract).** Schick et al. [8] note that language models are strong at in-context learning yet weak at basic functions such as arithmetic and factual lookup, where simple external tools excel. Toolformer teaches a model to decide which API to call, when to call it, which arguments to pass and how to incorporate the result into subsequent token prediction. Training is self-supervised: starting from a handful of demonstrations per API, the model samples candidate API calls in a large text corpus, executes them, and keeps only those calls whose results reduce the language-modelling loss on the following tokens; the model is then fine-tuned on the augmented corpus. The tools include a calculator, a question-answering system, two search engines, a translation system and a calendar. The abstract reports substantially improved zero-shot performance across a variety of downstream tasks, often competitive with much larger models, without loss of core language-modelling ability; specific figures are not given in the abstract. Limitations include one-step (non-chained) tool use, no interactive tool feedback, and a filtering criterion that depends on loss reduction rather than task success. For the orchestration framework, Toolformer is the closest precedent for a model that learns when to invoke an external capability; its loss-reduction filter is a self-supervised signal that could be adapted to learn when invoking a learning modality improves downstream confidence.

**Shinn et al. (2023) (E3; abstract).** Shinn et al. [9] propose Reflexion, which reinforces language agents through linguistic feedback rather than weight updates. After each trial, the agent verbally reflects on task feedback (scalar or free-form, external or self-generated) and stores the reflection in an episodic memory buffer that conditions subsequent attempts. Evaluated on sequential decision-making, coding and language reasoning tasks, Reflexion reaches 91% pass@1

on HumanEval, compared with 80% for GPT-4, and the authors ablate feedback types and incorporation methods. Limitations are dependence on a capable base model for useful self-reflection and memory limited by context length. For the framework, Reflexion is a low-cost, training-free improvement mechanism that complements LoRA fine-tuning of the orchestrator.

**Singh et al. (2023) (E4; abstract).** Singh et al. [10] address two weaknesses of LLM task planners: scoring requires enumerating all next actions, while free-form generation may propose infeasible actions. ProgPrompt prompts the LLM with Pythonic program-like specifications of available actions and objects plus example programs, so that plans are generated as executable programs with assertions for situated feedback. Ablations yield recommendations on prompt structure and generation constraints. The method reports state-of-the-art success rates on VirtualHome household tasks and is deployed on a physical robot arm for tabletop tasks; exact values are not in the abstract. Limitations include dependence on a hand-specified action and object vocabulary. For the framework, program-structured prompts are a practical way to constrain a small orchestrator’s outputs to valid modality calls.

**Huang et al. (2023) (E4; abstract).** Huang et al. [11] aim to remove the dependence of LLM-based manipulation on predefined motion primitives by synthesising dense 6-DoF end-effector trajectories for open-set instructions and objects. VoxPoser uses an LLM’s code-writing ability to query a vision-language model and compose 3D value maps encoding affordances and constraints in the robot’s observation space; a model-based planner then synthesises closed-loop trajectories zero-shot, robust to dynamic perturbations. The framework can also learn a dynamics model from online experience for contact-rich scenes. A large-scale study in simulated and real-robot environments covers many everyday manipulation tasks; no numbers appear in the abstract. Limitations include reliance on open-vocabulary perception quality and large cloud models. For the framework, the value maps are a spatially explicit perception output whose uncertainty could feed the orchestrator.

**Yu et al. (2023) (E4; abstract).** Yu et al. [12] argue that low-level robot actions are hardware-specific and poorly represented in LLM training data, and propose reward functions as the interface between language and control. An LLM translates instructions or corrections into reward parameters, which are optimised in real time by MuJoCo MPC, enabling interactive behaviour creation with immediate user feedback. On 17 tasks designed for a simulated quadruped and a dexterous manipulator, the method reliably solves 90% of tasks, against 50% for a Code-as-Policies baseline using primitive skills ([5]). Validation on a real robot arm shows emergent skills such as non-prehensile pushing. Limitations include dependence on a fast, accurate simulator model for MPC and hand-designed reward templates. For the framework, the method illustrates an LLM routing language into an optimisation-based learner rather than directly into actions.

**Song et al. (2023) (E3; abstract).** Song et al. [13] target the high data cost of instruction-following embodied agents by using LLMs as few-shot planners. LLM-Planner generates high-level plans from a few in-context examples and adds physical grounding by dynamically re-planning with objects observed in the current environment when the agent stalls or fails. On the ALFRED benchmark, the method achieves competitive few-shot performance using less than 0.5% of the paired training data required by baselines trained on the full dataset. Limitations are dependence on a separate low-level controller and object detector and the simulated setting. For the framework, grounded re-planning triggered by execution failure is a simple template for invoking recovery when confidence drops.

**Yao et al. (2023) (E2; abstract).** Yao et al. [14] generalise chain-of-thought prompting into Tree of Thoughts (ToT), in which an LLM explores multiple coherent intermediate ‘thoughts’, self-evaluates them and uses search (breadth- or depth-first) with lookahead and backtracking to make global decisions. Experiments on three tasks requiring planning or search — Game of 24, Creative Writing and Mini Crosswords — show large gains; in Game of 24, GPT-4 with chain-of-thought solves 4% of tasks whereas ToT solves 74%. The main limitation is computational: many more LLM calls per problem. For the framework, ToT’s self-evaluation of candidate thoughts is a form of LLM-internal confidence, but its inference cost is a poor match for a Raspberry-Pi-class orchestrator operating under a compute budget.

**Ren et al. (2023) (E4; abstract).** Ren et al. [15] tackle the tendency of LLM planners to produce confidently hallucinated plans. Their framework, KnowNo, measures and aligns the uncertainty of an LLM planner so that the robot knows when it does not know and asks for help. Candidate next steps are generated as multiple-choice options, and conformal prediction calibrated on held-out data turns the model’s option likelihoods into a prediction set; if the set contains more than one option the robot requests human clarification. This yields statistical guarantees on task completion at a user-specified level while minimising the amount of help requested in multi-step planning. Experiments across simulated and real robot setups span different modes of ambiguity, from spatial and numeric uncertainty to human preferences and Winograd schemas. The abstract reports that KnowNo compares favourably with baselines, including ensembles and extensive prompt tuning, in efficiency and autonomy while providing formal assurances; it gives no figures, which are given

only in the full text. KnowNo works out of the box with LLMs without fine-tuning, but relies on exchangeability between calibration and deployment distributions and on access to token likelihoods. For the orchestration framework this is a key reference: conformal calibration of an LLM’s choice distribution is directly transferable to calibrating the orchestrator’s modality selection, and the help request is a model for escalation when no modality is confidently appropriate.

**Chen et al. (2023) (E3; abstract).** Chen et al. [16] argue that language agents have mostly been built by few-shot prompting off-the-shelf models and investigate the neglected alternative of fine-tuning the backbone language model for agentic behaviour. Using question answering with a Google search API as the testbed, they systematically vary base models, prompting methods, fine-tuning data and QA tasks. Agents consistently improve after fine-tuning; for example, fine-tuning Llama2-7B on 500 agent trajectories generated by GPT-4 increases HotpotQA performance by 77%. They then propose FireAct, which fine-tunes on trajectories drawn from multiple tasks and several prompting methods (such as ReAct, [7], chain-of-thought and Reflexion-style formats), and show that more diverse fine-tuning data further improves agents. The study also reports findings on scaling, robustness, generalisation, efficiency and cost, and frames a set of open questions for agent fine-tuning. Limitations are the restriction to a single tool and text-only QA, reliance on a stronger teacher model to generate trajectories, and no embodied evaluation. For the orchestration framework, FireAct provides direct evidence that a small (7B) open model distilled from a larger teacher can become a competent agent with a few hundred trajectories — the same recipe the framework would use to specialise a local orchestrator, with LoRA making such fine-tuning affordable.

**Wang et al. (2024) (E3; abstract).** Wang et al. [17] introduce Voyager, an LLM-powered lifelong-learning agent in Minecraft that explores, acquires skills and makes discoveries without human intervention. It comprises an automatic curriculum maximising exploration, an ever-growing library of executable code skills, and iterative prompting that incorporates environment feedback, execution errors and self-verification. Compared with prior state of the art, Voyager obtains 3.3 times more unique items, travels 2.3 times longer distances and unlocks key tech-tree milestones up to 15.3 times faster, and it reuses its skill library in new worlds. Limitations are the reliance on GPT-4 via black-box queries and a game environment with a high-level API. For the framework, the skill library and self-verification loop are models for an orchestrator that accumulates reusable competences online.

**Liang et al. (2024) (E3; abstract).** Liang et al. [18] address two sources of risk in LLM-based robot planning: hallucination, which can lead robots to execute confidently wrong or unsafe plans, and genuine ambiguity in natural-language instructions. They propose introspective planning, which aligns the LLM’s uncertainty with the inherent ambiguity of the task. A knowledge base is built from introspective reasoning examples — post-hoc rationalisations of human-selected safe and compliant plans — and relevant examples are retrieved at deployment to guide the LLM’s reasoning before it commits to an action. Evaluations on three tasks, including a newly introduced safe mobile manipulation benchmark, show that introspection substantially improves both compliance and safety relative to state-of-the-art LLM planning methods. Combined with conformal prediction, introspective planning yields tighter confidence bounds, maintaining statistical success guarantees while reducing unnecessary clarification requests relative to direct conformal calibration as in KnowNo ([15]). The abstract provides no numerical results, so these are given only in the full text. Limitations include the cost of constructing the knowledge base with human labels, dependence on retrieval quality, and the same exchangeability assumptions as conformal prediction. For the orchestration framework, the work shows that retrieved, experience-derived reasoning can sharpen calibrated uncertainty, suggesting that the orchestrator’s replay buffer could double as a retrieval base for better-calibrated modality selection.

**Wang et al. (2024) (E3; abstract).** Wang et al. [19] consider task planning for teams of robots instructed in natural language, noting that existing LLM-based multi-robot planners lack performance guarantees. They introduce S-ATLAS, a distributed LLM-based planner that uses conformal prediction so that each robot reasons about its own uncertainty, acts when sufficiently certain and seeks help otherwise. They show theoretically and empirically that the planner achieves user-specified task success rates, assuming successful plan execution, while minimising help requests. Comparisons with related work indicate greater computational efficiency and lower help rates, with the advantage growing with team size; no figures appear in the abstract. The execution-success assumption is the main limitation. For the framework, S-ATLAS extends conformal escalation to distributed decision-makers, relevant if modality selection is decomposed.

**Dalal et al. (2024) (E3; full text).** Dalal et al. [20] observe that LLM planners for long-horizon robotics presuppose a library of low-level skills, and propose Plan-Seq-Learn (PSL), in which an LLM produces a high-level plan, a sequencing module uses vision and motion planning to reach the relevant region, and RL policies learn the contact-rich low-level interaction online. Experiments in MuJoCo with Sawyer and Panda arms span Meta-World, Robosuite, Kitchen and obstructed suites, covering over 25 tasks with up to 10 stages. PSL reports success rates above 85% from visual input; for example,  $96\% \pm 8\%$  on a four-stage nut-assembly task versus  $23\% \pm 21\%$  for SayCan ([1]), and 100% on 5–10-stage

kitchen tasks where baselines score 0%. The work is simulation-only. For the framework, PSL is a clear instance of an LLM delegating to a learned RL modality.

**Zeng et al. (2024) (E3; abstract).** Zeng et al. [21] note that open LLMs lag well behind commercial models when acting as agents that plan, memorise and use tools, and that prior work focused on prompting rather than improving the models' own agent capabilities. AgentTuning constructs AgentInstruct, a lightweight instruction-tuning dataset of high-quality interaction trajectories, and mixes it with general-domain open instructions in a hybrid tuning strategy so that agent skills improve without degrading general abilities. Applied to the Llama 2 series (7B, 13B and 70B), the resulting AgentLM models gain agent capabilities, and AgentLM-70B is reported as comparable to GPT-3.5-turbo on unseen agent tasks. Limitations are the text-only agent tasks and reliance on stronger models for trajectory generation. For the framework, the hybrid mixture is a practical guard against forgetting when fine-tuning an orchestrator.

**Yao et al. (2024) (E3; abstract).** Yao et al. [22] address the observation that language agents built on large LLMs are generally not optimised with environment-specific rewards: verbal self-refinement approaches allow iterative improvement but do not exploit gradient-based learning from rewards. They propose Retroformer, a framework that reinforces a language agent by learning a separate retrospective model. The main actor LLM is kept frozen, while a smaller pre-trained language model is fine-tuned with policy gradient on rewards collected across multiple environments and tasks; its role is to refine the actor's prompt by summarising the root causes of prior failed attempts and proposing action plans for the next trial. The abstract reports that language agents improve over time under this scheme and that the approach outperforms baselines that do not properly exploit gradients from the environment; the abstract gives no numerical results or list of environments. The authors suggest the approach could also optimise other agent components. Limitations include the need for repeated trials with reward signals and the indirect nature of improvement through prompt refinement. For the orchestration framework, Retroformer is a close architectural analogue: a small, trainable module improving a frozen larger model from reward, which maps onto LoRA-tuning a compact orchestrator from task outcomes.

**Zhai et al. (2024) (E3; full text).** Zhai et al. [23] argue that fine-tuning vision-language models on visual instruction data is insufficient for multi-step goal-directed decision-making, and propose fine-tuning a VLM end-to-end with reinforcement learning (PPO, [57]), where the model emits chain-of-thought reasoning before a text action that is parsed and executed, and task rewards update the whole model. Using a LLaVA-v1.6-Mistral-7B backbone on card-arithmetic tasks and ALFWorld, the method outperforms GPT-4V and Gemini; for example 50.0% on EZPoints versus 10.5% for GPT-4V, and 21.7% average on ALFWorld versus 19.4%. Removing chain-of-thought substantially degrades performance. Limitations are modest absolute ALFWorld scores and heavy RL compute. For the framework, this shows RL fine-tuning of a 7B multimodal agent is viable, though not on-device.

## VLA and robot foundation models

**Radford et al. (2021) (E3; abstract).** Radford et al. [24] propose learning visual representations from natural-language supervision rather than fixed label sets. CLIP jointly trains image and text encoders with a contrastive objective to predict which caption goes with which image, on 400 million image-text pairs collected from the internet. Natural language then enables zero-shot transfer by embedding class names as text. Benchmarked on over 30 computer-vision datasets, CLIP transfers non-trivially to most tasks and matches the accuracy of the original ResNet-50 on ImageNet zero-shot, without using any of its 1.28 million training examples. Limitations include weaker performance on fine-grained or abstract tasks and uncalibrated similarity scores. For the framework, CLIP-style encoders are a candidate perception backbone whose image-text similarities can be converted into calibrated perception confidence.

**Shridhar et al. (2021) (E4; abstract).** Shridhar et al. [25] combine semantic understanding and spatial precision for language-conditioned manipulation. CLIPort is a two-stream imitation-learning agent: a semantic ('what') pathway from CLIP ([24]) and a spatial ('where') pathway from Transporter networks, trained end-to-end to predict pick-and-place affordances without explicit object poses, segmentations, memory or symbolic states. Tasks range from packing unseen objects to folding cloths. Experiments in simulation and the real world show data efficiency in few-shot settings and generalisation to seen and unseen semantic concepts, and a single multi-task policy over 10 simulated and 9 real-world tasks performs better than or comparably to single-task policies. Limitations are restriction to planar pick-and-place primitives and a top-down camera. For the framework, CLIPort is a lightweight imitation baseline whose affordance heatmaps offer a spatial confidence signal.

**Reed et al. (2022) (E4; abstract).** Reed et al. [26] apply the large-scale sequence-modelling approach of language models to build Gato, a single generalist agent across modalities and embodiments. All data — images, text, proprioception, joint torques, button presses and discrete or continuous actions — are serialised into tokens, and one transformer

with a single set of weights is trained on them. The same network plays Atari, captions images, chats and stacks blocks with a real robot arm, deciding from context whether to output text, joint torques, button presses or other tokens. The abstract gives no headline metrics. Limitations include modest per-task performance relative to specialists and the cost of scale. For the framework, Gato represents the single-policy alternative to explicit modality orchestration.

**Shridhar et al. (2022) (E4; abstract).** Shridhar et al. [27] ask whether transformers can benefit data-scarce robotic manipulation given a suitable problem formulation. PerAct is a language-conditioned behaviour-cloning agent that encodes language goals and RGB-D voxel observations with a Perceiver Transformer and outputs discretised 6-DoF actions by detecting the next best voxel action, providing a strong 3D structural prior. A single multi-task model is trained on 18 RL Bench tasks (249 variations) and 7 real-world tasks (18 variations) from only a few demonstrations per task. PerAct significantly outperforms unstructured image-to-action agents and 3D ConvNet baselines across tabletop tasks. Limitations include reliance on calibrated RGB-D cameras, keyframe-based motion planning and voxel memory cost. For the framework, PerAct is a strong imitation-modality baseline, though its voxel inputs sit uneasily with LeKiwi’s RGB cameras.

**Brohan et al. (2023) (E5; abstract).** Brohan et al. [28] argue that general robot models require open-ended, task-agnostic training combined with high-capacity architectures able to absorb diverse robot data. They present Robotics Transformer 1 (RT-1), a model class that tokenises images and language instructions and outputs discretised actions, and study how generalisation scales with data size, model size and data diversity. The study uses a large-scale real-robot data collection on real-world tasks with mobile manipulators. The abstract gives no numbers; the paper reports task counts and success rates that should be quoted from its body. Limitations are the cost of large-scale real data collection and restriction to imitation of demonstrated behaviour. For the framework, RT-1 is a reference imitation-learning policy and scaling study rather than a component deployable on a Pi-class robot.

**Brohan et al. (2023) (E4; abstract).** Brohan et al. [29] study how vision-language models trained on internet-scale data can be incorporated directly into end-to-end robot control. They express robot actions as text tokens and co-fine-tune a VLM on robot trajectories together with vision-language tasks, yielding vision-language-action (VLA) models; the instantiation is RT-2. Across 6,000 evaluation trials, RT-2 shows improved generalisation to novel objects, interpretation of commands absent from the robot data, and rudimentary semantic reasoning, such as choosing an object usable as an improvised hammer or an appropriate drink for a tired person, aided by chain-of-thought. Limitations are very large model size, cloud-based inference and skills bounded by the demonstrated robot data. For the framework, RT-2 defines the VLA paradigm against which smaller, orchestrated modalities must justify themselves.

**Jiang et al. (2023) (E3; abstract).** Jiang et al. [30] show that diverse manipulation tasks can be specified with multi-modal prompts interleaving text and images. They build VIMA-Bench, a simulation benchmark of thousands of procedurally generated tabletop tasks with 600K+ expert trajectories and a four-level protocol for systematic generalisation, and VIMA, a transformer agent that encodes prompts and decodes motor actions autoregressively. VIMA achieves up to 2.9 times higher zero-shot task success than alternatives with the same training data, and 2.7 times better performance with 10 times less data than the best competing variant. Limitations are the simulated setting and high-level action primitives. For the framework, VIMA is an evaluation reference for prompt-conditioned imitation and systematic generalisation.

**Kim et al. (2024) (E5; abstract).** Kim et al. [31] address two barriers to adopting vision-language-action (VLA) models: most were closed, and efficient fine-tuning to new tasks had been little explored. OpenVLA is a 7-billion-parameter open-source VLA built on a Llama 2 language backbone with a visual encoder that fuses pretrained DINOv2 and SigLIP features, and trained on 970k real-world robot demonstrations drawn from diverse embodiments. In generalist manipulation evaluations across 29 tasks and multiple robot embodiments, OpenVLA outperforms the closed RT-2-X (55B parameters) by 16.5% in absolute task success rate while using seven times fewer parameters. The authors further show that OpenVLA can be fine-tuned effectively for new settings, particularly in multi-task environments involving several objects and strong language grounding, where it outperforms from-scratch imitation methods such as Diffusion Policy by 20.4%. They also report that low-rank adaptation ([131]) allows fine-tuning on consumer GPUs and that quantisation supports efficient serving without loss in downstream success. Limitations include single-image observations, inference rates well below high-frequency control, and a 7B footprint. For the orchestration framework, OpenVLA is both the main open VLA baseline for the imitation modality and a demonstration that LoRA adaptation of robot foundation models is practical, although even quantised it would require offloading from a Raspberry Pi 5 to a GPU host.

**Ghosh et al. (2024) (E5; abstract).** Ghosh et al. [32] lay groundwork for open, widely applicable generalist policies for robotic manipulation that can handle diverse sensors and action spaces and be fine-tuned efficiently. Octo is a large

transformer-based policy trained on 800k trajectories from the Open X-Embodiment dataset, instructable via language commands or goal images. It can be fine-tuned to robot setups with new sensory inputs and action spaces within a few hours on standard consumer GPUs. Experiments across 9 robotic platforms show that Octo is a versatile policy initialisation, and detailed ablations of architecture and training data guide future work. Limitations include the need for GPU fine-tuning per setup and variable zero-shot performance. For the framework, Octo is a smaller open alternative to OpenVLA ([31]) for initialising the imitation modality.

**Black et al. (2024) (E5; abstract).** Black et al. [33] discuss how generalist robot policies can address data, generalisation and robustness obstacles, and propose  $\pi_0$ , a flow-matching action architecture built on a pre-trained vision-language model to inherit internet-scale semantic knowledge. The model is trained on a large, diverse dataset from multiple dexterous platforms, including single-arm, dual-arm and mobile manipulators. Evaluation covers zero-shot task performance after pre-training, following language instructions from people and from a high-level VLM policy, and acquisition of new skills through fine-tuning, on tasks such as laundry folding, table cleaning and box assembly. The abstract reports no quantitative results. Limitations are model scale and proprietary training data. For the framework,  $\pi_0$ 's split between a high-level VLM and a low-level action expert parallels the orchestrator–modality hierarchy.

**Bousmalis et al. (2024) (E5; abstract).** Bousmalis et al. [34] propose RoboCat, a multi-embodiment, multi-task generalist agent for robotic manipulation, implemented as a visual goal-conditioned decision transformer that consumes action-labelled visual experience. Trained on a large, diverse dataset spanning simulated and real robot arms with different observation and action spaces, RoboCat generalises zero-shot to new tasks and robots and can adapt to a new task or embodiment with only 100–1,000 target examples. The authors also describe a self-improvement loop: the adapted agent generates additional data that are fed back into training, yielding a more capable agent. Evaluation covers three real robot embodiments plus simulation. Limitations include large-scale compute and a goal-image interface. For the framework, RoboCat's self-generated-data loop is a precedent for the orchestrator's replay-based online improvement.

**Li et al. (2024) (E3; abstract).** Li et al. [35] seek a simple way to adapt existing open vision-language models to robot manipulation. RoboFlamingo builds on OpenFlamingo, using the pretrained VLM for single-step vision-language comprehension and adding an explicit policy head that models sequential history; it is lightly fine-tuned by imitation learning on language-conditioned manipulation datasets only. This decomposition allows open-loop control and deployment on low-performance platforms. The abstract reports that RoboFlamingo exceeds the state of the art by a large margin on the tested benchmark (the CALVIN benchmark in the paper), without giving figures. Limitations include simulation-centric evaluation and the size of the VLM backbone. For the framework, RoboFlamingo's frozen-backbone plus small-head design is a template for cheap adaptation of an imitation modality.

**Liu et al. (2024) (E4; abstract).** Liu et al. [36] target two weaknesses of VLA models: limited reasoning on complex tasks and high computational cost of fine-tuning and inference. RoboMamba builds an end-to-end robotic VLA on the Mamba state-space model, whose inference complexity is linear in sequence length, combining a vision encoder with a Mamba language model to provide both reasoning and action capability. After aligning visual and language features, the model gains manipulation skills by fine-tuning only a small policy head, about 0.1% of the model's parameters. The authors report inference roughly three times faster than existing VLA models and competitive reasoning and manipulation results in simulation and the real world; exact benchmark values were not verified from the abstract. Limitations include a multi-billion-parameter backbone. For the framework, RoboMamba shows that efficiency-oriented architectures and tiny trainable heads can reduce the adaptation cost of an imitation modality.

**Shukor et al. (2025) (E4; abstract).** Shukor et al. [37] address the cost of vision-language-action (VLA) models, which typically comprise billions of parameters, incur high training costs and are difficult to deploy, and which are trained almost exclusively on academic and industrial datasets. They present SmolVLA, a small VLA built on a compact pretrained vision-language model and trained on community-collected data from affordable robotic platforms, designed to be trained on a single GPU and deployed on consumer-grade GPUs or even CPUs. To improve responsiveness they introduce an asynchronous inference stack that decouples perception and action prediction from action execution, permitting higher control rates with chunked action generation. The model is evaluated on both simulated and real-world robotic benchmarks, and the authors report performance comparable to VLAs roughly ten times larger; all code, pretrained models and training data are released within the LeRobot library. The abstract does not state parameter counts, success rates or measured latencies, so the scale of the gains and the exact hardware used for CPU deployment must be checked in the full text. For the orchestration framework SmolVLA is the most directly relevant imitation-learning component: it is native to the LeRobot stack and SO-101-class hardware used on LeKiwi, and its asynchronous chunked execution offers a natural point at which an orchestrator could inspect action-chunk statistics as a behavioural-fidelity signal.

**Wen et al. (2025) (E4; abstract).** Wen et al. [38] target two obstacles to deploying vision-language-action (VLA) models: slow inference and dependence on extensive pre-training on large robot datasets. They introduce TinyVLA, a family of compact VLA models built from two components: a policy backbone initialised from robust, high-speed multimodal models, and a diffusion-policy decoder integrated during fine-tuning to produce precise continuous actions. This design removes the separate robot-data pre-training stage. The authors evaluate TinyVLA in simulation and on real robots and report that it significantly outperforms OpenVLA (Kim 2024) in inference speed and data efficiency while achieving comparable or superior task performance. They also report generalisation across language instructions, novel objects, unseen positions, changes in object appearance, background variations and environmental shifts, often matching or exceeding OpenVLA. The abstract gives no numeric speed-ups, parameter counts or success rates, and the comparison is chiefly against a single, comparatively slow baseline; the real-robot platforms and task counts must be taken from the full text. For the orchestration framework TinyVLA is a candidate imitation component for constrained hardware, and its diffusion decoder offers a sampling-based route to behavioural-fidelity estimates, since dispersion across denoising samples can be read as a confidence signal by the orchestrator.

**Budzianowski et al. (2025) (E2; abstract).** Budzianowski et al. [39] consider the difficulty of deploying large vision-language-action (VLA) models such as OpenVLA (Kim 2024) on resource-constrained mobile manipulation systems. They propose Edge VLA (EVLA), which retains the representational power of VLM-based policies while targeting real-time inference on edge devices through two changes. First, they eliminate the autoregressive requirement for predicting end-effector positions, predicting the action jointly rather than token by token, which the authors report yields a 7x inference speed-up. Second, they replace the large language backbone with a small language model and report training characteristics comparable to larger models at substantially reduced computational demand. The paper is presented as early results from a workshop contribution: the abstract reports comparable training behaviour to OpenVLA and gains in inference speed and memory efficiency, but gives no task success rates, memory figures or specific edge hardware, and closed-loop real-robot evaluation is not described. Checkpoints and the training code are released. For the orchestration framework EVLA illustrates two budget-saving levers, non-autoregressive decoding and small backbones, that bear directly on whether a VLA-style imitation module can coexist with an LLM orchestrator on Raspberry-Pi-class compute; its evidence base is, however, preliminary and should be cited as such.

**Kim et al. (2025) (E4; abstract).** Kim, Finn and Liang [40] study how best to fine-tune vision-language-action (VLA) models for new robot set-ups, using OpenVLA (Kim 2024) as the base model. They systematically compare action decoding schemes, action representations and learning objectives, and distil an Optimised Fine-Tuning (OFT) recipe combining parallel decoding, action chunking, continuous actions and an L1 regression objective. OpenVLA-OFT raises average success on the four LIBERO suites from 76.5% to 97.1% and increases action-generation throughput by 26x. On a bimanual ALOHA robot it performs dexterous, high-frequency tasks and outperforms  $\pi 0$  and RDT-1B fine-tuned with their default recipes, as well as Diffusion Policy and ACT, by up to 15% absolute average success. The base model remains a 7B-parameter network requiring GPU inference. For the framework, OFT is a reference recipe for adapting an imitation module and a strong baseline; its deterministic L1 head, however, yields no native distributional confidence signal.

**Pertsch et al. (2025) (E4; abstract).** Pertsch et al. [41] examine action tokenisation for autoregressive vision-language-action policies, finding that per-dimension, per-timestep binning performs poorly on dexterous skills learned from high-frequency robot data. They propose Frequency-space Action Sequence Tokenization (FAST), a compression-based scheme that applies the discrete cosine transform to action chunks, enabling autoregressive VLAs to learn highly dexterous, high-frequency tasks on which standard discretisation fails completely. They also release FAST+, a universal tokeniser trained on 1M real robot action trajectories, usable as a black box across diverse action spaces and control frequencies. Combined with the  $\pi 0$  VLA (Black 2024), the method scales to 10k hours of robot data and matches diffusion VLAs while reducing training time by up to 5x. The abstract does not quantify inference latency, and autoregressive decoding of long token sequences remains costly on small hardware. For the framework, FAST offers token-level likelihoods that could serve as a behavioural-fidelity confidence signal.

**Wang et al. (2025) (E4; abstract).** Wang et al. [42] argue that VLA deployment on edge devices is limited by model size and that efficiency should be designed in during training rather than obtained by post-hoc compression alone. They propose BitVLA, a fully native 1-bit VLA in which every parameter is ternary ( $\{-1, 0, 1\}$ ), built on the 1-bit LLM BitNet b1.58 2B4T. To shrink the vision backbone they introduce Quantize-then-Distill, a quantisation-aware training strategy that compresses a full-precision vision encoder to 1.58-bit weights while a full-precision teacher guides representation alignment. Across simulation benchmarks and real-world tasks, BitVLA matches the full-precision OpenVLA-OFT baseline (Kim 2025) while reducing model memory by 11.0x and end-to-end latency by 4.4x. The work is marked as in progress, and the abstract does not specify the edge hardware or absolute latencies. For the framework,

BitVLA indicates that ternary-weight policies may approach Raspberry-Pi-class memory budgets, freeing capacity for an on-board orchestrator.

**Ravichandran et al. (2025) (E4; abstract).** Ravichandran et al. [43] observe that LLM-based robot planners usually depend on cloud-hosted models, which limits deployment where connectivity is unreliable, such as outdoor or industrial sites. They introduce PRISM, a framework for distilling compact on-device language models for robot planning with minimal human supervision. PRISM automatically generates diverse synthetic tasks and environments, elicits plans for them from a large teacher LLM, and trains a smaller model to act as a drop-in substitute for the teacher in the planning loop. The approach is evaluated in three planning domains, namely mapping, manipulation and household tasks, and the distilled planners are reported to transfer across robotic platforms, including ground and aerial robots, and across indoor and outdoor environments. Using synthetic data alone, a Llama-3.2-3B student improves from 10–20% of GPT-4o’s performance to over 93%. The abstract does not report on-device latency, memory use or the precise hardware, and performance is expressed relative to the teacher rather than as absolute task success, so failures inherited from GPT-4o are not separated out. Code, models and datasets are released. For the orchestration framework PRISM is the closest precedent for a small, cloud-free LLM controller: its synthetic-task distillation pipeline could be reused to train the LeKiwi orchestrator’s modality-selection policy before online LoRA refinement.

**Yu et al. (2025) (R; abstract).** Yu et al. [44] survey methods for making vision-language-action (VLA) models efficient, motivated by the prohibitive computational and data demands of large foundational VLAs and the absence of a unifying framework for the rapidly growing literature. They present what they describe as the first comprehensive review of efficient VLAs across the model–training–data pipeline, organised under a taxonomy of three pillars: efficient model design (architectures and compression), efficient training (reducing computational cost during learning) and efficient data collection (acquiring and using robot data). For the orchestration framework it provides a map of compression, quantisation and small-backbone techniques relevant to fitting imitation and perception modules within a Raspberry-Pi-class budget.

**Zhang et al. (2026) (E3; abstract).** Zhang et al. [45] address the growing compute and memory demands of VLA models by proposing QuantVLA, a training-free post-training quantisation framework which they describe as the first PTQ approach for VLA systems and the first to quantise a diffusion-transformer (DiT) action head successfully. It combines three scale-calibrated components: a selective layout that integerises all linear layers in the language backbone and DiT while keeping attention projections in floating point; attention temperature matching, a per-head scaling folded into dequantisation scales; and output head balancing, a per-layer residual calibration against post-projection energy drift. Only a small unlabelled calibration buffer is needed. On LIBERO with representative VLA models, QuantVLA exceeds full-precision task success and saves about 70% memory on the quantised components. The abstract gives no bit-widths, speed figures or real-robot tests. For the framework, QuantVLA is a candidate compression step for the imitation module under a fixed memory budget.

## Perception and epistemic uncertainty

**Gal and Ghahramani (2016) (E2; abstract).** Gal and Ghahramani [46] address the absence of model uncertainty in standard deep networks for regression and classification, noting that Bayesian models capture it but are usually computationally prohibitive. They develop a theoretical framework in which dropout training in deep neural networks is cast as approximate Bayesian inference in deep Gaussian processes. A direct consequence is that uncertainty can be extracted from existing dropout networks, by keeping dropout active at test time and averaging over stochastic forward passes (Monte Carlo dropout), without changing the training procedure or sacrificing test accuracy. The authors study the properties of this uncertainty across network architectures and non-linearities on regression and classification tasks, using MNIST as an example, and report considerable improvements in predictive log-likelihood and RMSE over the then state of the art; they also use dropout uncertainty to guide exploration in deep reinforcement learning. The abstract does not give the numerical gains. Subsequent work has shown that MC-dropout estimates can be poorly calibrated and depend strongly on the dropout rate, and the multiple forward passes multiply inference cost. For the orchestration framework MC dropout is the cheapest route to an epistemic-uncertainty signal for the perception module, since it requires no retraining; its per-sample cost, however, must be budgeted explicitly on a Raspberry Pi 5.

**Lakshminarayanan et al. (2017) (E2; abstract).** Lakshminarayanan, Pritzel and Blundell [47] address predictive uncertainty in deep networks, noting that Bayesian neural networks, then the state of the art, require substantial changes to training and are computationally costly. They propose deep ensembles, a non-Bayesian alternative in which several networks are trained independently from different random initialisations, optionally with proper scoring rules and adversarial

training, and their predictions combined. The method is simple to implement, readily parallelisable and requires little hyperparameter tuning. On classification and regression benchmarks it produces well-calibrated uncertainty estimates that are as good as or better than approximate Bayesian neural networks. To assess robustness to dataset shift, the authors evaluate uncertainty on test examples from known and unknown distributions and show that ensembles express higher uncertainty on out-of-distribution inputs; scalability is demonstrated on ImageNet. The abstract gives no specific metric values. The principal cost is that training and inference scale linearly with ensemble size, and members trained on the same data may still be correlated. For the orchestration framework deep ensembles are the reference epistemic-uncertainty signal against which cheaper estimators should be judged, and small ensembles of lightweight heads, or ensembles of Q-functions in the RL module, are realistic on Raspberry-Pi-class hardware.

**Kendall and Gal (2017) (E2; abstract).** Kendall and Gal [48] distinguish aleatoric uncertainty, arising from noise inherent in observations, from epistemic uncertainty, which reflects model ignorance and can be reduced with more data, and ask which is needed in Bayesian deep learning for computer vision. They present a framework that combines input-dependent (heteroscedastic) aleatoric uncertainty, predicted as an additional network output, with epistemic uncertainty obtained through approximate Bayesian inference such as Monte Carlo dropout (Gal 2016). They study the framework on per-pixel semantic segmentation and depth regression. The explicit uncertainty formulation yields new loss functions interpretable as learned attenuation, which makes training more robust to noisy data and gives new state-of-the-art results on segmentation and depth-regression benchmarks at the time; the abstract does not give specific figures. The epistemic component inherits the sampling cost of MC dropout, and the decomposition relies on the adequacy of the approximate posterior. For the orchestration framework this paper supplies the conceptual separation that the orchestrator needs: aleatoric uncertainty indicates irreducible sensor ambiguity, for which switching modality may not help, whereas epistemic uncertainty signals that further learning or a different modality is warranted.

**Guo et al. (2017) (E2; abstract).** Guo et al. [49] study confidence calibration, the property that a classifier’s predicted probabilities reflect its true likelihood of being correct. They report that modern neural networks, unlike those of a decade earlier, are poorly calibrated, typically over-confident, and through extensive experiments identify depth, width, weight decay and batch normalisation as important factors influencing miscalibration. They then evaluate a range of post-processing calibration methods on state-of-the-art architectures using image and document classification datasets, measuring calibration with metrics such as expected calibration error and reliability diagrams. Their practical conclusion is that, on most datasets, temperature scaling, a single-parameter variant of Platt scaling that divides the logits by a scalar fitted on a validation set, is surprisingly effective. The abstract provides no specific error values. Temperature scaling does not change the predicted class and is fitted in-distribution, so it offers no guarantee under dataset shift, which is precisely the regime a mobile robot encounters when it enters a new room or lighting condition. For the orchestration framework this paper supplies the basic calibration step that must be applied before confidence signals from perception, policy and fault modules can be compared or fused on a common scale; temperature scaling adds essentially no inference cost and is therefore compatible with a Raspberry Pi 5 budget.

**Hendrycks and Gimpel (2017) (E2; abstract).** Hendrycks and Gimpel [50] consider two related problems: detecting whether an input is misclassified and whether it is out-of-distribution (OOD). They present a simple baseline based on the maximum softmax probability, observing that correctly classified examples tend to have higher maximum softmax probabilities than misclassified and OOD examples, which permits their detection by thresholding. Performance is assessed on a set of detection tasks they define across computer vision, natural language processing and automatic speech recognition, showing the baseline to be effective in all three domains; they further show that it can sometimes be surpassed, establishing these as open research problems. The abstract gives no numerical results; the full paper reports threshold-independent metrics such as AUROC and AUPR, which should be verified before quotation. The baseline inherits the over-confidence of uncalibrated softmax outputs (Guo 2017) and can assign high confidence to far-from-distribution inputs. It nevertheless costs nothing beyond a standard forward pass. For the orchestration framework maximum softmax probability is the zero-overhead lower bound for a perception confidence signal, the baseline that more expensive estimators such as Monte Carlo dropout (Gal 2016) or ensembles (Lakshminarayanan 2017) must beat to justify their compute on a Raspberry Pi 5.

**Sensoy et al. (2018) (E2; abstract).** Sensoy, Kaplan and Kandemir [51] note that deterministic networks trained to minimise a prediction loss remain ignorant of their own confidence. Rather than inferring uncertainty indirectly through weight distributions, as Bayesian neural networks do, they model it explicitly using the theory of subjective logic. A Dirichlet distribution is placed over class probabilities, predictions are treated as subjective opinions, and a deterministic network learns to output the evidence behind these opinions; the resulting predictor for multi-class classification is itself a Dirichlet distribution whose parameters are given by the network’s continuous output. The authors provide a preliminary analysis of how their new loss function drives improved uncertainty estimation, and report what they describe as unprecedented success in detecting out-of-distribution queries and robustness to adversarial perturbations. The abstract gives no

numerical results or datasets. Later work has questioned whether evidential outputs faithfully represent epistemic uncertainty, and the loss introduces a regularisation weight that requires tuning. The key practical property is that uncertainty is obtained in a single deterministic forward pass. For the orchestration framework evidential classification is therefore an attractive perception confidence signal on a Raspberry Pi 5, offering an uncertainty mass comparable in cost to the maximum-softmax baseline (Hendrycks 2017) without the sampling overhead of Monte Carlo dropout.

**Amini et al. (2020) (E2; abstract).** Amini et al. Amini et al. [52] address the need for calibrated, robust and efficient uncertainty in deterministic networks deployed in safety-critical settings, focusing on regression. They train non-Bayesian networks to predict a continuous target together with its associated evidence, placing evidential priors over the Gaussian likelihood and training the network to infer the hyperparameters of the resulting evidential (Normal-Inverse-Gamma) distribution, from which both aleatoric and epistemic uncertainty are derived. A regulariser penalises predicted evidence that is misaligned with the correct output. The method requires neither sampling at inference nor out-of-distribution examples during training. The authors report well-calibrated uncertainty on various benchmarks, scaling to complex computer-vision tasks, and robustness to adversarial and out-of-distribution test samples; the abstract gives no numbers, and the full paper’s depth-estimation experiments should be consulted for figures. As with evidential classification (Sensoy 2018), the epistemic interpretation depends on the regulariser and has attracted later criticism. For the orchestration framework deep evidential regression is a strong candidate confidence signal for continuous outputs, such as depth, grasp pose or value estimates in the RL module, because it delivers both uncertainty types in one pass and so fits a Raspberry-Pi-class budget.

**Gawlikowski et al. (2023) (R; abstract).** Gawlikowski et al. Gawlikowski et al. [53] provide a comprehensive overview of uncertainty estimation in neural networks for readers without prior background. They introduce the main sources of uncertainty and their separation into reducible model uncertainty and irreducible data uncertainty, then review modelling approaches based on deterministic networks, Bayesian neural networks, ensembles and test-time data augmentation. They also discuss uncertainty measures, calibration methods, existing baselines and implementations, and the practical limitations of current methods for mission- and safety-critical applications. For the orchestration framework the survey is a reference for selecting and justifying the confidence estimators fused by the orchestrator.

**Ren et al. (2024) (E4; abstract).** Ren et al. Ren et al. [54] study embodied question answering, in which a robot must explore until it is confident of an answer. Vision-language models (VLMs) offer semantic reasoning but lack an internal scene memory and have miscalibrated confidence, causing premature stopping or over-exploration. The method builds a semantic map from depth and visual prompting of a VLM to guide exploration, and applies conformal prediction to calibrate the VLM’s answer confidence, giving a principled stopping rule. The authors contribute an EQA dataset of diverse, realistic human-robot scenarios built on HM3D. Simulated and real-robot experiments show gains in performance and efficiency over baselines that do not use the VLM for exploration or do not calibrate confidence. Reliance on a large VLM implies substantial compute. For the framework, conformal calibration of a foundation model’s confidence to decide when to stop gathering information is a directly transferable mechanism for the orchestrator.

## Reinforcement learning and policy variance

**Mnih et al. (2015) (E3; standard reference).** Mnih et al. Mnih et al. [55] address learning control policies directly from high-dimensional sensory input. They introduce the deep Q-network (DQN), which combines Q-learning with a convolutional network mapping raw pixels to action values, stabilised by experience replay and a periodically updated target network. The same architecture and hyperparameters are applied, from pixels and game score alone, across a suite of Atari 2600 games in the Arcade Learning Environment, where the agent is reported to reach a level comparable to a professional human games tester on many of the 49 games studied. DQN is restricted to discrete actions, is sample-inefficient and yields point estimates of value without uncertainty. For the framework it is a foundational RL baseline, and its experience replay and target-network mechanisms underpin the replay component of online improvement.

**Osband et al. (2016) (E3; abstract).** Osband et al. Osband et al. [56] address efficient exploration in complex reinforcement-learning environments, where dithering strategies such as epsilon-greedy explore inefficiently. They propose bootstrapped DQN, which maintains randomised value functions through multiple Q-value heads on a shared network, each trained on a bootstrapped subset of the data; at the start of each episode one head is sampled and followed, yielding temporally extended, or deep, exploration in a computationally and statistically efficient manner. The authors argue that such exploration can lead to exponentially faster learning than dithering, and demonstrate the benefits in complex stochastic Markov decision processes and in the large-scale Arcade Learning Environment, where bootstrapped DQN substantially improves learning times and performance across most Atari games. The abstract gives no per-game figures. The approach retains DQN’s restriction to discrete actions, and head diversity depends on bootstrap masks and random

initialisation rather than a principled posterior. For the orchestration framework bootstrapped DQN is significant because disagreement between heads provides a cheap epistemic-uncertainty estimate over values, a natural policy-variance signal for the RL module; with a shared torso the additional cost is only that of extra small output heads, compatible with Raspberry Pi 5 inference, and it bridges the ensemble uncertainty of Lakshminarayanan 2017 with value-based control.

**Schulman et al. (2017) (E3; abstract).** Schulman et al. [57] propose proximal policy optimisation (PPO), a family of policy-gradient methods that alternate between sampling data through environment interaction and optimising a surrogate objective by stochastic gradient ascent. Unlike standard policy-gradient methods, which perform one gradient update per sample, PPO’s objective, in its best-known form a clipped probability ratio, permits multiple epochs of minibatch updates on the same data. PPO retains some benefits of trust region policy optimisation while being simpler to implement, more general and empirically more sample-efficient. On simulated robotic locomotion and Atari, PPO outperforms other online policy-gradient methods and balances sample complexity, simplicity and wall-clock time. As an on-policy method it remains sample-hungry for real-robot learning. For the framework PPO is a standard RL baseline, and its stochastic policy’s entropy and variance offer a readily available policy-variance signal.

**Andrychowicz et al. (2017) (E4; abstract).** Andrychowicz et al. [58] tackle sparse rewards, one of the central difficulties of reinforcement learning, which otherwise demand careful reward engineering. They introduce Hindsight Experience Replay (HER), which enables sample-efficient learning from sparse, binary rewards by replaying stored episodes with substituted goals, typically states actually achieved later in the episode, so that failed attempts become informative examples of reaching some goal. HER can be combined with any off-policy RL algorithm and can be viewed as an implicit curriculum. The approach is demonstrated on manipulation with a simulated robotic arm across three tasks, pushing, sliding and pick-and-place, each using only a binary completion reward. Ablations show HER to be a crucial ingredient that makes training possible in these environments, and policies trained in physics simulation are deployed on a physical robot and complete the task. The abstract gives no success rates, and the real-robot demonstration is limited in scope. HER presumes goal-conditioned tasks with a known goal representation and a reward function that can be recomputed for substituted goals. For the orchestration framework HER is directly relevant to the replay component of online improvement: on LeKiwi, failed manipulation attempts could be relabelled to populate the replay buffer of the RL module, extracting learning signal from episodes the orchestrator flags as failures.

**Haarnoja et al. (2018) (E3; abstract).** Haarnoja et al. [59] identify two obstacles to applying model-free deep reinforcement learning to real-world domains: very high sample complexity and brittle convergence that demands meticulous hyperparameter tuning. They propose soft actor-critic (SAC), an off-policy actor-critic algorithm within the maximum-entropy RL framework, in which the actor maximises expected reward while also maximising entropy, that is, it seeks to succeed at the task while acting as randomly as possible. Whereas earlier maximum-entropy methods were formulated as Q-learning, SAC combines off-policy updates with a stable stochastic actor-critic formulation. On a range of continuous-control benchmark tasks it achieves state-of-the-art performance, outperforming prior on-policy and off-policy methods, and, unlike other off-policy algorithms, is very stable, with similar performance across random seeds. The abstract gives no numerical returns, and evaluation is confined to simulated benchmarks in this paper. The entropy temperature is a sensitive hyperparameter in this original formulation. For the orchestration framework SAC is the default off-policy learner for a data-efficient RL module on a real robot, and its stochastic Gaussian policy exposes a per-state policy variance and entropy that the orchestrator can read directly as a confidence signal; twin critics, as in Fujimoto 2018, further supply a value-disagreement signal.

**Fujimoto et al. (2018) (E3; abstract).** Fujimoto, van Hoof and Meger [60] show that the overestimation bias arising from function-approximation errors in value-based methods such as deep Q-learning persists in actor-critic settings, and propose mechanisms to reduce its effect on both actor and critic. Their algorithm, known as TD3, builds on Double Q-learning by taking the minimum value of a pair of critics to limit overestimation, connects target networks with overestimation bias, and delays policy updates to reduce per-update error. On the OpenAI Gym suite of tasks it outperforms the state of the art in every environment tested. The abstract gives no figures, and the policy is deterministic, so it offers no native action variance. For the framework TD3 is a strong continuous-control RL baseline, and the discrepancy between its twin critics is an inexpensive value-uncertainty signal for the orchestrator.

**Kalashnikov et al. (2018) (E5; abstract).** Kalashnikov et al. [61] study vision-based dynamic manipulation with scalable reinforcement learning, using grasping as the problem domain. Rather than selecting a grasp point and executing it open-loop, their method performs closed-loop control, continuously updating the grasp strategy from the latest observations to optimise long-horizon success. They introduce QT-Opt, a scalable self-supervised framework that uses over 580k real-world grasp attempts to train a deep Q-function with over 1.2M parameters, with continuous actions

selected by stochastic optimisation over the Q-function. The resulting policy, using only RGB input from an over-the-shoulder camera, reaches 96% grasp success on unseen objects and learns regrasping, probing, repositioning and other non-prehensile pre-grasp behaviours, responding dynamically to disturbances. The data requirement is far beyond a single low-cost platform. For the framework QT-Opt is an evaluation reference for real-world vision-based RL and a caution on the data cost of learning manipulation by RL alone.

**Burda et al. (2019) (E3; abstract).** Burda et al. [62] introduce an exploration bonus for deep reinforcement learning that is easy to implement and adds minimal computational overhead. The bonus, random network distillation (RND), is the error of a neural network trained to predict the features of observations produced by a fixed, randomly initialised network; the error is high for novel observations and falls as they are revisited. They also introduce a method for flexibly combining intrinsic and extrinsic rewards. The combination enables significant progress on several hard-exploration Atari games and state-of-the-art performance on Montezuma’s Revenge, where the authors report the first better-than-average-human performance without demonstrations or access to the underlying game state, occasionally completing the first level. The evaluation is confined to Atari. For the framework the RND prediction error is a cheap, single-pass novelty signal that the orchestrator could use to detect unfamiliar states and trigger exploration or fault handling.

**Kumar et al. (2020) (E3; abstract).** Kumar et al. [63] address offline reinforcement learning, in which policies are learned from static, previously collected datasets without further interaction, and where standard off-policy methods fail through value overestimation induced by distributional shift between the data and the learned policy. They propose conservative Q-learning (CQL), which learns a Q-function whose expected value under a policy lower-bounds the policy’s true value, adding a simple Q-value regulariser to the Bellman error that can be implemented on top of existing Q-learning and actor-critic code. They prove the lower bound and show that CQL can be embedded in policy learning with improvement guarantees. On discrete and continuous control domains CQL substantially outperforms existing offline RL methods, often attaining 2–5 times higher final return, especially on complex, multi-modal data. For the framework CQL is a route to pre-training the RL module from logged LeKiwi data and replay buffers before any online interaction.

**Hafner et al. (2025) (E3; abstract).** Hafner et al. [64] seek a general reinforcement-learning algorithm that can be applied to new domains without extensive per-domain tuning. They present DreamerV3, which learns a world model of the environment and improves behaviour by imagining future trajectories, using robustness techniques based on normalisation, balancing and transformations to learn stably across domains with a single configuration. Across over 150 diverse tasks it outperforms specialised methods, and, applied out of the box, it is the first algorithm to collect diamonds in Minecraft from scratch without human data or curricula, a long-horizon, sparse-reward, pixel-based challenge. Evaluation is in simulated and game environments, and the computational cost of world-model training is substantial. For the framework DreamerV3 is a reference for sample-efficient model-based RL, and world-model prediction error offers a potential signal of when the robot’s dynamics have changed.

**Yuan et al. (2025) (E3; abstract).** Yuan et al. [65] observe that large imitation-learning policies trained on extensive demonstrations are limited by the quantity, quality and diversity of those demonstrations, and ask how such offline-trained models can be improved through online interaction. They introduce Policy Decorator, which attaches a model-agnostic residual policy to a frozen large imitation model and refines its actions during online reinforcement learning. Controlled exploration strategies, which bound the residual’s influence and introduce it progressively, are used to make online learning stable and sample-efficient. The evaluation spans eight tasks across two benchmarks, ManiSkill and Adroit, and two state-of-the-art imitation models, Behavior Transformer and Diffusion Policy. The authors report that Policy Decorator effectively improves the offline-trained policies while preserving the smooth motion characteristic of imitation learning, avoiding the erratic behaviour of pure RL policies; the abstract gives no success-rate figures. All experiments are in simulation, and the approach still requires a reward signal and online interaction budget. For the orchestration framework Policy Decorator models exactly the hand-over between imitation and RL modalities that the orchestrator must manage: the magnitude of the residual correction relative to the base action is itself an informative signal of where the imitation policy’s behavioural fidelity is insufficient and RL refinement is being invoked.

**Ankile et al. (2025) (E3; abstract).** Ankile et al. [66] find that behaviour-cloning (BC) policies, though easy to teach, show unreliable performance that saturates with more data on precision tasks. They attribute this to distribution shift from offline data and to the lack of closed-loop correction caused by action chunking, arguing that chunked BC policies behave like trajectory planners rather than controllers. Their method, ResiP (Residual for Precise Manipulation), augments a frozen chunked BC model with a fully closed-loop residual policy trained by reinforcement learning, which corrects distribution shift and adds per-step corrections to the open-loop chunks, retaining BC’s ease of teaching and long-horizon capability. The abstract gives no success rates, tasks or robot details, which are given only in the full

text. For the framework ResiP is a second residual-RL template for combining imitation and RL modules, and its residual magnitude could indicate behavioural-fidelity shortfalls.

**Wagenmaker et al. (2025) (E4; abstract).** Wagenmaker et al. [67] consider behaviour-cloned (BC) robot policies whose initial performance is unsatisfactory in novel open-world settings; improving them conventionally requires collecting further human demonstrations, whereas reinforcement learning promises autonomous improvement but usually demands too many samples. Focusing on diffusion policies, they propose diffusion steering via reinforcement learning (DSRL), which adapts a BC policy by running RL over its latent-noise space: rather than altering the denoising network, an RL agent chooses the initial noise from which the frozen diffusion policy generates actions. DSRL requires only black-box access to the base policy, avoids the difficulties of fine-tuning diffusion weights, and leaves the base policy unchanged. The authors demonstrate DSRL on simulated benchmarks, on real-world robotic tasks and for adapting pretrained generalist policies, and report high sample efficiency and effective real-world autonomous improvement; the abstract gives no task counts or success rates, which should be taken from the full text. The method still depends on a reward signal, and its reach is bounded by what the base policy can express from some noise input. For the orchestration framework DSRL is attractive because it keeps the imitation module intact while exposing a small, low-dimensional RL interface, well suited to on-device learning, and it defines a clean boundary at which the orchestrator can switch between pure imitation and RL-steered execution.

**Sobol Mark et al. (2025) (E4; abstract).** Sobol Mark et al. [68] observe that most deep reinforcement-learning machinery is co-developed with assumptions about the policy class; for instance, SAC’s reparameterised gradient for Gaussian policies (Haarnoja 2018) is unstable for diffusion policies and intractable for autoregressive categorical ones. They propose policy-agnostic RL (PA-RL), an offline RL and online fine-tuning approach that replaces the policy-improvement step with a universal supervised loss applied to optimised actions. These are obtained by sampling several actions from the base policy, then applying global optimisation, re-ranking candidates by the Q-function, and local optimisation, taking gradient steps on the critic. PA-RL fine-tunes diffusion and transformer policies with tokenised or continuous outputs at different sizes, improving performance and sample efficiency by up to 2x over existing offline RL and online fine-tuning methods. It is reported as the first to fine-tune OpenVLA, a 7B generalist policy, autonomously with Cal-QL, raising real-world success from 40% to 70% in 40 minutes. Fine-tuning a 7B model still requires substantial GPU resources. For the orchestration framework PA-RL shows that a single critic can refine heterogeneous imitation policies, and the spread of Q-values over sampled candidate actions offers a policy-agnostic confidence signal.

**Chen et al. (2025) (E4; abstract).** Chen et al. [69] note that supervised fine-tuning of vision-language-action (VLA) models struggles to reach robust performance when demonstrations are limited and inconsistent, particularly in contact-rich tasks. They propose ConRFT, a reinforced fine-tuning approach comprising offline and online stages under a unified consistency-based training objective. In the offline stage, behaviour cloning and Q-learning are combined to extract a policy from a small demonstration set and to stabilise value estimation. In the online stage, the VLA is further fine-tuned through a consistency policy, with human interventions ensuring safe exploration and high sample efficiency. Evaluated on eight diverse real-world manipulation tasks, ConRFT reaches an average success rate of 96.3% within 45–90 minutes of online fine-tuning, a 144% improvement in success rate and 1.9x shorter episodes than prior supervised methods. The method depends on a human supervisor during online learning, and the GPU requirements of the VLA are not stated in the abstract. For the orchestration framework ConRFT demonstrates a staged imitation-then-RL pipeline with human intervention, which the orchestrator could emulate by invoking the RL modality only after the imitation policy’s fidelity signal falls, and its intervention rate is a natural metric for when autonomy has been achieved.

**Lu et al. (2025) (E3; abstract).** Lu et al. [70] argue that VLAs trained on offline demonstrations fail in out-of-distribution states and propose VLA-RL, a framework for improving pretrained autoregressive VLAs with online reinforcement learning. They formulate trajectory-level RL treating manipulation trajectories as multi-modal, multi-turn conversations, and address sparse rewards by fine-tuning a pretrained vision-language model as a process reward model trained on pseudo-labels from automatically extracted task segments. Stability and efficiency are improved by curriculum selection, GPU-balanced vectorised environments, batch decoding and critic warm-up. VLA-RL enables OpenVLA-7B to exceed the strongest fine-tuned baseline by 4.5% on 40 LIBERO tasks, matching  $\pi 0$ -FAST, and benefits from increased test-time optimisation. Evaluation is simulation-only and GPU-intensive. For the framework, the learned process reward model is a potential progress signal for the orchestrator.

**Li et al. (2026) (E4; abstract).** Li et al. [71] note two limits of supervised fine-tuning for VLA models: the cost of large-scale teleoperated trajectories and weak generalisation under distribution shift. Inspired by reinforcement learning’s success in large reasoning models, they introduce SimpleVLA-RL, an efficient RL framework built on veRL

with VLA-specific trajectory sampling, scalable parallelisation, multi-environment rendering and optimised loss computation, plus exploration-enhancing strategies. Applied to OpenVLA-OFT (Kim 2025), it achieves state-of-the-art results on LIBERO, outperforms  $\pi 0$  on RoboTwin 1.0 and 2.0, and surpasses SFT in real-world tasks. The authors also report ‘pushcut’, a phenomenon in which the policy discovers patterns absent from prior training. The abstract gives no numerical scores, and training is GPU-intensive. For the framework SimpleVLA-RL is an upper-bound reference for RL-refined imitation, but not a design feasible on edge hardware.

## Imitation learning and behavioral fidelity

**Ross et al. (2011) (E2; abstract).** Ross, Gordon and Bagnell [72] observe that imitation learning violates the i.i.d. assumption of statistical learning because future observations depend on the learner’s own past actions, which leads to compounding errors in theory and often in practice. Earlier remedies trained non-stationary or stochastic policies and required many iterations. They propose an iterative algorithm, DAgger, that trains a single stationary deterministic policy and can be viewed as a no-regret online-learning algorithm: the learner’s policy is executed, the expert labels the states visited, and the aggregated dataset is used to retrain. They prove that any such no-regret algorithm, under additional reduction assumptions, finds a policy performing well under its own induced state distribution. The approach outperforms previous methods on two imitation problems and a sequence-labelling benchmark. For the framework DAgger formalises corrective data collection, a template for fault recovery with human or scripted relabelling.

**Ho and Ermon (2016) (E3; abstract).** Ho and Ermon [73] address imitation from expert trajectories without access to a reward signal or further expert queries. Instead of recovering a cost function by inverse RL and then solving an RL problem, they extract a policy directly; one instantiation is analogous to a generative adversarial network, in which a discriminator separating expert from agent state-action pairs supplies a surrogate reward. The resulting model-free algorithm, generative adversarial imitation learning (GAIL), is reported to obtain significant performance gains over existing model-free imitation methods on complex, high-dimensional simulated control environments; the abstract gives no specific figures. GAIL needs many on-policy interactions and adversarial training can be unstable. For the orchestration framework, GAIL represents an imitation modality intermediate between behavioural cloning and RL, and its discriminator score offers a natural, if uncalibrated, measure of behavioural fidelity to the demonstrations.

**Duan et al. (2017) (E3; abstract).** Duan et al. [74] argue that imitation learning is usually applied to tasks in isolation, requiring task-specific feature engineering or many samples, whereas a robot should learn from very few demonstrations and generalise immediately to new instances. They frame this as meta-learning and propose one-shot imitation learning. Training uses a very large family of tasks, each with many instantiations; the running example is block stacking, where tasks differ in the target tower configuration and instances differ in block sets and initial states. A neural network receives one demonstration together with the current state and is trained to output actions so that the resulting state-action sequence matches a second demonstration of the same task. At test time a single demonstration of a new task instance is given and the network must perform new instances of that task. Soft attention over the demonstration and over the scene is reported to let the model generalise to conditions and tasks unseen during training. The abstract reports no quantitative results, and the setting is a simulated block-stacking domain with privileged state; the authors present broad generality as an anticipated outcome of scaling rather than a demonstrated one. For the orchestration framework, it shows how demonstration-conditioned imitation can be meta-learned, although no confidence measure is proposed.

**Osa et al. (2018) (R; abstract).** Osa et al. [75] provide a monograph-length introduction to imitation learning, motivated by the growing difficulty of manually programming robots in unstructured settings. They cover the underlying assumptions, the principal families of approaches and how they relate, the algorithms developed for each, and practical advice on tools and implementation. The text addresses two audiences: machine-learning researchers, for whom it highlights the distinctions between imitation learning and supervised learning or reinforcement learning, particularly in robotics; and roboticists, for whom it surveys the available frameworks. For the orchestration framework it serves as the reference taxonomy for the imitation modality, clarifying when behavioural cloning versus inverse RL is appropriate.

**Lynch et al. (2019) (E3; abstract).** Lynch et al. [76] propose scaling skill acquisition by self-supervising control on human teleoperated play data rather than on segmented task demonstrations. Play is cheap to collect, needing no segmentation, labelling or resets, and is reported to cover roughly 4x more interaction space than task demonstrations for equal collection time. Their method, Play-LMP, learns a latent plan space organising play behaviours and reuses it at test time for goal-conditioned control. In a simulated robotic tabletop environment, the play-supervised model substantially outperforms individual expert-trained policies across 18 user-specified visual manipulation tasks, is more robust to perturbations, exhibits retry-until-success behaviour, and organises its latent space around functional tasks without task labels. Evaluation is simulation-only and goal-image conditioned. For the orchestration framework, play data offers a low-cost

way to build the imitation modality on LeKiwi, and the latent plan distribution is a possible source of behavioural-fidelity signals.

**Florence et al. (2021) (E4; abstract).** Florence et al. [77] examine whether supervised policy learning should use explicit regression models or implicit, energy-based ones. They provide intuitive and theoretical arguments that implicit models better approximate discontinuous and multi-valued functions, and evaluate implicit behavioural cloning with energy-based models (EBMs), where actions are obtained by optimising the energy at inference time, against explicit mean-squared-error and mixture-density policies. Across a range of robot policy learning tasks, including high-dimensional action spaces and image inputs, implicit policies often outperform explicit ones, and on D4RL human-expert tasks they are competitive with or better than state-of-the-art offline RL despite using no reward. On real robots, implicit policies learn contact-rich behaviours from human demonstrations, including tasks requiring 1 mm precision. Inference by optimisation is costlier than a forward pass. For the framework, IBC is an imitation baseline, and its energy landscape offers an interpretable measure of action plausibility usable as a confidence signal.

**Jang et al. (2021) (E5; abstract).** Jang et al. [78] study whether scaling and broadening imitation data enables a vision-based manipulation system to generalise to novel tasks. They build an interactive imitation learning system (BC-Z) that learns from both demonstrations and human interventions, and conditions the policy on task embeddings derived either from pre-trained natural-language models or from videos of humans performing the task. After scaling real-robot data collection to more than 100 distinct tasks, the system performs 24 unseen manipulation tasks with an average success rate of 44%, without any robot demonstrations for those tasks. The absolute success rate on unseen tasks remains modest and the approach depends on a large, expensive data-collection effort. For the orchestration framework, BC-Z illustrates intervention-based data aggregation and language-conditioned imitation, and its intervention mechanism is a template for when a supervisor should hand control back to a human or recovery module.

**Chi et al. (2023) (E4; abstract).** Chi et al. [79] propose Diffusion Policy, which represents a visuomotor robot policy as a conditional denoising diffusion process over action sequences. Instead of regressing actions directly, the policy learns the gradient of the action-distribution score function and, at inference, iteratively refines sampled noise into actions through a stochastic Langevin-dynamics-style denoising procedure. The authors argue that this formulation handles multimodal action distributions gracefully, scales to high-dimensional action spaces and trains stably. Further technical contributions are receding-horizon control over predicted action sequences, visual conditioning of the denoiser, and a time-series diffusion transformer architecture. Across 12 tasks drawn from four manipulation benchmarks, Diffusion Policy consistently outperforms existing state-of-the-art robot learning methods, with a reported average improvement of 46.9%; code, data and training details are released. The principal cost is inference latency, since each action chunk requires multiple denoising iterations, and the approach remains data-hungry relative to its task scope. For the orchestration framework, Diffusion Policy is the de facto imitation baseline in the LeRobot stack; the spread of sampled action trajectories across denoising runs provides a natural behavioural-uncertainty signal, which later runtime monitors exploit ([90], [96]).

**Zhao et al. (2023) (E4; abstract).** Zhao et al. [80] target fine bimanual manipulation tasks, such as threading cable ties or slotting a battery, which demand precision, careful coordination of contact forces and closed-loop visual feedback. They present a low-cost teleoperation and robot system and a learning algorithm, Action Chunking with Transformers (ACT), which learns a generative model over sequences of future actions rather than single-step actions, reducing compounding error. Trained from teleoperated demonstrations, the system achieves 80–90% success on six complex real-world tasks using only ten minutes of demonstrations per task. Evaluation is restricted to one bespoke low-cost platform and a small task set. For the orchestration framework, ACT is directly relevant: it is a standard LeRobot policy for SO-100/101-class arms, is compact enough to consider for edge inference, and its latent-variable decoder could expose action variance as a behavioural-fidelity signal.

**Wu et al. (2024) (E3; full text).** Wu et al. [81] note that large-scale robot policies trained on diverse tasks and platforms show promise for general-purpose robots, but generalisation to new environmental conditions remains unreliable. They propose uncertainty-aware deployment of pre-trained, language-conditioned imitation learning agents. The models are first calibrated post hoc by temperature scaling. At deployment, rather than executing the single most probable action, the calibrated action distribution is used to make uncertainty-aware decisions by aggregating local information over candidate actions. The method is implemented in simulation with three pre-trained models, PerAct, RVT and CLIPort, on the RL Bench and Ravens benchmarks. According to the full text, success rates rise from 0.382 to 0.414 for PerAct and from 0.602 to 0.623 for RVT, and CLIPort’s mean reward rises from 0.803 to 0.833. Gains are larger under distractors: stack-blocks rises from 79.63% to 97.11% and assembling-kits from 48.32% to 55.44%. Improvements on nominal tasks are modest, the approach is restricted to policies with discretised action distributions, and all experiments are simulated.

For the orchestration framework, the paper shows that simple post-hoc calibration makes an imitation policy’s output distribution a usable behavioural-confidence signal, a lightweight input suitable for an edge orchestrator.

## Fault detection, adaptation and recovery

**Bongard et al. (2006) (E4; standard reference).** Bongard et al. [82] propose that a robot can recover from unanticipated damage by continuously maintaining an internal model of its own body. The robot alternates between generating candidate self-models consistent with sensor data, selecting actions that best discriminate between competing models, and using the current best model to synthesise new behaviours. Demonstrated on a legged physical robot, the approach allows the machine to infer its own morphology from its actions and, after part of a limb is removed, to update the self-model and generate compensatory locomotion. The robot and damage scenarios are narrow, and the method relies on offline behaviour synthesis on the updated model. For the orchestration framework, the work is an early exemplar of fault adaptation driven by disagreement among competing models, a form of epistemic uncertainty that could trigger the recovery modality.

**Cully et al. (2015) (E4; standard reference).** Cully et al. [83] address how robots operating outside controlled environments can keep functioning after damage without diagnosing its cause. Their Intelligent Trial and Error approach proceeds in two stages. Before deployment, a quality-diversity algorithm (MAP-Elites) explores a simulated model of the intact robot and builds a behaviour-performance map containing many diverse, high-performing behaviours indexed by low-dimensional descriptors. After damage, the robot uses this map as a prior for Bayesian optimisation with a Gaussian process, trying a small number of behaviours on the physical system, updating predicted performance from each trial, and stopping once a behaviour close to the predicted best is found. The authors demonstrate the method on a physical hexapod robot subjected to several damage conditions, such as broken or missing legs, and on a robotic arm, reporting that compensatory behaviours are found rapidly from a small number of physical trials. Limitations include the need for a reasonably accurate simulator and a hand-chosen behaviour descriptor, and adaptation is to a static new condition rather than continual change. For the orchestration framework, the method is a strong reference for the fault-adaptation modality: the Gaussian-process posterior variance over the prior map is a calibrated confidence signal that could indicate when recovery has succeeded, and the offline map fits a compute-budgeted edge deployment.

**Tobin et al. (2017) (E4; abstract).** Tobin et al. [84] address the reality gap between simulated and physical robotics by domain randomisation: rendering parameters in the simulator are randomised so extensively that the real world appears to the model as just another variation. They focus on object localisation as a stepping stone towards manipulation. A detector trained only on simulated RGB images with non-realistic random textures achieves real-world accuracy of 1.5 cm and is robust to distractors and partial occlusions; the detectors are used to grasp in clutter. The authors describe this as the first successful transfer of a deep network trained solely on simulated RGB images, without real-image pre-training, to real-world robotic control. The study is limited to localisation rather than end-to-end control. For the framework, domain randomisation is a cheap route to training perception and fault-recovery components off-board, and randomisation ranges define the region in which the perception module’s confidence can be trusted.

**Khalastchi and Kalech (2018) (R; standard reference).** Khalastchi and Kalech [85] survey fault detection and diagnosis in robotic systems. The review considers why robots are particularly susceptible to faults, arising in hardware, software and interaction with the environment, and organises detection and diagnosis methods into broad families, including data-driven, model-based and knowledge-based approaches, discussing their assumptions and suitability for different robot types. It predates the recent wave of deep-learning and foundation-model failure detectors. For the orchestration framework, the survey supplies the classical taxonomy against which the fault-adaptation modality, and learned detectors such as [90] and [96], should be positioned.

**Nagabandi et al. (2019) (E4; abstract).** Nagabandi et al. [86] note that real-world reinforcement learning faces two obstacles: samples are expensive, and unexpected perturbations or unseen situations cause proficient but specialised policies to fail at test time. Since training separate policies for every situation is impractical, they propose learning to adapt online. To keep adaptation sample-efficient they work within model-based RL: meta-learning is used to train a prior over dynamics models such that, combined with a small window of recent experience, the prior can be rapidly adapted to the local context, and the adapted model is used for planning. Two adaptation variants are described in the paper, a gradient-based and a recurrent one. Simulated agents adapt online to novel terrains, crippled body parts and highly dynamic environments. On a real dynamic legged millirobot, the learned model adapts quickly to a missing leg, novel terrains and slopes, miscalibration or pose-estimation errors, and pulling payloads. The abstract reports no quantitative figures, and the real-world validation uses a single small robot. For the orchestration framework, this work is a central

reference for the fault-adaptation modality: it shows that meta-learned priors plus short-horizon online updates can absorb faults cheaply, and the prediction error of the adapted dynamics model is a natural trigger signal for invoking recovery.

**Pang et al. (2021) (R; abstract).** Pang et al. [87] survey deep anomaly detection, also known as outlier or novelty detection, which they argue still presents distinctive complexities requiring advanced approaches. They propose a taxonomy of three high-level categories and 11 fine-grained categories of deep methods, and for each review key intuitions, objective functions, underlying assumptions, advantages and disadvantages, and how the methods address the identified challenges, before outlining future opportunities. The survey is domain-general rather than robotics-specific. For the orchestration framework it provides the methodological vocabulary for the fault-detection and out-of-distribution signals that should trigger the recovery modality.

**Kumar et al. (2021) (E4; abstract).** Kumar et al. [88] argue that real-world legged robots must adapt in real time to unseen conditions such as changing terrain, changing payloads, and wear and tear. They present Rapid Motor Adaptation (RMA), which combines a base policy with an adaptation module. In simulation, the base policy is trained with privileged access to environment parameters encoded as an extrinsics vector; the adaptation module is then trained to estimate that vector from the recent history of states and actions, so that at deployment the policy can adapt without privileged information. Training is performed entirely in simulation on a varied terrain generator with bioenergetics-inspired rewards and without domain knowledge such as reference trajectories or foot-trajectory generators. The policy is deployed on the Unitree A1 quadruped without any fine-tuning and adapts to novel situations in fractions of a second across rocky, slippery and deformable surfaces, grass, long vegetation, concrete, pebbles, stairs and sand. The authors report state-of-the-art performance across real-world and simulated experiments; the abstract gives no further figures. The work concerns locomotion rather than manipulation, and adaptation is limited to the range of variation covered in simulation. For the orchestration framework, RMA exemplifies fast, implicit fault adaptation using a lightweight history encoder, and discrepancies between estimated extrinsics and nominal values could serve as a fault-detection signal.

**Liu et al. (2023) (E4; abstract).** Liu et al. [89] argue that automatic detection and analysis of failed executions is essential for explainable and robust robots, and exploit the textual reasoning abilities of large language models for this purpose. Their framework, REFLECT, converts multisensory observations (the paper uses RGB-D vision, audio and robot state) into a hierarchical summary of the robot’s past experience, from which an LLM is queried to explain the cause of a failure. The explanation is then passed to a language-based planner, which generates a corrective plan so that the task can be completed. To evaluate the approach systematically the authors introduce the RoboFail dataset, comprising a variety of tasks and failure scenarios. They report that the LLM-based framework produces informative failure explanations that assist successful correction planning; the abstract gives no quantitative results. The pipeline depends on reliable perception-to-language conversion and on a large, typically cloud-hosted LLM, and it analyses failures after the fact rather than predicting them. For the orchestration framework, REFLECT is the closest reference for an LLM-mediated recovery modality: its hierarchical experience summary is a design pattern for feeding structured state to a local orchestrator, though running it with a small on-device LLM would need to be tested.

**Agia et al. (2024) (E4; abstract).** Agia et al. [90] observe that imitation-learned robot policies fail when conditions deviate from their training data and that runtime monitors providing early failure warnings are needed for scalable deployment. They propose Sentinel, which splits failure detection into two complementary categories. Erratic failures are detected with statistical measures of temporal action consistency, comparing action distributions predicted by a generative policy at successive time steps; this is fast and computationally negligible. Task-progression failures, in which the policy confidently and consistently takes actions that do not solve the task, are detected by querying a vision-language model, which is reserved for less time-sensitive failure modes. The approach is demonstrated with diffusion policies on mobile manipulation domains in simulation and the real world. Combining the two detectors detects 18% more failures than either detector alone and significantly outperforms baselines. Limitations include reliance on a large VLM for progress monitoring and evaluation on a limited set of domains. For the orchestration framework, Sentinel is a direct template: temporal action consistency is a cheap, Pi-feasible behavioural-fidelity signal for the imitation modality, while VLM progress checks map to a slower, budgeted orchestrator query.

**Sinha et al. (2024) (E3; abstract).** Sinha et al. [91] consider how foundation models, whose zero-shot generalisation makes them promising for detecting out-of-distribution failures, can be used online despite their computational cost and integrated safely into control. They propose a two-stage framework. A fast binary anomaly classifier operates on observations embedded in an LLM embedding space; when it flags an anomaly, a slower stage uses a generative LLM to reason about and select a fallback. These stages correspond to branch points in a model predictive control scheme that maintains joint feasibility of several fallback plans, so the system remains safe during the slow reasoner’s latency. The fast classifier is reported to outperform autoregressive reasoning with state-of-the-art GPT models even when instantiated with

relatively small language models. Experiments concern dynamic systems such as quadrotors and autonomous vehicles, largely in simulation, and the abstract gives no numerical results. The design of a cheap embedding-based detector gating an expensive reasoner is closely analogous to the budgeted orchestration proposed in this framework, where a small local model triggers costlier modalities only when needed.

**Shi et al. (2024) (E4; abstract).** Shi et al. [92] note that hierarchical policies combining language-level planning with low-level control struggle to reach high success rates on long-horizon dexterous tasks, since failure of any stage compromises the whole. They observe that a high-level policy indexing into sufficiently rich, language-conditioned low-level skills can be supervised directly through human language corrections, including fine-grained ones such as ‘move a bit to the left’, obtained from people occasionally observing the robot. The framework adapts in real time to verbal feedback and additionally incorporates corrections into iterative retraining of the high-level policy, improving its ability to correct both low-level execution and high-level decision errors. Evaluation on real hardware shows significant improvement in long-horizon dexterous manipulation without additional teleoperation; the abstract reports no figures. For the framework, the work illustrates a human-in-the-loop recovery channel and online improvement of a high-level selector from sparse corrections, closely related to updating a local orchestrator.

**Duan et al. (2025) (E4; abstract).** Duan et al. [93] argue that although vision-language and large language models have improved robots’ spatial reasoning and problem solving, they remain weak at recognising failures, which limits real-world applicability. They introduce AHA, an open-source VLM that frames failure detection as free-form natural-language reasoning, identifying failures and explaining them across robots, tasks and environments. AHA is fine-tuned on the AHA dataset, generated by FailGen, a scalable framework that procedurally perturbs successful simulated demonstrations to produce what the authors describe as the first large-scale dataset of robotic failure trajectories. Although trained only on this simulated data, AHA generalises to real-world failure datasets, other robotic systems and unseen tasks. It surpasses the second-best model, GPT-4o with in-context learning, by 10.3%, and exceeds the average of six compared models, including five state-of-the-art VLMs, by 35.3% across multiple metrics and datasets. Integrated into three manipulation frameworks using LLMs or VLMs for reinforcement learning, task and motion planning, and zero-shot trajectory generation, AHA’s feedback refines dense rewards, improves task planning and sub-task verification, raising task success by an average of 21.4% relative to GPT-4 models. A VLM of this size remains costly to run onboard. For the orchestration framework, AHA is a candidate failure-detection signal source and a benchmark for the fault-recovery modality, and FailGen suggests how to synthesise failure data for training a local detector.

**Ye et al. (2025) (E4; abstract).** Ye et al. [94] note that vision-language-action models are trained mainly on successful demonstrations and lack structured supervision for diagnosing and recovering from failures. They propose the RoboFAC framework and construct a failure-centric dataset of 9,440 erroneous manipulation trajectories and 78,623 question-answer pairs across 53 scenes in simulation and the real world, with systematically categorised failure types. On this dataset they train a lightweight multimodal model for task understanding, failure analysis and failure correction, designed for efficient local deployment. RoboFAC achieves 34.1% higher failure-analysis accuracy than GPT-4o, and, used as an external supervisor in a real-world VLA pipeline, yields a 29.1% relative improvement across four tasks with markedly lower latency than GPT-4o. Real-world evaluation covers only four tasks. For the framework, RoboFAC is the closest analogue to a local, lightweight supervisor for the recovery modality and a baseline for on-device failure analysis.

**Römer et al. (2025) (E4; abstract).** Römer et al. [95] address runtime failure prediction for generative imitation-learning policies based on diffusion or flow matching, which can behave unpredictably under distribution shift from unseen environments or compounding action errors. They propose FIPER, a framework for Failure Prediction at Runtime that requires no failure data. FIPER uses two indicators of impending failure: out-of-distribution observations, detected by random network distillation in the policy’s embedding space, and high uncertainty in generated actions, measured by a new action-chunk entropy score. Both scores are calibrated with conformal prediction using a small set of successful rollouts, and an alarm is raised only when both indicators, aggregated over short time windows, exceed their calibrated thresholds. This conjunction is intended to separate genuine failures from benign OOD situations. Evaluation spans five simulated and real-world environments with diverse failure modes, where FIPER distinguishes actual failures from benign OOD situations better and predicts failures more accurately and earlier than existing methods; the abstract provides no numerical results. Limitations include dependence on representative successful calibration rollouts and the assumption that failures co-occur with both signals. For the orchestration framework, FIPER is directly reusable: its two calibrated scores correspond to perceptual epistemic uncertainty and imitation behavioural variance, and conformal thresholds offer a principled way to fuse them into switching decisions.

**Xu et al. (2025) (E4; abstract).** Xu et al. [96] observe that as imitation policies based on diffusion and flow models tackle longer and more complex tasks, they exhibit diverse failure modes that are hard to anticipate, and that most existing

detectors require prior knowledge of failure modes and failure data for training. They present FAIL-Detect, a modular two-stage approach that frames failure detection as sequential out-of-distribution detection using only successful training data. In the first stage, policy inputs and outputs are distilled into scalar signals that correlate with failure and capture epistemic uncertainty; in the second, conformal prediction supplies thresholds with statistical guarantees. The authors compare learned and post-hoc scalar signal candidates on diverse robotic manipulation tasks and find learned signals to be mostly consistently effective, especially their new flow-based density estimator. FAIL-Detect detects failures more accurately and faster than state-of-the-art baselines; no numbers are stated in the abstract. The approach assumes successful training data adequately represent nominal behaviour, and OOD does not always imply failure. For the orchestration framework, FAIL-Detect provides a systematic catalogue of cheap, calibrated scalar signals for the imitation modality and a conformal recipe for turning them into switching thresholds.

**Gu et al. (2025) (E4; abstract).** Gu et al. [97] note that vision-language-action models achieve limited success on novel tasks out of the box and therefore need failure detectors that raise timely alerts so the robot can stop, backtrack or ask for help. Existing detectors are trained and tested on one or a few tasks, whereas generalist VLAs require detection that transfers to unseen tasks and environments. The authors formalise this multitask failure-detection problem and propose SAFE. Analysing the VLA feature space, they find that internal features encode high-level knowledge of task success and failure that is generic across tasks. SAFE learns from these internal features to predict a single scalar indicating failure likelihood; it is trained on successful and failed rollouts and evaluated on unseen tasks, and is compatible with different policy architectures. Experiments with OpenVLA,  $\pi 0$  and  $\pi 0$ -FAST in simulated and real-world environments show state-of-the-art failure detection and the best trade-off between accuracy and detection time when thresholds are set by conformal prediction; the abstract gives no numbers. Unlike failure-data-free methods, SAFE requires failed rollouts for training. For the orchestration framework, SAFE shows that a policy’s own hidden features can yield a cheap, task-general confidence scalar, a strong candidate input for the orchestrator’s modality-switching decisions.

## Meta-learning

**Duan et al. (2016) (E3; standard reference).** Duan et al. [98] observe that deep reinforcement learning typically requires very many trials, whereas animals learn new tasks from few, benefiting from prior knowledge. Their  $RL^2$  approach represents the reinforcement learning algorithm itself as a recurrent neural network whose weights are learned from data with a general-purpose, slow RL algorithm. The RNN receives observations, actions, rewards and termination flags, and its hidden state is retained across episodes within a task, so that the fast learning procedure for a new task is carried out implicitly by the network’s activations rather than by gradient updates. Training therefore occurs over a distribution of tasks, such as multi-armed bandits, tabular Markov decision processes and visual navigation problems, and at test time the network adapts to a new task within a few episodes. The authors compare the learned behaviour against hand-designed algorithms with theoretical guarantees on the small problems and study scalability on vision-based navigation. The fetched page did not provide the full abstract, so no quantitative results are cited here. Known limitations include difficulty scaling to long horizons and the need for a well-specified task distribution. For the orchestration framework,  $RL^2$  is the canonical example of context-based meta-RL, suggesting that an orchestrator’s selection behaviour could itself be meta-learned in-context from interaction histories without weight updates on the edge device.

**Andrychowicz et al. (2016) (E2; abstract).** Andrychowicz et al. [99] note that while machine learning has moved from hand-designed to learned features, optimisation algorithms are still designed by hand. They cast optimiser design as a learning problem: an LSTM, applied coordinate-wise to gradients, outputs parameter updates and is trained so that the optimisee’s loss decreases rapidly, allowing the optimiser to exploit structure in the problems of interest. The learned optimisers outperform generic, hand-designed competitors on the tasks for which they are trained and generalise to new tasks with similar structure. Demonstrations include simple convex problems, training neural networks and neural-art image styling; the abstract gives no figures. Generalisation beyond the training distribution of problems is limited. For the framework, learned optimisers are a meta-learning route to efficient on-device adaptation, for instance of LoRA parameters, though the added meta-training cost must be justified against standard optimisers.

**Finn et al. (2017) (E3; abstract).** Finn et al. [100] propose model-agnostic meta-learning (MAML), a meta-learning algorithm compatible with any model trained by gradient descent and applicable to classification, regression and reinforcement learning. The objective of meta-learning is to train on a variety of tasks so that new tasks can be solved from few samples. MAML explicitly optimises the initial parameters so that a small number of gradient steps on a small amount of data from a new task produce good generalisation on that task; in effect, the model is trained to be easy to fine-tune. This requires differentiating through the inner-loop updates, involving second-order derivatives, although a first-order approximation is also considered. The authors report state-of-the-art performance on two few-shot image-classification benchmarks, good results on few-shot regression, and accelerated fine-tuning for policy-gradient RL with neural-network

policies; specific accuracies are not given in the abstract. Limitations include the computational and memory cost of second-order meta-gradients, sensitivity to hyperparameters, and the assumption that test tasks share structure with the training distribution. For the orchestration framework, MAML is the canonical gradient-based meta-learning method: meta-training an orchestrator or LoRA adapter initialisation offline so that a few on-device gradient steps suffice is a direct application, keeping online compute within the Pi 5 budget.

**Snell et al. (2017) (E3; abstract).** Snell et al. [101] address few-shot classification, in which a classifier must generalise to classes unseen during training from only a few labelled examples of each. Prototypical networks learn an embedding space in which each class is represented by a prototype, the mean of its embedded support examples, and queries are classified by their distances to these prototypes. The authors argue that this simpler inductive bias suits the limited-data regime and show that some simple design decisions yield substantial improvements over recent approaches relying on complicated architectures and meta-learning. The method achieves excellent few-shot results and, extended to zero-shot learning, state-of-the-art results on the CU-Birds dataset; the abstract gives no figures. Its assumptions of a fixed embedding and unimodal classes limit expressiveness. For the framework, prototype distances provide a cheap, interpretable novelty or confidence signal for the perception modality and a lightweight way to add new object classes on-device.

**Nichol et al. (2018) (E2; abstract).** Nichol et al. [102] study meta-learning for a distribution of tasks, where the goal is an agent that learns quickly on an unseen task drawn from that distribution. They analyse a family of algorithms that learn a parameter initialisation suitable for fast fine-tuning while using only first-order derivatives in the meta-update. The family includes and generalises first-order MAML, which ignores second-order terms, and introduces Reptile, which repeatedly samples a task, trains on it with ordinary gradient descent, and moves the initialisation towards the resulting weights. Extending earlier results by [100], they show that first-order methods perform well on established few-shot classification benchmarks, and provide theoretical analysis of why they work; the abstract gives no figures. Results are limited to supervised few-shot benchmarks. For the framework, Reptile’s simplicity and low memory cost make it the most plausible meta-learning procedure for adapter initialisation within an edge compute budget.

**Yu et al. (2019) (E3; abstract).** Yu et al. [103] argue that meta-reinforcement learning research relies on overly narrow task distributions, such as varying running velocities for a simulated robot, which cannot support generalisation to entirely new behaviours. They propose Meta-World, an open-source simulated benchmark for meta-RL and multi-task RL comprising 50 distinct robotic manipulation tasks, intended to evaluate the acquisition of held-out tasks. Evaluating seven state-of-the-art meta-RL and multi-task algorithms, they find that although each task and its variations, for example with different object positions, can be learned with reasonable success, the algorithms struggle to learn multiple tasks simultaneously, even with as few as ten training tasks. The benchmark is simulation-only with state-based observations by default. For the framework, Meta-World is a candidate simulation testbed for pre-training and evaluating the meta-learning and RL modalities before transfer to LeKiwi.

**Rakelly et al. (2019) (E3; abstract).** Rakelly et al. [104] note that meta-reinforcement learning promises learning new skills from little experience but is impractical because existing methods rely heavily on on-policy data, limiting sample efficiency, and lack mechanisms to reason about task uncertainty, limiting effectiveness under sparse rewards. They develop an off-policy meta-RL algorithm, PEARL, that disentangles task inference from control. An encoder performs online probabilistic filtering of a latent task variable from small amounts of experience, and the policy and value functions, trained with an off-policy actor-critic (soft actor-critic in the paper), are conditioned on this variable. The probabilistic formulation enables posterior sampling of the task variable, giving structured and efficient exploration in new tasks. Integrating the latent task variables with off-policy RL yields efficiency both during meta-training and during adaptation. On several meta-RL benchmarks the method outperforms prior algorithms in sample efficiency by 20–100x and also in asymptotic performance. Evaluation is in simulated continuous-control domains, and the method assumes task distributions resembling those seen during meta-training. For the orchestration framework, PEARL’s posterior over task latents is an explicit task-uncertainty signal: its variance could inform when the RL modality is operating in an unfamiliar context, and the off-policy design is compatible with replay-based on-device improvement.

**Zintgraf et al. (2020) (E3; abstract).** Zintgraf et al. [105] address the exploration-exploitation trade-off in unknown environments. A Bayes-optimal policy handles this optimally by conditioning its actions on both the environment state and its uncertainty about the environment, but computing such a policy is intractable except for the smallest tasks. They introduce variational Bayes-Adaptive Deep RL (variBAD), which meta-learns to perform approximate inference over the unknown task. A variational auto-encoder with a recurrent encoder infers a posterior over a latent task embedding from the interaction history, trained with a decoder that reconstructs rewards and transitions, and the policy conditions on this posterior, so that task uncertainty is incorporated directly into action selection. In a grid-world domain the authors

illustrate structured online exploration as a function of task uncertainty, and on MuJoCo domains widely used in meta-RL variBAD achieves higher online return than existing methods; the abstract reports no figures. The approach remains approximate, is evaluated only in simulation, and depends on a task distribution available at meta-training time. For the orchestration framework, variBAD formalises the principle of acting on explicit uncertainty: its belief posterior is a template for an orchestrator that conditions modality choice on fused, calibrated uncertainty rather than on point estimates.

**Hospedales et al. (2022) (R; abstract).** Hospedales et al. [106] survey meta-learning, or learning to learn, which aims to improve the learning algorithm itself from experience over multiple learning episodes rather than solving each task from scratch with a fixed algorithm. They argue this paradigm can address data and computation bottlenecks and generalisation. The survey defines meta-learning and positions it relative to transfer learning and hyperparameter optimisation, proposes a new taxonomy of methods, reviews applications such as few-shot learning and reinforcement learning, and discusses open challenges. For the orchestration framework it frames the design space for meta-learning the orchestrator’s adaptation procedure within a constrained compute budget.

**Zhu et al. (2025) (E4; full text).** Zhu et al. [107] argue that continual adaptation is essential for general autonomous agents, for example a household robot that must learn tasks specific to each home after pre-training. Building on parameter-efficient fine-tuning from language models, earlier work adapts pretrained policies with lightweight adapters, which preserve pre-trained features but treat each new task independently, limiting knowledge transfer. The authors propose Online Meta-Learned Adapters (OMLA), which replace direct adapter training with a meta-learning objective that transfers knowledge from previously learned tasks to the current one during sequential adaptation. Experiments cover simulated and real-world settings. On the LIBERO benchmark, OMLA’s average forward transfer exceeds that of LoRA on LIBERO-OBJECT (0.86 vs 0.71), LIBERO-SPATIAL (0.69 vs 0.57) and LIBERO-GOAL (0.58 vs 0.52). On a real Kinova arm performing pick-up tasks, OMLA reaches 84.0% success with 20 demonstrations against 60.0% for LoRA, and 48.0% against 34.0% with 10 demonstrations. Real-world evaluation is limited to one arm and simple pick-up tasks, and the compute cost of the meta-objective relative to plain LoRA is not reported in the abstract. For the orchestration framework, OMLA is the most directly relevant method for combining LoRA-style adapters with meta-learning to improve a pretrained policy online, and is a baseline for the proposed on-device adaptation loop.

## Uncertainty fusion and calibration

**McMahan et al. (2017) (E3; abstract).** McMahan et al. [108] observe that data on mobile devices could greatly improve on-device models, for instance language models for speech recognition and text entry, but is often privacy-sensitive, large, or both, which may preclude logging it to a data centre for conventional training. They advocate federated learning, in which training data remain distributed on devices and a shared model is learned by aggregating locally computed updates. They present a practical method for federated learning of deep networks based on iterative model averaging (FederatedAveraging): each selected client performs several local stochastic-gradient epochs on its own data, and the server averages the resulting weights, weighted by client data size. An extensive empirical evaluation covers five model architectures and four datasets and shows that the approach is robust to the unbalanced and non-IID data distributions characteristic of this setting. Treating communication as the principal constraint, the method reduces the number of communication rounds required by 10–100x relative to synchronised stochastic gradient descent. The analysis is empirical, and privacy guarantees are not formally provided by averaging alone. For the orchestration framework, FedAvg offers a cloud-free, or at least data-local, mechanism by which several low-cost robots could pool LoRA or orchestrator updates without sharing raw sensor data, although a coordinating aggregator is still required.

**Kendall et al. (2018) (E3; abstract).** Kendall, Gal and Cipolla [109] address a practical obstacle in multi-task deep learning: performance depends strongly on the relative weighting of the task losses, and hand-tuning these weights is expensive. They propose weighting each loss by the homoscedastic (task-dependent) aleatoric uncertainty of that task, learnt jointly with the network parameters, which yields a principled combination of regression and classification objectives expressed in different units and scales. The method is demonstrated on a single network that predicts per-pixel depth, semantic segmentation and instance segmentation from a monocular RGB image. The abstract reports that the learnt weighting allows the joint model to outperform separate models trained individually on each task; specific metrics and datasets are not given in the abstract. A limitation is that homoscedastic uncertainty is constant per task, not per input, so it captures neither input-dependent noise nor epistemic uncertainty, and the evaluation concerns scene understanding rather than control. For the orchestration framework, the work offers a compact, learnable mechanism for balancing heterogeneous objectives by uncertainty, directly relevant to weighting confidence signals from perception, reinforcement and imitation modules within a single fused score, and to multi-head perception networks that share one backbone on constrained hardware.

**Baltrušaitis et al. (2019) (R; abstract).** Baltrušaitis, Ahuja and Morency [110] survey multimodal machine learning, the study of models that process and relate information from modalities such as vision, language and audio. Instead of organising the field by application, they propose a taxonomy that goes beyond early/late fusion and identifies five core challenges: representation, translation, alignment, fusion and co-learning, reviewing methods under each and outlining open directions. The review reports no experiments and predates large vision-language(-action) models. For the orchestration framework it supplies standard vocabulary for how the orchestrator fuses camera streams, proprioception and module confidence signals on one embodied platform.

**Li et al. (2020) (E2; abstract).** Li and [111] colleagues address two properties that distinguish federated learning from classical distributed optimisation: systems heterogeneity, where devices differ in compute and connectivity, and statistical heterogeneity, where data are not identically distributed across clients. They introduce FedProx, a generalisation and re-parameterisation of FedAvg that adds a proximal term to each local objective and tolerates a variable amount of local work per device. They provide convergence guarantees under non-identical data and device-level constraints, and evaluate on a suite of realistic federated datasets. In highly heterogeneous settings FedProx improves absolute test accuracy by 22% on average relative to FedAvg and converges more stably. The analysis is for supervised learning and does not address embodied or reinforcement settings. For the framework, FedProx is a reference for optional fleet-level sharing of adapters between LeKiwi units without centralising raw data, while the core design remains cloud-free.

**Kairouz et al. (2021) (R; abstract).** Kairouz, McMahan, Avent and [112] more than fifty co-authors survey federated learning, in which many clients, such as mobile devices or organisations, collaboratively train a model under a central server’s orchestration while keeping data decentralised. The monograph reviews advances in optimisation, privacy, robustness and systems, and compiles an extensive list of open problems. It reports no single experimental result, and its perspective is server-centred rather than robotic. For the framework it is the reference account of decentralised learning trade-offs, useful when positioning a strictly local, cloud-free orchestrator against fleet-level alternatives.

**Qi et al. (2021) (R; abstract).** Qi, Zhou and Zheng [113] survey federated reinforcement learning (FRL), which applies the federated principle of sharing model updates rather than raw data to improve RL while preserving privacy. After tutorials on federated learning and RL, they divide FRL into horizontal and vertical variants, give formal definitions, trace the area’s evolution and summarise applications in edge computing, communication, control optimisation and attack detection. The review reports no experiments and says little about physical manipulation. For the framework it maps how RL experience might be pooled across several low-cost robots without a cloud dataset.

**Han et al. (2021) (E3; abstract).** Han, Zhang, Fu and Zhou [114] argue that multi-view classification should not only integrate views to improve accuracy but also assess, per sample, how trustworthy each view is. They propose trusted multi-view classification (TMC), in which each view produces evidence parameterising a Dirichlet distribution over class probabilities, following evidential deep learning, and views are combined at the evidence level using Dempster–Shafer theory. The combination rule down-weights views with high uncertainty and yields an overall uncertainty for the fused decision. The abstract states that experiments validate the method in accuracy, reliability and robustness, including for out-of-distribution samples; datasets and figures are not given in the abstract. A limitation is that the evaluation concerns static classification benchmarks, and Dempster–Shafer combination assumes views provide independent evidence. For the orchestration framework, TMC offers a concrete, lightweight rule for fusing calibrated confidence from several sources, for example the two LeKiwi cameras or the perception, reinforcement and imitation modules, into one trust score with explicit conflict handling, and is a natural baseline for the orchestrator’s signal-fusion step.

**Han et al. (2022) (E3; abstract).** Han, Zhang, Fu and Zhou [115] extend their trusted multi-view classification (TMC) framework into a journal version. The motivation is that multi-view classifiers should ensure reliable integration and decisions for noisy, corrupted and out-of-distribution inputs, which requires assessing each view’s trustworthiness per sample. Each view outputs evidence parameterising a variational Dirichlet distribution over class probabilities; views are then integrated with Dempster–Shafer theory, so that uncertain views contribute less and the fused prediction carries its own uncertainty. The unified learning framework is claimed to induce accurate uncertainty and to confer robustness to noise and corruption, supported by both theoretical and experimental results in accuracy, robustness and trustworthiness; specific benchmark figures are not given in the abstract. As with the conference version, evaluation is on static classification data, and the independence assumptions of evidence combination may not hold for correlated sensors such as two cameras viewing the same scene. For the orchestration framework, the dynamic evidential fusion rule is a directly reusable, low-cost mechanism for combining per-module confidence signals into a single trust estimate, and its explicit conflict term could indicate when perception and policy modules disagree and fault recovery should be invoked.

**Han et al. (2022) (E3; abstract).** Han, Yang, Huang, Zhang and Yao [116] study trustworthy fusion of heterogeneous, high-dimensional modalities, motivated by safety-critical uses such as medical diagnosis from multi-omics data. Their method, Multimodal Dynamics, estimates informativeness at two levels for each sample: a sparse gating mechanism captures the variation in informativeness of individual features within a modality, and the true class probability is used to assess each modality’s classification confidence. A transparent fusion rule then weights modalities according to these dynamically estimated informativeness scores. The authors state that this is the first work to model both feature-level and modality-level variation jointly for trustworthy fusion. Experiments on multimodal medical classification datasets are reported to show superior performance and trustworthiness relative to state-of-the-art methods; specific figures are not given in the abstract. Limitations include the tabular medical domain, far from real-time sensorimotor data, and reliance on a learnt true-class-probability estimator whose own calibration must be trusted. For the orchestration framework, the paper contributes the idea of sample-wise, learnt confidence weighting at two granularities, which maps onto weighting individual signals within a module and whole modules within the orchestrator.

**Lindemann et al. (2023) (E3; abstract).** Lindemann, Cleaveland, Shim and Pappas [117] propose safe planning in unknown dynamic environments with probabilistic guarantees. A model predictive controller uses trajectory predictions of surrounding agents together with prediction regions obtained by conformal prediction from offline trajectory data. Because the regions hold with a user-defined probability, the resulting MPC is provably safe with respect to that probability, and the approach is compatible with arbitrary predictors such as RNNs and LSTMs without distributional assumptions. The method is illustrated in the CARLA driving simulator at a pedestrian-filled intersection. The guarantees rely on exchangeability between calibration and deployment data and were shown only in simulation. For the framework, the work shows how conformal calibration converts any predictor’s output into a statistically valid confidence set, a template for calibrating module signals before the orchestrator acts on them.

**Zollo and Zemel (2025) (E3; abstract).** Zollo and Zemel [118] present what they describe as the first study of confidence calibration in vision-language-action (VLA) foundation models, which map images and language instructions to low-level robot actions. They argue that trustworthy behaviour requires not only task success but a reliable estimate of the probability of success. The study establishes a confidence baseline for VLAs, examines how task success relates to calibration error and how calibration evolves over the course of an episode, and introduces two lightweight remedies for the miscalibration observed: prompt ensembles, which average confidence over paraphrased instructions, and action-wise Platt scaling, which recalibrates confidence separately for each action dimension. The revised version adds experiments with further VLA variants. Benchmarks, models and calibration-error figures are not given in the abstract. Limitations are that calibration is assessed for existing VLAs, typically in simulation, and post-hoc recalibration requires held-out labelled episodes. For the orchestration framework, this work is central: it indicates that raw VLA confidence cannot be used unmodified as an imitation-module signal, and it provides cheap post-hoc calibration steps that could run on the robot before the orchestrator compares confidence across modalities.

**Römer et al. (2026) (E3; abstract).** Römer and [119] colleagues address the absence of confidence estimates in VLAs whose action heads are trained by flow matching, which leaves them prone to failing without warning outside their pretraining distribution. They derive an efficient estimator of epistemic uncertainty, velocity-field disagreement (VFD), computed as the disagreement between the velocity fields of a small ensemble of flow-matching heads. VFD is used for failure detection at deployment and for active fine-tuning through SAVE, a framework for uncertainty-guided active multitask fine-tuning that prioritises which tasks receive costly expert demonstrations. Experiments on the LIBERO benchmark show that VFD yields better-calibrated uncertainty that predicts downstream performance, performs strongly in failure detection, and that SAVE requires at least 22% fewer samples than baselines. Limitations are the simulation-only evaluation reported in the abstract and the added cost of an ensemble of heads. For the orchestration framework, VFD is a principled epistemic-uncertainty signal for a flow-based imitation module, and SAVE exemplifies the uncertainty-driven decision about when to request more data, which the orchestrator must also make under a budget.

**Yuan et al. (2026) (E4; abstract).** Yuan, Wu and Bajcsy [120] study policy steering, in which a learnt verifier selects among action samples proposed by a pretrained policy such as a diffusion policy. They observe that vision-language model verifiers are often overconfident, which degrades steering under both semantic uncertainty in the task specification and low-level incapability of the base policy. Their uncertainty-aware policy steering (UPS) framework reasons jointly about these two sources and selects one of three strategies: execute a high-confidence action, ask a clarifying natural-language question, or request a human action intervention when the policy is judged incapable. Conformal prediction calibrates the composition of the VLM and base policy, giving statistical assurance that the correct strategy is chosen, and residual learning on collected interventions improves the base policy over time. Experiments in simulation and on hardware show that UPS separates confident, ambiguous and incapable situations and requires fewer user interventions than uncalibrated baselines and prior human- or robot-gated continual-learning methods; numeric results are not given in the abstract. It

relies on a human in the loop and a large VLM. For the framework, UPS is the closest competing approach: a calibrated arbiter that chooses between acting, asking and learning, analogous to selecting learning modalities.

## On-device adaptation and continual learning

**Schaul et al. (2016) (E3; abstract).** Schaul, Quan, Antonoglou and Silver [121] observe that experience replay, as used in deep reinforcement learning, samples stored transitions uniformly and therefore replays them at the frequency they were experienced, irrespective of how informative they are. They develop a framework for prioritised experience replay, in which transitions are sampled more often when they are expected to yield more learning progress, so that important experiences are replayed more frequently and learning becomes more efficient. The framework is applied to Deep Q-Networks (DQN) on the Atari benchmark. DQN with prioritised replay achieved a new state of the art at the time, outperforming DQN with uniform replay on 41 of 49 games. Limitations noted in the literature include the bias introduced by non-uniform sampling, which must be corrected, and the additional bookkeeping for priority updates; the evaluation is confined to discrete-action video games rather than robotic control. For the orchestration framework, prioritised replay is a direct candidate for the replay component of online improvement: the orchestrator’s own decision log and the modules’ experience buffers could be sampled by priority, and the priority itself, such as prediction error, is a further signal of where a module is least competent.

**Rusu et al. (2016) (E3; abstract).** Rusu and [122] colleagues address learning sequences of tasks while exploiting transfer and avoiding catastrophic forgetting. Progressive neural networks add a new network column for each task, freeze previously trained columns, and connect them to the new column through lateral connections, so earlier knowledge is reused but never overwritten. The architecture is evaluated on reinforcement learning tasks including Atari and 3D maze games, where it outperforms pretraining-and-finetuning baselines, and a new sensitivity measure shows that transfer occurs at both low-level sensory and high-level control layers. Its main cost is parameter growth that is linear in the number of tasks. For the framework, progressive columns represent the architectural extreme of isolation, against which LoRA adapters offer a far cheaper per-task increment on constrained hardware.

**Kirkpatrick et al. (2017) (E3; abstract).** Kirkpatrick and [123] colleagues challenge the view that catastrophic forgetting is an inevitable feature of connectionist models trained on tasks in sequence. Their method, elastic weight consolidation (EWC), remembers earlier tasks by selectively slowing learning on weights that are important for those tasks: a quadratic penalty anchors each parameter to its previous value with a strength given by an estimate of its importance, derived from the Fisher information. The approach is demonstrated on a sequence of classification tasks built from MNIST handwritten digits and on learning several Atari 2600 games sequentially, where networks retain expertise on tasks not experienced for a long time. Specific quantitative results are not given in the abstract. Limitations include the diagonal Fisher approximation, the need to know task boundaries in order to consolidate, and degrading effectiveness over long task sequences, as later surveys such as DeLange 2022 discuss. For the orchestration framework, EWC is a low-cost regulariser for protecting previously acquired skills when LoRA adapters or the orchestrator’s own policy are updated online, and the Fisher importance values provide a measure of which parameters encode established competence.

**Lopez-Paz and Ranzato (2017) (E3; abstract).** Lopez-Paz and Ranzato [124] study continual learning in which a model observes examples from a sequence of tasks once and one by one. They first propose metrics that assess not only test accuracy but also backward and forward transfer across tasks. They then introduce Gradient Episodic Memory (GEM), which stores a small episodic memory per task and constrains each update so that the loss on stored examples does not increase, which alleviates forgetting while allowing beneficial backward transfer. Experiments on variants of MNIST and CIFAR-100 show strong performance against the state of the art. Limitations include the per-step quadratic programme, whose cost grows with the number of tasks, and the need for task identifiers. For the framework, GEM’s transfer metrics are an evaluation reference for online updates, and its constrained-gradient idea is a candidate guard against regression.

**Rebuffi et al. (2017) (E3; abstract).** Rebuffi, Kolesnikov, Sperl and Lampert [125] address class-incremental learning, where new classes arrive over time and only data for a few classes are available simultaneously. Their method, iCaRL, learns classifiers and a data representation together, unlike earlier methods restricted to fixed representations. It combines a bounded exemplar memory, selected to approximate class means, with a nearest-mean-of-exemplars classifier and representation learning that uses distillation to preserve previous outputs. Experiments on CIFAR-100 and ImageNet ILSVRC 2012 show that iCaRL keeps learning classes over long periods where other strategies quickly fail. Limitations are its reliance on stored exemplars and its focus on image classification. For the framework, iCaRL is a reference for incrementally extending the perception module’s object vocabulary on the robot with a small exemplar buffer.

**Houlsby et al. (2019) (E3; abstract).** Houlsby and [126] colleagues note that fine-tuning a large pretrained model for each downstream task is parameter-inefficient, requiring a full model copy per task. They propose adapter modules, small bottleneck layers inserted into each Transformer block while the original weights stay frozen, so that tasks can be added without revisiting previous ones. Transferring BERT to 26 text-classification tasks including GLUE, adapters come within 0.4% of full fine-tuning on GLUE while adding only 3.6% parameters per task, compared with 100% for fine-tuning. Adapters add sequential layers and hence some inference latency, a drawback later addressed by LoRA (Hu 2022). For the framework, adapters are the conceptual basis for per-skill or per-modality modules attached to a shared frozen backbone on the robot.

**Rolnick et al. (2019) (E3; abstract).** Rolnick, Ahuja, Schwarz, Lillicrap and Wayne [127] examine catastrophic forgetting in reinforcement learning when an agent encounters tasks in sequence and, unlike most prior work, receives no explicit signal of task boundaries, the most general case for an agent receiving continuous experience. They investigate a simple and, they argue, overlooked remedy: CLEAR, which retains experience replay buffers for all past events and combines on-policy learning on new experience with off-policy learning and behavioural cloning on replayed experience. Experiments in Atari and DMLab show that this strategy still learns new tasks quickly while substantially reducing forgetting, even matching methods that require task identities. When storage is constrained, randomly discarding data allows a limited buffer to perform almost as well as an unbounded one. Numerical results are not given in the abstract. Limitations are the domains, which are simulated games, and a replay budget that remains substantial for long lifetimes. For the orchestration framework, the result supports simple replay with random eviction as a strong default for online updating on a memory-limited robot, and the combination of RL with behavioural cloning on replayed data parallels mixing reinforcement and imitation modalities.

**Cai et al. (2020) (E3; abstract).** Cai, Gan, Zhu and Han [128] address on-device learning, where edge devices must adapt models to new data within tight memory limits. They argue that reducing the number of trainable parameters does not directly save memory, because the dominant training cost is storing intermediate activations. Tiny-Transfer-Learning (TinyTL) therefore freezes the weights and learns only bias modules, which removes the need to store activations for weight gradients. To retain adaptation capacity, it adds a memory-efficient lite residual module that refines the feature extractor by learning small residual feature maps with only 3.8% memory overhead. TinyTL saves up to 6.5 times memory with little accuracy loss relative to full fine-tuning, improves accuracy by up to 34.1% over last-layer fine-tuning, and, combined with feature-extractor adaptation, achieves 7.3–12.9 times memory saving without sacrificing accuracy compared with fine-tuning the full Inception-V3. The evaluation concerns image-classification transfer rather than control or language models. For the orchestration framework, TinyTL’s central insight, that activation memory rather than parameter count limits on-device training, should shape how LoRA-style updates of perception modules or the orchestrator are budgeted on a Raspberry Pi 5.

**Lin et al. (2020) (E3; abstract).** Lin and [129] colleagues target deep learning on microcontrollers, whose memory is two to three orders of magnitude smaller than that of mobile phones. MCUNet co-designs a two-stage neural architecture search (TinyNAS), which first optimises the search space to fit resource constraints, with a memory-efficient inference engine (TinyEngine) that schedules memory by network topology. TinyEngine reduces memory by 4.8 times and speeds inference by 1.7–3.3 times relative to TF-Lite Micro and CMSIS-NN. MCUNet is the first to exceed 70% ImageNet top-1 accuracy on an off-the-shelf microcontroller, with 3.5 times less SRAM and 5.7 times less Flash than quantised MobileNetV2 and ResNet-18. It addresses inference only. For the framework, it shows how architecture and runtime co-design can fit perception well below Raspberry Pi 5 budgets.

**Li and Liang (2021) (E3; abstract).** Li and Liang [130] observe that fine-tuning a pretrained language model modifies all parameters and therefore requires a full model copy per task. They propose prefix-tuning, which keeps the language model frozen and optimises a small continuous task-specific vector, the prefix, to which subsequent tokens attend as if it were a sequence of virtual tokens. Applied to GPT-2 for table-to-text generation and to BART for summarisation, prefix-tuning learns only 0.1% of the parameters yet matches fine-tuning with full data, outperforms it in low-data settings and extrapolates better to topics unseen in training. The prefix occupies part of the context and was evaluated only on text generation. For the framework, prefix-tuning is an alternative to LoRA for specialising a small local orchestrator LLM, with very small per-task storage.

**Hu et al. (2022) (E3; abstract).** Hu and [131] colleagues note that full fine-tuning becomes infeasible as pretrained language models grow; deploying independent fine-tuned copies of GPT-3 175B, for example, is prohibitively expensive. Low-Rank Adaptation (LoRA) freezes the pretrained weights and injects trainable rank-decomposition matrices into each Transformer layer, so that each weight update is represented as the product of two small low-rank matrices. Compared with GPT-3 175B fine-tuned with Adam, LoRA reduces the number of trainable parameters by 10,000 times and the GPU

memory requirement by three times. It performs on par with or better than fine-tuning in model quality on RoBERTa, DeBERTa, GPT-2 and GPT-3, with higher training throughput and, unlike adapters, no additional inference latency, because the low-rank update can be merged into the frozen weights. Limitations are that the rank and target layers must be chosen, and memory for activations and the frozen model remains, so training large models still needs GPUs. For the orchestration framework, LoRA is the chosen mechanism for online improvement of the local orchestrator LLM: small, swappable adapters can be trained from logged decisions, stored per skill or robot, and merged or removed without altering the base model.

**Lin et al. (2022) (E3; abstract).** Lin, Zhu, Chen, Wang, Gan and Han [132] pursue on-device training, which lets models adapt to sensor data without sending it to the cloud, on IoT devices whose memory is far too small for conventional training. They identify two obstacles: quantised networks are hard to optimise because of low bit precision and absent normalisation, and limited hardware cannot support full back-propagation. Quantisation-Aware Scaling calibrates gradient scales to stabilise 8-bit quantised training, and Sparse Update skips gradient computation for less important layers and sub-tensors. These are implemented in a Tiny Training Engine, which prunes the backward graph and moves automatic differentiation to compile time. The framework is the first to train convolutional networks on-device within 256 KB of SRAM and 1 MB of Flash without auxiliary memory, using less than one thousandth of the memory of PyTorch and TensorFlow while matching accuracy on the visual wake words (VWW) task. The work addresses small convolutional networks, and the sparse-update schedule is chosen offline. For the orchestration framework, the paper demonstrates that local, cloud-free continual adaptation is feasible under extreme budgets, and its sparse-update principle can be applied to deciding which layers of a perception module to update on a Raspberry Pi 5.

**Liu et al. (2022) (E3; abstract).** Liu and [133] colleagues compare few-shot in-context learning (ICL), which processes all training examples at every prediction, with parameter-efficient fine-tuning (PEFT), which trains a small set of parameters. They show that PEFT gives better accuracy at dramatically lower computational cost. They introduce (IA)<sup>3</sup>, a PEFT method that scales activations by learned vectors and adds very few parameters, and T-Few, a recipe based on the T0 model that applies to new tasks without task-specific tuning. On the RAFT benchmark of unseen tasks, T-Few attains super-human performance for the first time and exceeds the prior state of the art by 6% absolute. The study is limited to NLP classification-style tasks. For the framework, it justifies fine-tuning a small orchestrator over long in-context prompts, which are costly on a Pi.

**De Lange et al. (2022) (R; full text).** De Lange and [134] colleagues survey continual learning for task-incremental classification, where tasks arrive sequentially with clear boundaries. They offer a taxonomy of replay, regularisation and parameter-isolation methods, propose a framework for continually setting the stability–plasticity trade-off, and compare 11 methods and 4 baselines, including iCaRL, GEM, EWC and PackNet, on Tiny ImageNet, iNaturalist and RecogSeq. The comparison assumes known task boundaries and image classification. For the framework, it is the standard reference for choosing among replay, regularisation and isolation when updating modules online.

**Dettmers et al. (2023) (E3; abstract).** Dettmers, Pagnoni, Holtzman and Zettlemoyer [135] present QLoRA, an efficient fine-tuning approach that backpropagates gradients through a frozen pretrained language model quantised to 4 bits into Low-Rank Adapters (LoRA, Hu 2022). The technique reduces memory enough to fine-tune a 65-billion-parameter model on a single 48 GB GPU while, the authors report, preserving the task performance of full 16-bit fine-tuning. Using the method, they train more than a thousand models across instruction datasets and model families. Their best model family, Guanaco, outperforms previously released open models on the Vicuna benchmark, reaching 99.3% of ChatGPT’s performance level after only 24 hours of fine-tuning on a single GPU. Limitations include reliance on GPU kernels for quantised back-propagation and on automated, chatbot-oriented benchmarks for evaluation. For the orchestration framework, QLoRA is the most direct precedent for fine-tuning a quantised local orchestrator LLM: a 4-bit base model with LoRA adapters is the memory profile most plausible on a Raspberry Pi 5, although QLoRA’s CUDA-dependent training path means that on-robot updates would need a CPU implementation or must be scheduled on a nearby accelerator.

**Liu et al. (2023) (E4; abstract).** Liu, Nasiriany, Zhang, Bao and Zhu [136] present Sirius, a framework in which a partially autonomous robot handles most decisions while a human monitors and intervenes in difficult situations, keeping deployment safe while generating data. A learning algorithm then improves the policy from deployment data by re-weighting samples according to approximated human trust and training with weighted behavioural cloning. On contact-rich manipulation tasks in simulation and on real hardware, Sirius improves policy success over state-of-the-art methods by 8% in simulation and 27% on the real robot, converges twice as fast and reduces memory size by 85%. It depends on a human operator. For the framework, intervention-derived weights are a behavioural-fidelity signal, and Sirius is a reference for learning during deployment.

**Liu et al. (2023) (E3; abstract).** Liu, Zhu, Gao, Feng, Liu, Zhu and Stone [137] introduce LIBERO, a benchmark for lifelong learning in decision-making, where, unlike image or text domains, agents must transfer procedural knowledge such as actions and behaviours as well as declarative knowledge about entities. LIBERO targets five research questions: transfer of declarative, procedural or mixed knowledge; policy architecture design; lifelong-learning algorithms; robustness to task ordering; and the effect of pretraining. An extendible procedural generation pipeline can in principle generate unlimited tasks, and four task suites comprising 130 tasks are provided, each with high-quality human-teleoperated demonstrations to support sample-efficient learning. Experiments yield several unexpected findings: sequential fine-tuning outperforms existing lifelong-learning methods in forward transfer, no single visual encoder architecture excels at all types of transfer, and naive supervised pretraining can hinder subsequent lifelong learning. Limitations are that the benchmark is simulated, uses a single-arm tabletop setting and relies on demonstrations. For the orchestration framework, LIBERO is the principal evaluation reference: it is used by several VLA uncertainty and continual-learning papers in this survey, such as Romer 2026a and Romer 2026b, and a simulated stand-in for LeKiwi tasks could be evaluated with its protocols before hardware trials.

**Zhu et al. (2023) (S; abstract).** Zhu, Hu, Lin, Wang, Chen, Gan and Han [138] present PockEngine, an on-device learning engine for fine-tuning models on edge devices with diverse hardware and constraints. It supports sparse back-propagation, pruning the backward graph and updating the model sparsely with measured memory and latency savings while maintaining model quality. It follows a compilation-first design, deriving the full training graph at compile time, including forward, backward and optimiser steps, so that training-graph optimisations such as operator reordering and backend switching can be applied. PockEngine supports PyTorch, TensorFlow and JAX front-ends and deploys to mobile CPUs, GPUs and DSPs. Reported results include up to 15 times speed-up over off-the-shelf TensorFlow on a Raspberry Pi, 5.6 times memory saving for back-propagation on a Jetson AGX Orin, and fine-tuning of LLaMA-2-7B on the Jetson AGX Orin at 550 tokens per second, 7.9 times faster than PyTorch. Limitations are that sparse-update schemes must be chosen per model and that the LLM result relies on a Jetson-class GPU. For the orchestration framework, PockEngine is the most relevant systems result, showing substantial training speed-ups on a Raspberry Pi and a route to on-device LLM fine-tuning without the cloud.

**Malladi et al. (2023) (E3; abstract).** Malladi and [139] colleagues address the prohibitive memory cost of back-propagation when fine-tuning large language models. MeZO adapts classical zeroth-order SGD to operate in place, estimating gradients from two forward passes, so fine-tuning has the same memory footprint as inference. On one A100 80 GB GPU, MeZO trains a 30-billion-parameter model, whereas back-propagation can fine-tune only a 2.7B model with the same budget. Across masked and autoregressive models up to 66B, MeZO outperforms in-context learning and linear probing and matches back-propagation fine-tuning on several tasks, with up to 12 times less memory and up to twice fewer GPU-hours; it also works with LoRA and prefix-tuning and optimises non-differentiable objectives. Zeroth-order methods require many more steps. For the framework, MeZO is a candidate for fine-tuning the orchestrator on a Pi within inference-level memory.

**Liu et al. (2024) (E3; abstract).** Liu, Zhang, Asadi, Liu, Zhao, Sabach and Fakoor [140] argue that large pretrained models remain under-used in robot control because of data scarcity and the cost of fine-tuning, and that real deployments require data-efficient, continual adaptation to new tasks rather than one-off single-task adaptation. They introduce TAIL (Task-specific Adapters for Imitation Learning), a framework that adapts a large pretrained policy to each new control task with limited demonstrations by training parameter-efficient modules. They compare bottleneck adapters, P-Tuning and Low-Rank Adaptation (LoRA) against full fine-tuning and other adaptation baselines on large-scale language-conditioned manipulation tasks. TAIL with LoRA achieves the best post-adaptation performance while using only 1% of the trainable parameters of full fine-tuning, avoiding catastrophic forgetting and preserving plasticity in continual learning. Limitations are that the abstract reports no real-robot evaluation and that task identity is needed to select the appropriate adapter at test time. For the orchestration framework, TAIL provides direct evidence that LoRA adapters are the right unit for continual imitation updates, and it motivates the orchestrator’s role as the component that chooses which adapter, and hence which skill or modality, to activate.

**Wan et al. (2024) (E4; abstract).** Wan, Zhu, Shah and Zhu [141] introduce LOTUS, a continual imitation learning algorithm intended to let a physical robot keep learning new manipulation tasks efficiently throughout its lifespan. The central idea is an ever-growing skill library built from a sequence of tasks, each taught with a small number of human demonstrations. LOTUS first performs continual skill discovery with an open-vocabulary vision model, extracting skills as recurring patterns in unsegmented demonstrations; the discovery process updates existing skills to avoid catastrophic forgetting of earlier tasks and adds new skills when novel tasks require them. A meta-controller is trained to compose skills flexibly to solve vision-based manipulation tasks during lifelong learning. The authors report that LOTUS outperforms state-of-the-art baselines by more than 11% in success rate, indicating superior knowledge transfer. Details of the benchmark and real-robot evaluation are not given in the abstract. Limitations include reliance on human demonstrations,

the computational cost of an open-vocabulary vision model and the assumption that tasks decompose into recurring skills. For the orchestration framework, LOTUS’s meta-controller that selects among a growing set of skills is structurally analogous to the proposed orchestrator selecting among learning modalities, and its skill-library growth offers a mechanism for adding capabilities without retraining.

**Li et al. (2025) (E3; abstract).** Li and [142] colleagues observe that on-device fine-tuning of large language models is prohibitive on mobile devices because of memory and training time, even with parameter-efficient methods. MobiLLM keeps only a frozen backbone on the device and offloads back-propagation of a trainable side network of parallel adapters to a server. Because the adapters sit beside rather than inside the backbone, the device needs only one-way transfer of low-width quantised activations to the server, raw data remain on the device, and device and server computations run in parallel. The authors report that even a CPU-only device can fine-tune LLMs with significantly reduced convergence time and memory; figures are not given in the abstract. The approach depends on a server. For the framework it is a contrasting, server-assisted design that the fully local orchestrator must match or justify departing from.

**Römer et al. (2026) (E3; abstract).** Römer, Zhang and Schoellig [143] address continual learning for vision-language-action (VLA) models, where robots must learn new tasks without forgetting earlier ones and, ideally, without storing exemplars from past tasks. Their framework, CLARE, is parameter-efficient and exemplar-free: it inserts lightweight modular adapters into the feed-forward layers of the VLA and expands them autonomously only where needed, guided by layer-wise feature similarity between new and previous tasks. At inference, an autoencoder-based routing mechanism selects the appropriate adapters without being told the task identity, so the model can operate task-agnostically. Experiments on the LIBERO benchmark are reported to show strong performance on new tasks while mitigating catastrophic forgetting, surpassing even exemplar-based baselines; specific figures are not given in the abstract. Limitations suggested by the summary are the simulation-only evaluation and the growth of adapter count with task diversity. For the orchestration framework, CLARE is closely aligned: its autoencoder router is itself a confidence-like signal indicating how familiar an input is to each adapter, and adapter expansion guided by feature similarity offers a concrete rule for deciding when new learning, rather than reuse, is required, which the orchestrator must also decide.

## Adaptive computation, mixture of experts and routing

**Jacobs et al. (1991) (E2; abstract).** Jacobs, Jordan, Nowlan and Hinton [144] present a supervised learning procedure for systems composed of many separate networks, each of which learns to handle a subset of the complete set of training cases. The procedure can be viewed either as a modular version of a multilayer supervised network or as an associative version of competitive learning. In the well-known formulation, a gating network assigns input-dependent mixing probabilities to a set of expert networks, and training encourages the experts to specialise on different regions of the input space rather than to cooperate on every case, reducing interference between sub-tasks. The paper evaluates the approach on a multi-speaker vowel discrimination task; its quantitative results are not reported here. Limitations of the original formulation include training instability of the gate, a small number of experts, and no explicit treatment of computational cost. For the orchestration framework, this is the foundational reference: the orchestrator is conceptually a gating network that routes each situation to the learning modality best placed to handle it, and later mixture-of-experts work in the survey builds on this formulation with sparse, budget-aware routing.

**Bengio et al. (2015) (E3; full text).** Bengio et al. Bengio et al. [145] address the cost of evaluating and training deep networks by learning conditional computation policies that activate only parts of a network per input. They cast the choice of which blocks of hidden units to drop as a reinforcement learning problem: a per-layer policy, conditioned on the current activations, samples block-wise dropout masks, and is trained with a policy-gradient method on a loss that rewards parsimonious activation while preserving prediction accuracy. A regularisation mechanism encourages diversity in the dropout policy so that different inputs use different blocks and no block is permanently idle. Experiments are on standard image classification datasets (MNIST, CIFAR-10 and SVHN) with fully connected networks; the authors report that the learned policies improve wall-clock speed without materially degrading accuracy, with larger speed-ups on CPU than on GPU because sparse block computation maps poorly to dense GPU kernels. The evaluation is small in scale, restricted to feed-forward classifiers, and the REINFORCE-style estimator is high-variance. For the orchestration framework, the work is an early template for treating ‘which computation to run’ as a learned, input-dependent policy trained against an explicit compute penalty, which is exactly the structure of a budget-aware modality selector; its observation that sparsity only pays off when hardware can exploit it is directly relevant to a Raspberry Pi 5 CPU target.

**Graves (2016) (E3; full text).** Graves [146] introduces Adaptive Computation Time (ACT), which lets a recurrent network learn how many internal update steps to perform between receiving an input and emitting an output. At each step a sigmoidal halting unit outputs a halting probability; computation continues until the cumulative probability reaches a

threshold, and the output is a mean-field weighted average over intermediate states, keeping the mechanism deterministic and differentiable. A 'ponder cost', essentially the number of steps plus a fractional remainder, is added to the training loss with a weight  $\tau$ , so the network trades accuracy against computation. Experiments cover four synthetic tasks (parity, logic operations, integer addition and sorting), where ACT markedly improves performance and computation is allocated according to task difficulty, and character-level language modelling on the Hutter Prize Wikipedia corpus, where ACT gives little gain in predictive performance but concentrates computation at word boundaries and punctuation, which the author interprets as evidence of structure detection. Limitations include sensitivity to the ponder penalty  $\tau$ , which must be tuned per task, and evaluation restricted to sequence problems. For the framework, ACT supplies the canonical differentiable formulation of 'how much to think' with an explicit compute penalty, a direct analogue of deciding whether to escalate from a cheap modality to an expensive one under a budget.

**Shazeer et al. (2017) (E3; abstract).** Shazeer et al. [147] make conditional computation practical at scale with the Sparsely-Gated Mixture-of-Experts (MoE) layer, revisiting the mixture-of-experts idea of Jacobs 1991. The layer contains up to thousands of feed-forward expert sub-networks, and a trainable gating network selects a sparse (top-k) combination of experts for each example, so that parameter count grows without a proportional increase in computation. The authors address the algorithmic and systems obstacles to this in practice, including batch shrinkage per expert, network bandwidth, and load balancing across experts through auxiliary losses and noisy gating. MoE layers with up to 137 billion parameters are applied convolutionally between stacked LSTM layers for language modelling and machine translation. The abstract reports greater than 1000x improvements in model capacity with only minor losses in computational efficiency on GPU clusters, and results significantly better than the then state of the art at lower computational cost. The approach depends on large distributed GPU infrastructure, and routing introduces instability and load-imbalance issues that later work sought to simplify. For the framework, the sparse top-k gate with balancing losses is the reference mechanism for a learned router that dispatches inputs to specialist modules; the orchestrator can be read as a very coarse-grained, interpretable gate over four modality 'experts', where the balancing and noisy-gating tricks remain relevant.

**Rosenbaum et al. (2018) (E3; abstract).** Rosenbaum, Klinger and Riemer [148] propose routing networks to mitigate task interference in multi-task learning. A routing network comprises a router and a set of function blocks (e.g. fully connected or convolutional layers); for each input the router chooses a block to apply, passes the output back to itself and repeats until a fixed recursion depth, so that a different composition of blocks is assembled per input. Because the routing decisions are discrete, the router and function blocks are trained jointly with a collaborative multi-agent reinforcement learning formulation, in which routing agents (e.g. per task) receive rewards reflecting prediction quality, while function blocks are trained by back-propagation. Evaluation is on multi-task versions of MNIST, mini-ImageNet and CIFAR-100 against cross-stitch networks and shared-layer baselines. The authors report significantly improved accuracy with sharper convergence, nearly constant per-task training cost (whereas cross-stitch networks scale linearly with the number of tasks), and cross-stitch-level performance on 20-task CIFAR-100 with an 85% reduction in training time. Limitations are the small-scale image benchmarks, fixed recursion depth and the known instability of RL-trained routers. For the framework, routing networks show that an RL-trained selector over reusable modules can outperform static sharing; the multi-agent RL routing formulation is a plausible design for learning which modality to invoke from task outcome rewards.

**Riquelme et al. (2021) (E3; abstract).** Riquelme et al. [149] bring sparsely gated mixtures of experts to computer vision with the Vision MoE (V-MoE), a sparse variant of the Vision Transformer in which some MLP blocks are replaced by expert layers selected per image patch. They also propose Batch Prioritised Routing, which ranks inputs across the whole batch so that lower-priority patches can be dropped, giving adaptive per-image compute and a smooth test-time trade-off between performance and computation. On image recognition, V-MoE matches state-of-the-art dense networks while requiring as little as half the compute at inference, and a 15-billion-parameter model attains 90.35% on ImageNet. The work relies on large-scale pre-training and TPU infrastructure. For the framework, the test-time adjustable compute knob is the relevant idea: a perception module whose cost can be dialled down under budget pressure is a natural 'arm' for the orchestrator.

**Hoque et al. (2021) (E4; abstract).** Hoque et al. [150] present ThriftyDagger, an interactive imitation learning algorithm that requests human interventions only within a desired budget. Building on Dagger (Ross 2011), a learned switching policy solicits help at states that are sufficiently novel, where the robot has no reference behaviour to imitate, or risky, where the robot has low confidence in task completion; risk is estimated with a novel metric. In simulation and on physical cable routing it balances performance and supervisor burden better than prior methods, with a 100% success rate at execution time on both tasks, and, in a user study (N=10) with a three-robot fleet, 58% and 80% improvements in human and robot performance over the next-best algorithm. Limitations include reliance on a human supervisor. For the framework, the novelty and risk gates are ready-made confidence signals for deciding when to escalate from autonomous execution to imitation or recovery.

**Fedus et al. (2022) (E3; abstract).** Fedus, Zoph and Shazeer [151] introduce the Switch Transformer, which simplifies mixture-of-experts routing (Shazeer 2017) by sending each token to a single expert. Together with reduced communication cost, improved initialisation and selective precision, this allows large sparse models to be trained stably, for the first time in bfloat16. Models based on T5-Base and T5-Large obtain up to 7x pre-training speed-ups with the same computational resources, gains extend over mT5-Base across all 101 languages, and trillion-parameter models pre-trained on the C4 corpus achieve a 4x speed-up over T5-XXL. The results depend on large accelerator clusters and concern language modelling rather than control. For the framework, the key lesson is that top-1 routing is simpler and more stable than top-k, supporting a single-choice modality selector rather than weighted mixtures.

**Puigcerver et al. (2024) (E3; abstract).** Puigcerver et al. [152] propose Soft MoE, a fully differentiable alternative to sparse mixture-of-experts routing. Rather than assigning discrete tokens to experts, Soft MoE passes different weighted combinations of all input tokens to each expert slot, so each expert processes only a small number of combined tokens; this removes token dropping, eases training instability and allows many more experts. In visual recognition, Soft MoE outperforms dense ViTs and the Tokens Choice and Experts Choice MoE variants; Soft MoE Huge/14 with 128 experts in 16 MoE layers has over 40x more parameters than ViT Huge/14 with only 2% more inference time and substantially better quality. The method assumes batch-level token mixing, complicating autoregressive use, and is evaluated at large scale. For the framework, it shows that soft, differentiable routing avoids brittle discrete choices, an option to weigh against hard modality selection.

**Wang et al. (2024) (E4; full text).** Wang et al. [153] address the computational cost and catastrophic forgetting of universal multitask robot policies with Sparse Diffusion Policy (SDP), which inserts mixture-of-experts layers into a transformer-based diffusion policy (building on Chi 2023). A task-specific router activates only a small subset of experts per task, so new tasks can be learned by adding and routing to new experts while freezing existing ones, and previously trained experts can be reused and recombined for transfer. Evaluation spans 2D multitask simulation (MimicGen, eight tasks), 3D simulation (DexArt and Adroit), continual learning, task transfer and real-robot manipulation (pulling, pick-and-place and hanging). According to the full text, SDP reaches 0.76 average success on the eight MimicGen tasks with 53.3M active parameters versus 52.6M for a single-task baseline, outperforming task-conditioned diffusion, Octo and multi-head baselines; it keeps active parameters constant (9.2M) across continual-learning stages while retaining 0.94 success on earlier tasks; and in a coffee-preparation transfer task it reaches 0.80 success training only 0.1M parameters versus 0.70 for training from scratch. Limitations include reliance on known task identity for routing, and relatively few real-robot trials. For the framework, SDP is a concrete example of expert-level sparsity and reuse inside a single imitation policy, offering a low-cost way to add skills without forgetting that complements orchestration at the modality level.

**Chen et al. (2024) (E3; abstract).** Chen, Zaharia and Zou [154] study how to use commercial large language model APIs while reducing cost and improving performance. They first document that popular APIs (e.g. GPT-4, ChatGPT, J1-Jumbo) have heterogeneous pricing whose fees can differ by two orders of magnitude, and they outline three cost-reduction strategies: prompt adaptation, LLM approximation and LLM cascades. As an instance of the last, FrugalGPT learns which combinations of LLMs to query for different inputs: queries are sent sequentially through a list of models, and a learned scoring function judges whether the current answer is reliable enough to stop or whether to escalate to a more expensive model, with the list and thresholds optimised under a budget. Experiments show that FrugalGPT can match the performance of the best individual LLM (e.g. GPT-4) with up to 98% cost reduction, or improve accuracy over GPT-4 by 4% at the same cost. Limitations are per-task scorer training, a cloud-API cost model and single-turn text tasks rather than sequential decision making. For the framework, the cascade with a learned reliability score is the closest analogue to confidence-gated escalation between modalities: the orchestrator can be framed as a cascade that tries cheap modalities first and escalates when calibrated confidence is low, with compute and latency replacing API price.

**Ding et al. (2024) (E3; abstract).** Ding et al. [155] propose Hybrid LLM, a hybrid inference scheme that combines a small model deployable on lower-cost (e.g. edge) devices with a large cloud-hosted model. A router predicts query difficulty, specifically the probability that the small model’s response quality is close to that of the large model, and assigns each query to the small or large model accordingly; a threshold on this prediction sets the desired quality level and can be tuned at test time to trade quality for cost. Experiments with small/large model pairs (e.g. Llama-2 variants versus GPT-3.5-turbo) show up to 40% fewer calls to the large model with no drop in response quality. According to the full text, the router is fast (reported at around 0.036 s per query versus 0.46–14.61 s for the LLMs), and about 20% of queries are answered as well or better by Llama-2 13B than by GPT-3.5-turbo. Limitations include dependence on an automatic quality metric, a binary small-versus-large choice, and a single-turn setting. For the framework, Hybrid LLM is a direct precedent for an edge-first design: a cheap, calibrated difficulty predictor decides when local capacity suffices, and a tunable threshold turns the compute budget into an explicit, adjustable operating point.

**Aggarwal et al. (2024) (E3; abstract).** Aggarwal, Madaan and [156] colleagues present AutoMix, a method for routing queries from a smaller language model to larger ones based on the approximate correctness of the small model’s output. It has two components. First, a few-shot self-verification step in which the small model estimates the reliability of its own answer against the given context, without extensive training. Second, because such self-verification is noisy, a router formulated as a partially observable Markov decision process (POMDP) treats the verifier’s confidence as a noisy observation of the true answer quality and decides whether to accept the answer or escalate to a larger model. Experiments across five language models and five challenging reasoning and question-answering datasets show that AutoMix consistently surpasses strong baselines, reducing computational cost by over 50% for comparable performance. Limitations are the dependence on context-grounded tasks where self-verification is meaningful, the additional calls needed for verification, and evaluation limited to text. For the framework, AutoMix is particularly relevant because it explicitly models the confidence signal as noisy and makes the escalation decision under partial observability; this is a principled template for an orchestrator that fuses imperfectly calibrated signals such as perception uncertainty, policy variance and imitation fidelity before committing to a more expensive modality.

**Yue et al. (2024) (E3; abstract).** Yue et al. Yue et al. [157] tackle the mismatch between the computation and memory demands of multimodal large language models (MLLMs) and the limited resources of robot platforms. DeeR-VLA is a dynamic early-exit framework for vision-language-action models: the MLLM is given a multi-exit architecture so that processing can terminate once a sufficient portion of the model has been activated for the current situation, avoiding redundant computation. The authors develop algorithms that set early-termination criteria conditioned on predefined resource demands, namely average computational cost (a proxy for power consumption), peak computational consumption (latency) and GPU memory usage, so that the model operates within a specified budget. On the CALVIN long-horizon language-conditioned manipulation benchmark (Mees 2022), DeeR reduces the LLM’s computational cost by 5.2–6.5x and its GPU memory by 2–6x without compromising task performance. Limitations are that evaluation is in simulation only, that the exit criteria are tuned offline against budget targets rather than learned from task outcomes, and that the underlying models still require a GPU. For the framework, DeeR is a close methodological neighbour: it makes ‘how much model to run’ a budget-conditioned decision at each step, and its exit criteria provide a reference for turning compute, latency and memory constraints into switching thresholds, although at the level of network depth rather than learning modality.

**Reuss et al. (2025) (E3; abstract).** Reuss et al. Reuss et al. [158] address the growing computational demands of diffusion policies for imitation learning with Mixture-of-Denoising Experts (MoDE). MoDE is a transformer diffusion policy with sparse experts whose routing is conditioned on the noise level, together with noise-conditioned self-attention. Because routing depends only on the noise level, expert choices can be pre-computed and cached, reducing active parameters by 40% and inference costs by 90%. MoDE achieves state-of-the-art performance on 134 tasks across four benchmarks from CALVIN and LIBERO; with pre-training on diverse robot data it scores 4.01 on CALVIN ABC and 0.95 on LIBERO-90, and it surpasses CNN- and transformer-based diffusion policies by an average of 57% while using 90% fewer FLOPs. Evaluation is simulation-based. For the framework, MoDE shows how sparse routing can cut the cost of the imitation modality, making a diffusion-style policy more plausible on constrained hardware.

**Huang et al. (2025) (E4; full text).** Huang, Zhu, Du and Zhao [159] present MoE-LoCo, a mixture-of-experts framework for multitask legged locomotion. A single reinforcement learning policy handles diverse terrains, including bars, pits, stairs, slopes and baffles, and supports both quadrupedal and bipedal gaits on one robot. The authors argue that multitask RL suffers from conflicting gradients between tasks, and they replace the shared actor and critic layers with mixture-of-experts layers so that different experts absorb different tasks; gradient-conflict analysis supports this claim. According to the full text, training uses IsaacGym with 4,096 parallel robots, six experts and a staged curriculum, and the policy is deployed on a Unitree Go2 quadruped at 50 Hz with onboard NVIDIA Jetson Orin compute; in simulation the mixed-terrain success rate is reported as 87.9%, and real-world deployment succeeds across all nine tasks (five quadrupedal and four bipedal). Experts specialise in distinct behaviours, which the authors exploit for skill composition and task migration. Limitations are the focus on locomotion rather than manipulation and the use of privileged simulation training. For the framework, MoE-LoCo demonstrates on a real robot that experts specialise and that expert-level routing mitigates interference between skills, supporting modular designs in which an orchestrator selects among specialised capabilities rather than training one monolithic policy.

**Ong et al. (2025) (E3; abstract).** Ong et al. Ong et al. [160] present RouteLLM, a framework for learning routers that choose, per query, between a stronger and a weaker LLM so as to balance cost and response quality. The main innovation is the use of human preference data, principally pairwise comparisons from Chatbot Arena, as the training signal: the router learns the probability that the strong model’s answer would be preferred over the weak one, and a threshold on this probability sets the cost–quality operating point. Several lightweight router architectures are studied, and training data are augmented to improve performance. On widely used benchmarks, the routers reduce costs by over 2x in certain

cases without compromising response quality, and they transfer to different strong/weak model pairs at test time without retraining. Limitations include the binary two-model setting, reliance on preference data distributions that may not match the deployment domain, and evaluation limited to text. For the framework, RouteLLM contributes two practical ideas: routers can be learned from comparative outcome data rather than absolute correctness labels, and lightweight routers can generalise across the models they route between, which suggests that a modality selector trained on logged comparative outcomes might remain valid when individual modality implementations are updated.

**Hao et al. (2026) (E4; abstract).** Hao, Zhai, Liu and Soh [161] address the cost of scaling diffusion-based manipulation policies to many tasks. They introduce the Skill Mixture-of-Experts Policy (SMP), a diffusion policy in which a compact, orthogonal basis of skills is learned as experts, and a sticky routing scheme composes actions from a small, task-relevant subset of experts at each step, keeping expert selection temporally consistent within a manipulation phase. A variational training objective supports learning the skill basis and routing, and adaptive expert activation at inference yields fast sampling without an oversized backbone. The method is validated in simulation and on a real dual-arm platform in multi-task learning and transfer learning settings, where SMP achieves higher success rates and markedly lower inference cost than large diffusion baselines, and enables few-shot adaptation to new tasks by reusing learned skills. The abstract does not quantify these gains; a likely limitation is the need for diverse multi-task demonstrations to learn a reusable basis. For the framework, SMP embodies the principle of ‘learn reusable skills once, activate only what is needed’: sticky, phase-consistent expert activation is a useful analogue for an orchestrator that should avoid rapid switching between modalities, and its inference savings bear directly on edge feasibility of imitation policies.

**Liu et al. (2026) (E3; full text).** Liu et al. Liu et al. [162] ask when an embodied robot agent should invoke expensive LLM-based reasoning and when it should simply act. Invoking reasoning incurs latency and resource overhead that can delay actions and degrade reliability, while too little reasoning leads to incorrect decisions and task failures. They propose RARRL (Resource-Aware Reasoning via Reinforcement Learning), a hierarchical framework for resource-aware orchestration of embodied agents, in which a high-level policy trained with reinforcement learning decides whether, and how much, to reason given the current state and resource considerations, while lower-level modules execute actions. Evaluation uses the ALFRED benchmark in the AI2-THOR simulator with navigation, inspection and multi-step delivery tasks. According to the full text, RARRL reduces LLM inference time by over 60% and token consumption by 50–75% while keeping success rates close to always-reasoning baselines, for example  $82.7 \pm 1.7\%$  versus 84.0% on navigation,  $76.4 \pm 1.9\%$  versus 78.5% on inspection and  $69.5 \pm 2.1\%$  versus 71.2% on delivery. Limitations are simulation-only evaluation and a small but consistent success drop relative to full reasoning. For the framework, RARRL is the closest existing work to the proposed orchestrator: it learns, with RL, a meta-level policy that trades task success against computational cost when deciding whether to invoke an expensive capability, and it provides a direct baseline and evaluation protocol, albeit for reasoning rather than learning modalities.

**Zhang et al. (2026) (E4; abstract).** Zhang et al. Zhang et al. [163] target the computational and storage demands that hinder real-world deployment of MLLM-based vision-language-action (VLA) models. They argue that existing sparsification techniques such as early exit and token pruning neglect the final LLM layers, which encode the semantic information most relevant to downstream robotic tasks. Motivated by the Shallow Brain Hypothesis in neuroscience and by mixtures of experts, MoLe-VLA treats each LLM layer as an expert and uses a Spatial-Temporal Aware Router (STAR) to activate only some layers based on the robot’s current state. To compensate for the cognitive ability lost by skipping layers, a Cognition Self-Knowledge Distillation (CogKD) framework distills task-relevant cognitive features from the full model. Experiments in RL Bench simulation (James 2020) and real-world environments show an 8% improvement in mean success rate across ten tasks and computational cost reductions of up to 5.6x relative to standard LLMs. Limitations are a GPU-bound MLLM backbone, a modest task set and reliance on distillation. For the framework, MoLe-VLA and DeeR-VLA (Yue 2024) together define the state of the art in state-conditioned dynamic depth for VLA models; the STAR router is an example of a learned, state-dependent compute allocator that could serve as a within-modality mechanism beneath a modality-level orchestrator.

## Platforms, benchmarks and evaluation

**James et al. (2020) (S; abstract).** James et al. James et al. [164] introduce RL Bench, a benchmark and learning environment for vision-guided robot manipulation. It provides 100 unique, hand-designed tasks ranging from simple target reaching and door opening to multi-stage tasks such as opening an oven and placing a tray inside. Observations include proprioception and RGB, depth and segmentation masks from an over-the-shoulder stereo camera and an eye-in-hand camera. Each task comes with an unlimited supply of demonstrations generated by motion planners over task-defined waypoints, and new tasks can be created and validated with provided tools. The benchmark targets reinforcement learning, imitation learning, multi-task and few-shot learning, and proposes a large-scale few-shot challenge. The abstract

reports no performance figures, and the simulated arm differs from low-cost hardware. For the framework, RL Bench is a candidate simulated testbed in which RL and imitation modalities can be compared under identical tasks and observations.

**Mandlekar et al. (2021) (E4; abstract).** Mandlekar et al. [165] conduct a systematic study of learning manipulation policies from offline human demonstrations, motivated by a lack of open human datasets and reproducible methods. They evaluate six offline learning algorithms, spanning behavioural cloning variants and offline reinforcement learning, on five simulated and three real-world multi-stage manipulation tasks with datasets of varying quality. Lessons include sensitivity to algorithmic design choices, strong dependence on demonstration quality, and variability arising from policy selection because training and evaluation objectives differ. The study also shows that learning from human data can yield proficient policies on multi-stage tasks beyond current RL, and scales to raw sensory inputs. The abstract gives no summary figures. For the framework, the work provides an evaluation reference for the imitation modality and highlights that demonstration quality and checkpoint selection strongly affect behavioural fidelity, both of which bear on how imitation confidence should be estimated.

**Mees et al. (2022) (S; abstract).** Mees et al. [166] present CALVIN (Composing Actions from Language and Vision), an open-source simulated benchmark for long-horizon, language-conditioned robot manipulation. Agents must solve sequences of manipulation tasks from onboard sensors, specified only via natural language, and the benchmark supports flexible sensor suites. CALVIN tasks are described as more complex than existing vision-and-language datasets in sequence length, action space and language. Evaluation covers zero-shot generalisation to novel language instructions and to novel environments and objects. A baseline based on multi-context imitation learning performs poorly, indicating substantial room for improvement. The abstract reports no figures. For the framework, CALVIN is the benchmark used by DeeR-VLA (Yue 2024) and MoDE (Reuss 2025), making it a natural reference for comparing compute-aware policy selection against published efficiency results, though its simulated arm differs from LeKiwi hardware.

**Kemp et al. (2022) (E4; abstract).** Kemp et al. [167] describe the design of Stretch, a compact, lightweight mobile manipulator for indoor human environments intended to lower the size, weight and cost barriers to adoption without unduly restricting capability. The core design comprises a two-wheeled differential-drive base, a lift and a telescoping arm arranged to produce Cartesian motion at the end of the arm; extensions include a 1-DOF wrist to stow a tool, a 2-DOF dexterous wrist and a compliant gripper. The design is justified with anthropometry and mathematical models of static stability, and empirical support comes from teleoperating and autonomously controlling the commercial Stretch RE1 to perform tasks in real homes. The abstract provides no quantitative results. For the framework, Stretch is the principal reference point for low-cost mobile manipulation against which the cheaper LeKiwi platform can be positioned, particularly regarding stability and reach trade-offs.

**Fu et al. (2024) (E4; abstract).** Fu, Zhao and Finn [168] extend imitation learning from table-top settings to bimanual mobile manipulation requiring whole-body control. They build Mobile ALOHA, a low-cost whole-body teleoperation system that augments ALOHA (Zhao 2023) with a mobile base and whole-body interface, and use it to collect demonstrations. Policies are trained by supervised behaviour cloning, and co-training with existing static ALOHA datasets boosts performance. With 50 demonstrations per task, co-training increases success rates by up to 90%, enabling autonomous completion of tasks such as sautéing and serving shrimp, storing heavy pots in a two-door cabinet, calling and entering a lift, and rinsing a pan. Limitations include task-specific policies and reliance on substantial teleoperation effort and GPU-class compute. For the framework, Mobile ALOHA is the nearest mobile-manipulation analogue to LeKiwi and a reference for how co-training can raise imitation performance on a low-cost mobile platform.

**Cadene et al. (2024) (S; full text).** Cadene et al. [169] release LeRobot, an open-source PyTorch library from Hugging Face that provides models, datasets and tools for real-world robotics with the stated aim of lowering the barrier to entry. According to the repository, it implements imitation learning policies (e.g. ACT and Diffusion Policy, after Zhao 2023 and Chi 2023, and VQ-BeT), reinforcement learning methods (HIL-SERL and TDMPC) and vision-language-action models including  $\pi 0$  variants (Black 2024) and SmolVLA (Shukor 2025), among others. It natively supports low-cost hardware including the SO-100 arm family, LeKiwi and Koch, with a plugin mechanism for other robots. Data are stored in the LeRobotDataset format, with synchronised MP4 videos or images for vision and Parquet files for state and action, hosted on the Hugging Face Hub, and standard simulation benchmarks such as LIBERO and MetaWorld are supported. The library is Apache-2.0 licensed. As a rapidly evolving repository, its contents change frequently, so any description is tied to a particular release, and reproducibility requires pinning versions. For the framework, LeRobot is the software substrate: it supplies the LeKiwi and SO-101 drivers, the dataset format for logging demonstrations and orchestrator decisions, and reference imitation, RL and VLA implementations that can serve as the modality modules the orchestrator invokes.

**Collaboration (2024) (E5; abstract).** The Open X-Embodiment Collaboration [170] asks whether robotics can follow NLP and vision towards generalist pretrained backbones that adapt efficiently to new robots, tasks and environments. They assemble a dataset in standardised formats from 22 different robots, contributed by 21 institutions, covering 527 skills (160,266 tasks), and train high-capacity models, termed RT-X, on this data. RT-X exhibits positive transfer, improving the capabilities of multiple robots by leveraging experience from other platforms. The abstract gives no per-robot figures, and the models are large transformers trained on accelerators. For the framework, OXE is the canonical source of cross-embodiment pre-training data and a reference point for whether a generalist policy, rather than an orchestrated set of modalities, suffices for a low-cost platform.

**Kress-Gazit et al. (2024) (E4; abstract).** Kress-Gazit et al. [171] argue that robot learning should be treated as an empirical science with more rigorous evaluation practice. They observe that the dominant metric, especially in physical experiments, is success rate, often reported with little or no information about the number of runs, initial conditions or success criteria, scant description of failures and little statistical analysis. To move the field forward, they propose best practices for evaluating and comparing learned policies on physical robots: explicitly reporting experimental conditions, measuring several metrics designed to complement success rate, conducting statistical analysis of the findings, and adding qualitative descriptions of failure modes. These practices are illustrated through an evaluation on physical robots of several learned policies for manipulation tasks. The paper is a methodological position with an illustrative case study rather than a new algorithm, and the abstract gives no quantitative results. Its recommendations raise evaluation cost for small laboratories. For the framework, this paper sets the evaluation standard that the orchestration experiments should meet: success rates on the LeKiwi platform should be accompanied by trial counts, initial-condition protocols, complementary metrics such as completion time and intervention counts, statistical tests and failure-mode descriptions, particularly when claiming that adaptive modality selection outperforms fixed-modality baselines.

**Snyder et al. (2025) (E4; abstract).** Snyder et al. [172] address the statistical comparison of imitation learning policies when the feasible number of evaluation trials is small (e.g. 10 or 50), owing to human effort and limited policy inference throughput. Prior work relies on batch testing with a fixed, predetermined number of trials; adding trials after inspecting results risks inadvertent p-hacking. The authors propose a sequential statistical test for comparing two policies that lets researchers decide whether to run more trials based on intermediate results while preserving probabilistic correctness, adapting the number of trials to the difficulty of the comparison. Extensive numerical simulation and real-world robot manipulation experiments show that the test achieves near-optimal stopping, reducing the number of evaluation trials by up to 32% relative to state-of-the-art baselines while preserving correctness and statistical power; the method is strongest for the hardest comparisons, and in a multi-task comparison it saves more than 160 simulation rollouts. Limitations are that it addresses pairwise comparison of binary success outcomes rather than richer metrics, and that assumptions such as independence between trials must hold. For the framework, this test is directly applicable to comparing the adaptive orchestrator against fixed-modality baselines on real hardware, where each trial is costly: it allows rigorous claims with fewer real-robot runs and guards against the optional-stopping errors common in small-sample robot evaluations.

**Team et al. (2025) (E5; abstract).** The TRI LBM Team [173] rigorously evaluates multitask 'large behaviour models' that extend Diffusion Policy (Chi 2023) and are trained on a mixture of simulated and real-world manipulation data. They propose a statistical evaluation pipeline that compares multitask policies against single-task baselines through blind, randomised trials in simulation and on real dexterous manipulation tasks. The findings are that multitask pretraining makes policies more successful and robust, enables new tasks to be learned faster with less data, and that performance increases predictably with the scale and diversity of pretraining. The abstract gives no specific figures. The models require substantial GPU compute. For the framework, this study is an evaluation exemplar and evidence that pretraining helps the imitation modality, while its blind, randomised protocol complements Kress-Gazit 2024 and Snyder 2025.

**Cadene et al. (2026) (S; full text).** Cadene et al. [174] present lerobot as a research contribution, arguing that robot learning is slowed by fragmented, closed-source tools addressing only parts of the robotics stack. The library integrates the whole pipeline, from low-level middleware for motor control through large-scale dataset collection, storage and streaming, to training and deploying policies. According to the paper, the ecosystem hosted over 16,000 datasets from more than 2,200 contributors as of September 2025. The paper profiles policies on an RTX 4090: ACT (52M parameters) uses about 211 MB peak memory with roughly 5 ms inference latency, Diffusion Policy about 614 ms, and  $\pi 0$  (3.5B parameters) about 13.3 GB and 209 ms. An asynchronous inference scheme achieved 73.3% average success versus 78.3% for synchronous inference on real tasks while roughly doubling throughput (19 versus 9 cubes in 60 s). Profiling is on a desktop GPU rather than embedded hardware. For the framework, the paper provides the citable description of the software stack and the most relevant published latency and memory figures for candidate modality policies; the gap between ACT and diffusion or VLA policies is exactly the cost structure the orchestrator must reason about, and asynchronous inference is a mechanism for hiding the latency of expensive modalities on a Pi-hosted robot.

## Safe learning

**García and Fernández (2015) (R; abstract).** García and Fernández [175] survey safe reinforcement learning, defined as learning policies that maximise expected return where reasonable system performance or safety constraints must be respected during learning and/or deployment. They classify approaches into two families: those that modify the optimality criterion, adding a safety factor to the discounted return (e.g. worst-case, risk-sensitive or constrained criteria), and those that modify exploration by incorporating external knowledge or guidance from a risk metric. They use this taxonomy to organise the literature and suggest future directions. For the framework, the taxonomy frames how the RL modality can be constrained, and suggests risk metrics as signals for deferring to safer modalities.

**Alshiekh et al. (2018) (E2; abstract).** Alshiekh et al. [176] address the absence of safety guarantees in reinforcement learning during both learning and execution. Given a safety specification in temporal logic and an abstraction of the environment dynamics, they synthesise a reactive system called a shield that enforces the specification. Two placements are considered. In preemptive shielding, the shield acts before the agent decides, providing at each step the list of safe actions from which the learner may choose. In post-posed shielding, the shield sits after the agent, monitors its chosen actions and corrects them only when they would violate the specification. The authors discuss the conditions under which a shield preserves the convergence guarantees of the learning algorithm, and they demonstrate the approach on several reinforcement learning scenarios. The abstract reports no quantitative results. Limitations are the need for a formal specification and a sufficiently faithful finite abstraction of the environment, which becomes difficult for high-dimensional, continuous robot dynamics. For the framework, shielding offers a clean architectural pattern for the fault adaptation and recovery modality: a lightweight, verified monitor that sits beside a learned policy and minimally overrides unsafe actions. The post-posed variant in particular corresponds to an orchestrator-level safety layer that can veto or correct the output of whichever modality is active.

**Phan et al. (2020) (E3; abstract).** Phan et al. [177] present the Neural Simplex Architecture (NSA), a runtime-assurance approach that provides safety guarantees for neural controllers (e.g. RL-trained) in autonomous and other complex systems without unduly sacrificing performance. NSA builds on the classical Simplex architecture, in which a decision module switches control from a high-performance but unverified controller to a verified baseline controller when safety is threatened. NSA extends this in two ways. First, it allows control to be switched back to the advanced neural controller once the system has returned to a safe region, rather than remaining on the conservative controller. Second, it corrects unsafe behaviour of the neural controller through online retraining, using experience gathered when the baseline controller intervened. The approach is demonstrated on case studies including a target-seeking ground rover navigating an obstacle field and a neural controller for an artificial pancreas system; no figures in the abstract. Limitations include reliance on a verified baseline controller and a verifiable safety region, which are hard to obtain for contact-rich manipulation. For the framework, NSA is a close conceptual template for the fault adaptation and recovery modality: switching between a learned policy and a safe fallback based on a monitor, returning control when safe, and using interventions as training data parallels the orchestrator’s online improvement through replay and fine-tuning.

**Brunke et al. (2022) (R; abstract).** Brunke et al. [178] review safe learning methods for real-world robotic deployment, drawing together contributions from the control and reinforcement learning communities with an emphasis on unifying their language and frameworks for decision making under uncertainty. The review covers three strands: learning-based control approaches that safely improve performance by learning uncertain dynamics; reinforcement learning approaches that encourage safety or robustness; and methods that formally certify the safety of a learned control policy, such as safety filters. The authors discuss when and how data-driven methods should be leveraged in safety-critical settings, such as operating near humans, identify open challenges, and call for realistic physics-based benchmarks to enable fair comparison between control and RL approaches. As a review, it offers no new empirical results. For the framework, the review provides the conceptual map for positioning the fault adaptation and recovery modality: it clarifies the distinction between encouraging safety through the learning objective and certifying it externally, and it motivates treating a certified filter as a component orthogonal to modality selection. Its emphasis on uncertainty-aware learned dynamics also supports using model uncertainty as one of the orchestrator’s confidence inputs.

**Wabersich et al. (2023) (R; full text).** Wabersich et al. [179] provide a tutorial overview of data-driven safety filters: modular components that augment an existing, possibly learned, controller with safety guarantees by intercepting potentially unsafe inputs and minimally modifying them. They cover three classes: Hamilton-Jacobi reachability, which computes the maximal control-invariant set offline but scales poorly with dimension; control barrier functions, which enforce invariance through derivative conditions solved online by convex optimisation but require careful barrier synthesis; and predictive safety filters, which use receding-horizon optimisation to certify that the system can be steered to a terminal safe set. Each is discussed for uncertain systems, and all three are compared on an inverted pendulum

design example. For the framework, the article offers practical options for a certified recovery layer, with control barrier functions the most compatible with Pi-class compute.

**Hsu et al. (2024) (R; abstract).** Hsu, Hu and Fisac [180] review safety filter approaches for safety-critical control in autonomous systems. They argue that traditional model-based safe control struggles with generalisability and scalability, while emerging data-driven approaches tend to lack well-understood guarantees and can fail unpredictably, so the next generation of autonomous robots must integrate the strengths of both. The article proposes a unified technical framework for understanding, comparing and combining safety filters, exposing a shared modular structure across seemingly disparate classes, such as filters based on Hamilton-Jacobi reachability, control barrier functions and predictive or rollout-based methods. In this view, a filter comprises a monitor that assesses whether the current or proposed action preserves safety, and an intervention mechanism that modifies the action when needed. The framework suggests directions for more scalable synthesis, robust monitoring and efficient intervention. As a review, it contributes no new empirical results. For the framework, the monitor–intervention decomposition maps directly onto the orchestration architecture: a safety monitor can supply one of the calibrated confidence signals the orchestrator fuses, while the intervention step corresponds to invoking the fault adaptation and recovery modality. The review also clarifies that monitoring and intervention can be designed and verified separately from the learned policies, which is valuable when those policies are continually updated through LoRA fine-tuning and replay.

## References

- [1] M. Ahn, A. Brohan, N. Brown *et al.*, “Do as i can, not as i say: grounding language in robotic affordances,” in *Conference on Robot Learning (CoRL 2022)*, 2022.
- [2] W. Huang, F. Xia, T. Xiao *et al.*, “Inner monologue: embodied reasoning through planning with language models,” in *Conference on Robot Learning (CoRL 2022)*, 2022.
- [3] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: extracting actionable knowledge for embodied agents,” in *Int. Conf. on Machine Learning (ICML 2022)*, 2022.
- [4] D. Shah, B. Osinski, B. Ichter, and S. Levine, “Lm-nav: robotic navigation with large pre-trained models of language, vision, and action,” in *Conference on Robot Learning (CoRL 2022)*, 2022.
- [5] J. Liang, W. Huang, F. Xia *et al.*, “Code as policies: language model programs for embodied control,” in *IEEE Int. Conf. on Robotics and Automation (ICRA 2023)*, 2023.
- [6] D. Driess, F. Xia, M. Sajjadi *et al.*, “Palm-e: an embodied multimodal language model,” in *Int. Conf. on Machine Learning (ICML 2023)*, 2023.
- [7] S. Yao, J. Zhao, D. Yu *et al.*, “React: synergizing reasoning and acting in language models,” in *Int. Conf. on Learning Representations (ICLR 2023)*, 2023.
- [8] T. Schick, J. Dwivedi-Yu, R. Dessì *et al.*, “Toolformer: language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [10] I. Singh, V. Blukis, A. Mousavian *et al.*, “Progprompt: generating situated robot task plans using large language models,” in *IEEE Int. Conf. on Robotics and Automation (ICRA 2023)*, 2023.
- [11] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “Voxposer: composable 3d value maps for robotic manipulation with language models,” in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [12] W. Yu, N. Gileadi, C. Fu *et al.*, “Language to rewards for robotic skill synthesis,” in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [13] C. Song, J. Wu, C. Washington, B. Sadler, W.-L. Chao, and Y. Su, “Llm-planner: few-shot grounded planning for embodied agents with large language models,” in *IEEE/CVF Int. Conf. on Computer Vision (ICCV 2023)*, 2023.
- [14] S. Yao, D. Yu, J. Zhao *et al.*, “Tree of thoughts: deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [15] A. Ren, A. Dixit, A. Bodrova *et al.*, “Robots that ask for help: uncertainty alignment for large language model planners,” in *Conference on Robot Learning (CoRL 2023)*, 2023.

- [16] B. Chen, C. Shu, E. Shareghi, N. Collier, K. Narasimhan, and S. Yao, “Fireact: toward language agent fine-tuning,” 2023.
- [17] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: an open-ended embodied agent with large language models,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [18] K. Liang, Z. Zhang, and J. Fisac, “Introspective planning: aligning robots’ uncertainty with inherent task ambiguity,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [19] J. Wang, G. He, and Y. Kantaros, “Probabilistically correct language-based multi-robot planning using conformal prediction,” 2024, later IEEE RA-L.
- [20] M. Dalal, T. Chiruvolu, D. Chaplot, and R. Salakhutdinov, “Plan-seq-learn: language model guided rl for solving long horizon robotics tasks,” in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [21] A. Zeng, M. Liu, R. Lu *et al.*, “Agenttuning: enabling generalized agent abilities for llms,” in *Findings of ACL 2024*, 2024.
- [22] W. Yao, S. Heinecke, J. Niebles *et al.*, “Retroformer: retrospective large language agents with policy gradient optimization,” in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [23] Y. Zhai, H. Bai, Z. Lin *et al.*, “Fine-tuning large vision-language models as decision-making agents via reinforcement learning,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [24] A. Radford, J. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Int. Conf. on Machine Learning (ICML 2021)*, 2021.
- [25] M. Shridhar, L. Manuelli, and D. Fox, “Cliport: what and where pathways for robotic manipulation,” in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [26] S. Reed, K. Zolna, E. Parisotto *et al.*, “A generalist agent,” *Transactions on Machine Learning Research (TMLR)*, 2022.
- [27] M. Shridhar, L. Manuelli, and D. Fox, “Perceiver-actor: a multi-task transformer for robotic manipulation,” in *Conference on Robot Learning (CoRL 2022)*, 2022.
- [28] A. Brohan, N. Brown, J. Carbajal *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems (RSS 2023)*, 2023.
- [29] —, “Rt-2: vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [30] Y. Jiang, A. Gupta, Z. Zhang *et al.*, “Vima: general robot manipulation with multimodal prompts,” in *Int. Conf. on Machine Learning (ICML 2023)*, 2023.
- [31] M. Kim, K. Pertsch, S. Karamcheti *et al.*, “Openvla: an open-source vision-language-action model,” in *Conference on Robot Learning (CoRL 2024)*, 2024.
- [32] D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari *et al.*, “Octo: an open-source generalist robot policy,” in *Robotics: Science and Systems (RSS 2024)*, 2024.
- [33] K. Black, N. Brown, D. Driess *et al.*, “ $\pi_0$ : a vision-language-action flow model for general robot control,” in *Robotics: Science and Systems XXI (RSS 2025)*, 2024.
- [34] K. Bousmalis, G. Vezzani, D. Rao *et al.*, “Robocat: a self-improving generalist agent for robotic manipulation,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [35] X. Li, M. Liu, H. Zhang *et al.*, “Vision-language foundation models as effective robot imitators,” in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [36] J. Liu, M. Liu, Z. Wang *et al.*, “Robomamba: efficient vision-language-action model for robotic reasoning and manipulation,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [37] M. Shukor, D. Aubakirova, F. Capuano *et al.*, “Smolvla: a vision-language-action model for affordable and efficient robotics,” 2025, hugging Face.
- [38] J. Wen, Y. Zhu, J. Li *et al.*, “Tinyvla: towards fast, data-efficient vision-language-action models for robotic manipulation,” *IEEE Robotics and Automation Letters* (2025), 2025.

- [39] P. Budzianowski, W. Maa, M. Freed *et al.*, “Edgevla: efficient vision-language-action models,” 2025, iROS 2024 MoMa workshop.
- [40] M. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: optimizing speed and success,” in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [41] K. Pertsch, K. Stachowicz, B. Ichter *et al.*, “Fast: efficient action tokenization for vision-language-action models,” in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [42] H. Wang, C. Xiong, R. Wang, and X. Chen, “Bitvla: 1-bit vision-language-action models for robotics manipulation,” 2025.
- [43] Z. Ravichandran, I. Hounie, F. Cladera *et al.*, “Distilling on-device language models for robot planning with minimal human intervention,” 2025.
- [44] Z. Yu, B. Wang, P. Zeng *et al.*, “A survey on efficient vision-language-action models,” 2025.
- [45] J. Zhang, Y. Hsieh, Z. Wan *et al.*, “Quantvla: scale-calibrated post-training quantization for vision-language-action models,” in *IEEE/CVF CVPR 2026*, 2026.
- [46] Y. Gal and Z. Ghahramani, “Dropout as a bayesian approximation: representing model uncertainty in deep learning,” in *Int. Conf. on Machine Learning (ICML 2016)*, 2016.
- [47] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [48] A. Kendall and Y. Gal, “What uncertainties do we need in bayesian deep learning for computer vision?” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [49] C. Guo, G. Pleiss, Y. Sun, and K. Weinberger, “On calibration of modern neural networks,” in *Int. Conf. on Machine Learning (ICML 2017)*, PMLR 70, 2017, pp. 1321–1330.
- [50] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Int. Conf. on Learning Representations (ICLR 2017)*, 2017.
- [51] M. Sensoy, L. Kaplan, and M. Kandemir, “Evidential deep learning to quantify classification uncertainty,” in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018.
- [52] A. Amini, W. Schwarting, A. Soleimany, and D. Rus, “Deep evidential regression,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [53] J. Gawlikowski, C. Tassi, M. Ali *et al.*, “A survey of uncertainty in deep neural networks,” *Artificial Intelligence Review*, 56(Suppl 1), 1513–1589, 2023.
- [54] A. Ren, J. Clark, A. Dixit *et al.*, “Explore until confident: efficient exploration for embodied question answering,” in *Robotics: Science and Systems (RSS 2024)*, 2024.
- [55] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [56] I. Osband, C. Blundell, A. Pritzel, and B. Van Roy, “Deep exploration via bootstrapped dqn,” in *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, 2016.
- [57] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017.
- [58] M. Andrychowicz, F. Wolski, A. Ray *et al.*, “Hindsight experience replay,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [59] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Int. Conf. on Machine Learning (ICML 2018)*, 2018.
- [60] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Int. Conf. on Machine Learning (ICML 2018)*, 2018.
- [61] D. Kalashnikov, A. Irpan, P. Pastor *et al.*, “Qt-opt: scalable deep reinforcement learning for vision-based robotic manipulation,” in *Conference on Robot Learning (CoRL 2018)*, 2018.
- [62] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, “Exploration by random network distillation,” in *Int. Conf. on Learning Representations (ICLR 2019)*, 2019.

- [63] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative q-learning for offline reinforcement learning,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [64] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse control tasks through world models,” *Nature*, vol. 640, no. 8059, pp. 647–653, 2025.
- [65] X. Yuan, T. Mu, S. Tao *et al.*, “Policy decorator: model-agnostic online refinement for large policy model,” in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [66] L. Ankile, A. Simeonov, I. Shenfeld *et al.*, “From imitation to refinement – residual rl for precise assembly,” in *IEEE ICRA 2025*, 2025.
- [67] A. Wagenmaker, M. Nakamoto, Y. Zhang *et al.*, “Steering your diffusion policy with latent space reinforcement learning,” 2025.
- [68] M. Sobol Mark, T. Gao, G. Sampaio *et al.*, “Policy agnostic rl: offline rl and online rl fine-tuning of any class and backbone,” in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [69] Y. Chen, S. Tian, S. Liu *et al.*, “Conrft: a reinforced fine-tuning method for vla models via consistency policy,” in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [70] G. Lu, W. Guo, C. Zhang *et al.*, “Vla-rl: towards masterful and general robotic manipulation with scalable reinforcement learning,” 2025.
- [71] H. Li, Y. Zuo, J. Yu *et al.*, “Simplevla-rl: scaling vla training via reinforcement learning,” in *Int. Conf. on Learning Representations (ICLR 2026)*, 2026.
- [72] S. Ross, G. Gordon, and J. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Int. Conf. on Artificial Intelligence and Statistics (AISTATS 2011)*, 2011.
- [73] J. Ho and S. Ermon, “Generative adversarial imitation learning,” in *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, 2016.
- [74] Y. Duan, M. Andrychowicz, B. Stadie *et al.*, “One-shot imitation learning,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [75] T. Osa, J. Pajarinen, G. Neumann, J. Bagnell, P. Abbeel, and J. Peters, “An algorithmic perspective on imitation learning,” *Foundations and Trends in Robotics*, vol. 7, no. 1–2, pp. 1–179, 2018.
- [76] C. Lynch, M. Khansari, T. Xiao *et al.*, “Learning latent plans from play,” in *Conference on Robot Learning (CoRL 2019)*, 2019.
- [77] P. Florence, C. Lynch, A. Zeng *et al.*, “Implicit behavioral cloning,” in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [78] E. Jang, A. Irpan, M. Khansari *et al.*, “Bc-z: zero-shot task generalization with robotic imitation learning,” in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [79] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS 2023)*, 2023.
- [80] T. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems (RSS 2023)*, 2023.
- [81] B. Wu, B. Lee, K. Daniilidis, B. Bucher, and N. Matni, “Uncertainty-aware deployment of pre-trained language-conditioned imitation learning policies,” 2024.
- [82] J. Bongard, V. Zykov, and H. Lipson, “Resilient machines through continuous self-modeling,” *Science*, vol. 314, no. 5802, pp. 1118–1121, 2006.
- [83] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, “Robots that can adapt like animals,” *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.
- [84] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS 2017)*, 2017.
- [85] E. Khalastchi and M. Kalech, “On fault detection and diagnosis in robotic systems,” *ACM Computing Surveys*, vol. 51, no. 1, p. 9, 2018.

- [86] A. Nagabandi, I. Clavera, S. Liu *et al.*, “Learning to adapt in dynamic, real-world environments through meta-reinforcement learning,” in *Int. Conf. on Learning Representations (ICLR 2019)*, 2019.
- [87] G. Pang, C. Shen, L. Cao, and A. van den Hengel, “Deep learning for anomaly detection: a review,” *ACM Computing Surveys*, vol. 54, no. 2, p. 38, 2021.
- [88] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “Rma: rapid motor adaptation for legged robots,” in *Robotics: Science and Systems (RSS 2021)*, 2021.
- [89] Z. Liu, A. Bahety, and S. Song, “Reflect: summarizing robot experiences for failure explanation and correction,” in *Conference on Robot Learning (CoRL 2023)*, 2023.
- [90] C. Agia, R. Sinha, J. Yang *et al.*, “Unpacking failure modes of generative policies: runtime monitoring of consistency and progress,” in *Conference on Robot Learning (CoRL 2024)*, PMLR 270, 2024.
- [91] R. Sinha, A. Elhafsi, C. Agia *et al.*, “Real-time anomaly detection and reactive planning with large language models,” in *Robotics: Science and Systems (RSS 2024)*, 2024.
- [92] L. Shi, Z. Hu, T. Zhao *et al.*, “Yell at your robot: improving on-the-fly from language corrections,” in *Robotics: Science and Systems (RSS 2024)*, 2024.
- [93] J. Duan, W. Pumacay, N. Kumar *et al.*, “Aha: a vision-language-model for detecting and reasoning over failures in robotic manipulation,” in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [94] Z. Ye, W. Lu, M. Ye, T. Lin, S. Yang, J. Yan, and B. Zhao, “Robofac: a comprehensive framework for robotic failure analysis and correction,” 2025.
- [95] R. Römer, A. Kobras, L. Worbis, and A. Schoellig, “Failure prediction at runtime for generative robot policies,” in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.
- [96] C. Xu, T. Nguyen, E. Dixon *et al.*, “Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies,” in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [97] Q. Gu, Y. Ju, S. Sun *et al.*, “Safe: multitask failure detection for vision-language-action models,” in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.
- [98] Y. Duan, J. Schulman, X. Chen, P. Bartlett, I. Sutskever, and P. Abbeel, “ $RL^2$ : fast reinforcement learning via slow reinforcement learning,” 2016.
- [99] M. Andrychowicz, M. Denil, S. Gomez *et al.*, “Learning to learn by gradient descent by gradient descent,” in *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, 2016.
- [100] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *Int. Conf. on Machine Learning (ICML 2017)*, 2017.
- [101] J. Snell, K. Swersky, and R. Zemel, “Prototypical networks for few-shot learning,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [102] A. Nichol, J. Achiam, and J. Schulman, “On first-order meta-learning algorithms,” 2018.
- [103] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-world: a benchmark and evaluation for multi-task and meta reinforcement learning,” in *Conference on Robot Learning (CoRL 2019)*, 2019.
- [104] K. Rakelly, A. Zhou, D. Quillen, C. Finn, and S. Levine, “Efficient off-policy meta-reinforcement learning via probabilistic context variables,” in *Int. Conf. on Machine Learning (ICML 2019)*, 2019.
- [105] L. Zintgraf, K. Shiarlis, M. Igl *et al.*, “Varibad: a very good method for bayes-adaptive deep rl via meta-learning,” in *Int. Conf. on Learning Representations (ICLR 2020)*, 2020.
- [106] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey, “Meta-learning in neural networks: a survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5149–5169, 2022.
- [107] R. Zhu, E. Sun, G. Huang, and O. Celiktutan, “Efficient continual adaptation of pretrained robotic policy with online meta-learned adapters,” 2025.
- [108] H. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Int. Conf. on Artificial Intelligence and Statistics (AISTATS 2017)*, 2017.

- [109] A. Kendall, Y. Gal, and R. Cipolla, “Multi-task learning using uncertainty to weigh losses for scene geometry and semantics,” in *IEEE/CVF CVPR 2018*, 2018, pp. 7482–7491.
- [110] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “Multimodal machine learning: a survey and taxonomy,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, 2019.
- [111] T. Li, A. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proc. Machine Learning and Systems (MLSys 2020)*, 2020.
- [112] P. Kairouz, H. McMahan, B. Avent *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [113] J. Qi, Q. Zhou, L. Lei, and K. Zheng, “Federated reinforcement learning: techniques, applications, and open challenges,” *Intelligence & Robotics*, vol. 1, no. 1, pp. 18–57, 2021.
- [114] Z. Han, C. Zhang, H. Fu, and J. Zhou, “Trusted multi-view classification,” in *Int. Conf. on Learning Representations (ICLR 2021)*, 2021.
- [115] —, “Trusted multi-view classification with dynamic evidential fusion,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- [116] Z. Han, F. Yang, J. Huang, C. Zhang, and J. Yao, “Multimodal dynamics: dynamical fusion for trustworthy multimodal classification,” in *IEEE/CVF CVPR 2022*, 2022, pp. 20 707–20 717.
- [117] L. Lindemann, M. Cleaveland, G. Shim, and G. Pappas, “Safe planning in dynamic environments using conformal prediction,” *IEEE Robotics and Automation Letters*, 2023.
- [118] T. Zollo and R. Zemel, “Confidence calibration in vision-language-action models,” 2025.
- [119] R. Römer, Seeliger, et al. (8 authors, last Schoellig, and A.P.), “Uncertainty quantification for flow-based vision-language-action models,” 2026.
- [120] J. Yuan, Y. Wu, and A. Bajcsy, “When to act, ask, or learn: uncertainty-aware policy steering,” in *Robotics: Science and Systems (RSS 2026)*, 2026.
- [121] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *Int. Conf. on Learning Representations (ICLR 2016)*, 2016.
- [122] A. Rusu, N. Rabinowitz, G. Desjardins *et al.*, “Progressive neural networks,” 2016.
- [123] J. Kirkpatrick, R. Pascanu, N. Rabinowitz *et al.*, “Overcoming catastrophic forgetting in neural networks,” in *Proceedings of the National Academy of Sciences*, 114(13), 2017, pp. 3521–3526.
- [124] D. Lopez-Paz and M. Ranzato, “Gradient episodic memory for continual learning,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [125] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. Lampert, “icarl: incremental classifier and representation learning,” in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR 2017)*, 2017.
- [126] N. Houlsby, A. Giurgiu, S. Jastrzebski *et al.*, “Parameter-efficient transfer learning for nlp,” in *Int. Conf. on Machine Learning (ICML 2019)*, 2019.
- [127] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, “Experience replay for continual learning,” in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.
- [128] H. Cai, C. Gan, L. Zhu, and S. Han, “Tinytl: reduce memory, not parameters for efficient on-device learning,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [129] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, “McuNet: tiny deep learning on IoT devices,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [130] X. Li and P. Liang, “Prefix-tuning: optimizing continuous prompts for generation,” in *Proc. ACL-IJCNLP 2021*, 2021.
- [131] E. Hu, Y. Shen, P. Wallis *et al.*, “Lora: low-rank adaptation of large language models,” in *Int. Conf. on Learning Representations (ICLR 2022)*, 2022.
- [132] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, “On-device training under 256kb memory,” in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

- [133] H. Liu, D. Tam, M. Muqeeth *et al.*, “Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning,” in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- [134] M. De Lange, R. Aljundi, M. Masana *et al.*, “A continual learning survey: defying forgetting in classification tasks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3366–3385, 2022.
- [135] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: efficient finetuning of quantized llms,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [136] H. Liu, S. Nasiriany, L. Zhang *et al.*, “Robot learning on the job: human-in-the-loop autonomy and learning during deployment,” in *Robotics: Science and Systems (RSS 2023); IJRR 2025*, 2023.
- [137] B. Liu, Y. Zhu, C. Gao *et al.*, “Libero: benchmarking knowledge transfer for lifelong robot learning,” in *NeurIPS 2023 Datasets and Benchmarks*, 2023.
- [138] L. Zhu, L. Hu, J. Lin *et al.*, “Pockengine: sparse and efficient fine-tuning in a pocket,” in *IEEE/ACM MICRO 2023*, 2023.
- [139] S. Malladi, T. Gao, E. Nichani *et al.*, “Fine-tuning language models with just forward passes,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [140] Z. Liu, J. Zhang, K. Asadi *et al.*, “Tail: task-specific adapters for imitation learning with large pretrained models,” in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [141] W. Wan, Y. Zhu, R. Shah, and Y. Zhu, “Lotus: continual imitation learning for robot manipulation through unsupervised skill discovery,” in *IEEE ICRA 2024*, 2024.
- [142] L. Li, X. Yang, W. Wu, H. Wang, T. Ohtsuki, X. Fu, M. Pan, and X. Shen, “Mobillm: enabling llm fine-tuning on the mobile device via server assisted side tuning,” 2025.
- [143] R. Römer, Y. Zhang, and A. Schoellig, “Clare: continual learning for vision-language-action models via autonomous adapter routing and expansion,” 2026.
- [144] R. Jacobs, M. Jordan, S. Nowlan, and G. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [145] E. Bengio, P.-L. Bacon, J. Pineau, and D. Precup, “Conditional computation in neural networks for faster models,” 2015, iCLR 2016 Workshop.
- [146] A. Graves, “Adaptive computation time for recurrent neural networks,” 2016.
- [147] N. Shazeer, A. Mirhoseini, K. Maziarz *et al.*, “Outrageously large neural networks: the sparsely-gated mixture-of-experts layer,” in *Int. Conf. on Learning Representations (ICLR 2017)*, 2017.
- [148] C. Rosenbaum, T. Klinger, and M. Riemer, “Routing networks: adaptive selection of non-linear functions for multi-task learning,” in *Int. Conf. on Learning Representations (ICLR 2018)*, 2018.
- [149] C. Riquelme, J. Puigcerver, B. Mustafa *et al.*, “Scaling vision with sparse mixture of experts,” in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.
- [150] R. Hoque, A. Balakrishna, E. Novoseller *et al.*, “Thriftydagger: budget-aware novelty and risk gating for interactive imitation learning,” in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [151] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [152] J. Puigcerver, C. Riquelme, B. Mustafa, and N. Houlsby, “From sparse to soft mixtures of experts,” in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.
- [153] Y. Wang, Y. Zhang, M. Huo, R. Tian, X. Zhang, Y. Xie *et al.*, “Sparse diffusion policy: a sparse, reusable, and flexible policy for robot learning,” in *Proc. 8th Conference on Robot Learning (CoRL 2024)*, PMLR 270, 2024, pp. 649–665.
- [154] L. Chen, M. Zaharia, and J. Zou, “Frugalgpt: how to use large language models while reducing cost and improving performance,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [155] D. Ding, A. Mallick, C. Wang *et al.*, “Hybrid llm: cost-efficient and quality-aware query routing,” in *Int. Conf. on Learning Representations (ICLR 2024)*, 2024.

- [156] P. Aggarwal, A. Madaan, A. Anand *et al.*, “Automix: automatically mixing language models,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [157] Y. Yue, Y. Wang, B. Kang *et al.*, “Deer-vla: dynamic inference of multimodal large language models for efficient robot execution,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- [158] M. Reuss, J. Pari, P. Agrawal, and R. Lioutikov, “Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning,” in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [159] R. Huang, S. Zhu, Y. Du, and H. Zhao, “Moe-loco: mixture of experts for multitask locomotion,” in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS 2025)*, 2025, pp. 14 218–14 225.
- [160] I. Ong, A. Almahairi, V. Wu *et al.*, “Routellm: learning to route llms with preference data,” in *Int. Conf. on Learning Representations (ICLR 2025)*, 2025.
- [161] C. Hao, X. Zhai, Y. Liu, and H. Soh, “Abstracting robot manipulation skills via mixture-of-experts diffusion policies,” 2026.
- [162] J. Liu, P. Zhao, Z. Kong *et al.*, “When should a robot think? resource-aware reasoning via reinforcement learning for embodied robotic decision-making,” 2026.
- [163] R. Zhang, M. Dong, Y. Zhang *et al.*, “Mole-vla: dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation,” in *Proc. AAAI Conference on Artificial Intelligence (AAAI 2026)*, 2026.
- [164] S. James, Z. Ma, D. Arrojo, and A. Davison, “Rlbench: the robot learning benchmark & learning environment,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [165] A. Mandlekar, D. Xu, J. Wong *et al.*, “What matters in learning from offline human demonstrations for robot manipulation,” in *Conference on Robot Learning (CoRL 2021)*, 2021.
- [166] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “Calvin: a benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [167] C. Kemp, A. Edsinger, H. Clever, and B. Matulevich, “The design of stretch: a compact, lightweight mobile manipulator for indoor human environments,” in *IEEE ICRA 2022*, 2022.
- [168] Z. Fu, T. Zhao, and C. Finn, “Mobile aloha: learning bimanual mobile manipulation with low-cost whole-body teleoperation,” in *Conference on Robot Learning (CoRL 2024)*, 2024.
- [169] R. Cadene, S. Alibert, A. Soare *et al.*, “Lerobot: state-of-the-art machine learning for real-world robotics in pytorch,” *GitHub repository, Hugging Face*, 2024. [Online]. Available: <https://github.com/huggingface/lerobot>
- [170] O. X.-E. Collaboration, “Open x-embodiment: robotic learning datasets and rt-x models,” in *IEEE Int. Conf. on Robotics and Automation (ICRA 2024)*, 2024.
- [171] H. Kress-Gazit, K. Hashimoto, N. Kuppuswamy *et al.*, “Robot learning as an empirical science: best practices for policy evaluation,” 2024.
- [172] D. Snyder, A. Hancock, A. Badithela *et al.*, “Is your imitation learning policy better than mine? policy comparison with near-optimal stopping,” in *Robotics: Science and Systems (RSS 2025)*, 2025.
- [173] TRI LBM Team, J. Barreiros, A. Beaulieu *et al.*, “A careful examination of large behavior models for multitask dexterous manipulation,” *Science Robotics*, 2025.
- [174] R. Cadene, S. Alibert, F. Capuano *et al.*, “Lerobot: an open-source library for end-to-end robot learning,” in *Int. Conf. on Learning Representations (ICLR 2026)*, 2026.
- [175] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, pp. 1437–1480, 2015.
- [176] M. Alshiekh, R. Bloem, R. Ehlers *et al.*, “Safe reinforcement learning via shielding,” in *Proc. AAAI Conference on Artificial Intelligence (AAAI 2018)*, 2018.
- [177] D. Phan, R. Grosu, N. Jansen *et al.*, “Neural simplex architecture,” in *NASA Formal Methods (NFM 2020)*, 2020.
- [178] L. Brunke, M. Greeff, A. Hall *et al.*, “Safe learning in robotics: from learning-based control to safe reinforcement learning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.

- [179] K. Wabersich, A. Taylor, J. Choi, K. Sreenath, C. Tomlin, A. Ames, and M. Zeilinger, “Data-driven safety filters: Hamilton-jacobi reachability, control barrier functions, and predictive methods for uncertain systems,” *IEEE Control Systems Magazine*, 2023.
- [180] K.-C. Hsu, H. Hu, and J. Fisac, “The safety filter: a unified view of safety-critical control in autonomous systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, 2024.